

(12) **United States Patent**
Mertes et al.

(10) **Patent No.:** **US 12,399,640 B2**
(45) **Date of Patent:** **Aug. 26, 2025**

(54) **MIGRATING SIMILAR DATA TO A SINGLE DATA REDUCTION POOL**

(56) **References Cited**
U.S. PATENT DOCUMENTS

(71) Applicant: **PURE STORAGE, INC.**, Mountain View, CA (US)
(72) Inventors: **Lawrence Mertes**, San Jose, CA (US); **Marco Sanvido**, Belmont, CA (US)
(73) Assignee: **PURE STORAGE, INC.**, Santa Clara, CA (US)

5,706,210	A	1/1998	Kumano et al.
5,799,200	A	8/1998	Brant et al.
5,933,598	A	8/1999	Scales et al.
6,012,032	A	1/2000	Donovan et al.
6,085,333	A	7/2000	Dekoning et al.
6,643,641	B1	11/2003	Snyder
6,647,514	B1	11/2003	Umberger et al.
6,789,162	B1	9/2004	Talagala et al.

(Continued)

(*) Notice: Subject to any disclaimer, the term of this patent is extended or adjusted under 35 U.S.C. 154(b) by 57 days.

FOREIGN PATENT DOCUMENTS

EP	0725324	A2	8/1996
WO	2012087648	A2	6/2012

(Continued)

(21) Appl. No.: **18/146,899**

(22) Filed: **Dec. 27, 2022**

OTHER PUBLICATIONS

(65) **Prior Publication Data**
US 2023/0138462 A1 May 4, 2023

Bellamy-McIntyre et al., “OpenID and the Enterprise: A Model-based Analysis of Single Sign-On Authentication”, 15th IEEE International Enterprise Distributed Object Computing Conference (EDOC), Aug. 29, 2011, pp. 129-138, IEEE Computer Society, USA, DOI: 10.1109/EDOC.2011.26, ISBN: 978-1-4577-0362-1.
(Continued)

Related U.S. Application Data

(63) Continuation of application No. 15/642,305, filed on Jul. 5, 2017, now Pat. No. 11,561,714.

Primary Examiner — Chie Yew

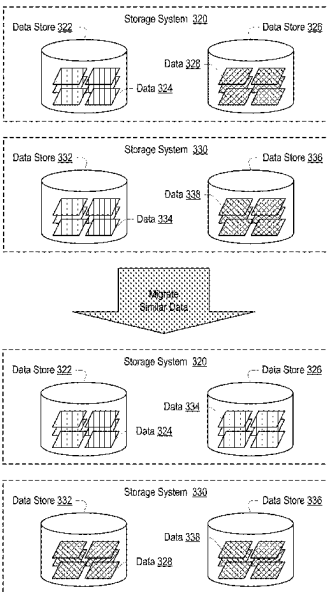
(51) **Int. Cl.**
G06F 3/06 (2006.01)
(52) **U.S. Cl.**
CPC **G06F 3/0647** (2013.01); **G06F 3/0608** (2013.01); **G06F 3/0641** (2013.01); **G06F 3/0683** (2013.01)

(57) **ABSTRACT**

Migrating similar data to a single data reduction pool, including: determining that storage space consumption may be reduced by migrating similar data between a first storage system and a second storage system; and initiating a migration of the similar data, including selecting a migration direction from one of either migrating the first data from the first storage system to the second storage system and migrating the second data from the second storage system to the first storage system.

(58) **Field of Classification Search**
CPC G06F 3/0647–065; G06F 3/0608; G06F 3/0641; G06F 3/0683
See application file for complete search history.

20 Claims, 23 Drawing Sheets



(56)

References Cited

U.S. PATENT DOCUMENTS

6,895,485	B1	5/2005	DeKoning et al.	2004/0193814	A1	9/2004	Erickson et al.
7,089,272	B1	8/2006	Garthwaite et al.	2004/0260967	A1	12/2004	Guha et al.
7,107,389	B2	9/2006	Inagaki et al.	2005/0160416	A1	7/2005	Jamison et al.
7,146,521	B1	12/2006	Nguyen	2005/0188246	A1	8/2005	Emberly et al.
7,334,124	B2	2/2008	Pham et al.	2005/0216800	A1	9/2005	Bicknell et al.
7,437,530	B1	10/2008	Rajan	2006/0015771	A1	1/2006	Van Gundy et al.
7,484,056	B2	1/2009	Madnani et al.	2006/0129817	A1	6/2006	Borneman et al.
7,484,057	B1	1/2009	Madnani et al.	2006/0161726	A1	7/2006	Lasser
7,493,424	B1	2/2009	Bali et al.	2006/0206675	A1	9/2006	Sato et al.
7,669,029	B1	2/2010	Mishra et al.	2006/0230245	A1	10/2006	Gounares et al.
7,689,609	B2	3/2010	Lango et al.	2006/0236056	A1	10/2006	Nagata
7,743,191	B1	6/2010	Liao	2006/0239075	A1	10/2006	Williams et al.
7,757,059	B1	7/2010	Ofer et al.	2007/0022227	A1	1/2007	Miki
7,899,780	B1	3/2011	Shmuylovich et al.	2007/0028068	A1	2/2007	Golding et al.
7,975,115	B2	7/2011	Wayda et al.	2007/0055702	A1	3/2007	Fridella et al.
8,042,163	B1	10/2011	Karr et al.	2007/0109856	A1	5/2007	Pellicone et al.
8,086,585	B1	12/2011	Brashers et al.	2007/0150689	A1	6/2007	Pandit et al.
8,200,887	B2	6/2012	Bennett	2007/0168321	A1	7/2007	Saito et al.
8,271,700	B1	9/2012	Annem et al.	2007/0220227	A1	9/2007	Long
8,387,136	B2	2/2013	Lee et al.	2007/0294563	A1	12/2007	Bose
8,402,250	B1	3/2013	Juillard	2007/0294564	A1	12/2007	Reddin et al.
8,437,189	B1	5/2013	Montierth et al.	2008/0005587	A1	1/2008	Ahlquist
8,465,332	B2	6/2013	Hogan et al.	2008/0077825	A1	3/2008	Bello et al.
8,484,356	B1	7/2013	Douglis et al.	2008/0126437	A1	5/2008	Chiba
8,504,797	B2	8/2013	Mimatsu	2008/0162674	A1	7/2008	Dahiya
8,527,544	B1	9/2013	Colgrove et al.	2008/0195833	A1	8/2008	Park
8,566,546	B1	10/2013	Marshak et al.	2008/0256141	A1	10/2008	Wayda et al.
8,578,442	B1	11/2013	Banerjee	2008/0270678	A1	10/2008	Cornwell et al.
8,589,504	B1	11/2013	Ofer et al.	2008/0282045	A1	11/2008	Biswas et al.
8,613,066	B1	12/2013	Brezinski et al.	2009/0077340	A1	3/2009	Johnson et al.
8,620,970	B2	12/2013	English et al.	2009/0100115	A1	4/2009	Park et al.
8,633,838	B2	1/2014	Falls et al.	2009/0198889	A1	8/2009	Ito et al.
8,700,634	B2	4/2014	Apte et al.	2010/0052625	A1	3/2010	Cagno et al.
8,751,463	B1	6/2014	Chamness	2010/0211723	A1	8/2010	Mukaida
8,762,642	B2	6/2014	Bates et al.	2010/0246266	A1	9/2010	Park et al.
8,769,622	B2	7/2014	Chang et al.	2010/0257142	A1	10/2010	Murphy et al.
8,800,009	B1	8/2014	Beda et al.	2010/0262764	A1	10/2010	Liu et al.
8,812,860	B1	8/2014	Bray	2010/0306500	A1	12/2010	Mimatsu
8,822,155	B2	9/2014	Sukumar et al.	2010/0325345	A1	12/2010	Ohno et al.
8,850,546	B1	9/2014	Field et al.	2010/0332754	A1	12/2010	Lai et al.
8,898,346	B1	11/2014	Simmons	2011/0035540	A1	2/2011	Fitzgerald et al.
8,909,854	B2	12/2014	Yamagishi et al.	2011/0072290	A1	3/2011	Davis et al.
8,931,041	B1	1/2015	Banerjee	2011/0125955	A1	5/2011	Chen
8,949,863	B1	2/2015	Coatney et al.	2011/0131231	A1	6/2011	Haas et al.
8,984,602	B1	3/2015	Bailey et al.	2011/0167221	A1	7/2011	Pangal et al.
8,990,905	B1	3/2015	Bailey et al.	2012/0023144	A1	1/2012	Rub
9,081,713	B1	7/2015	Bennett	2012/0054264	A1	3/2012	Taugh et al.
9,124,569	B2	9/2015	Hussain et al.	2012/0079318	A1	3/2012	Colgrove et al.
9,134,922	B2	9/2015	Rajagopal et al.	2012/0102280	A1	4/2012	Nasu et al.
9,189,334	B2	11/2015	Bennett	2012/0131253	A1	5/2012	McKnight et al.
9,209,973	B2	12/2015	Aikas et al.	2012/0303919	A1	11/2012	Hu et al.
9,250,823	B1	2/2016	Kamat et al.	2012/0311000	A1	12/2012	Post et al.
9,280,678	B2	3/2016	Redberg	2013/0007845	A1	1/2013	Chang et al.
9,300,660	B1	3/2016	Borowiec et al.	2013/0031414	A1	1/2013	Dhuse et al.
9,311,182	B2	4/2016	Bennett	2013/0036272	A1	2/2013	Nelson
9,395,922	B2	7/2016	Nishikido et al.	2013/0071087	A1	3/2013	Motiwala et al.
9,444,822	B1	9/2016	Borowiec et al.	2013/0145447	A1	6/2013	Maron
9,507,532	B1	11/2016	Colgrove et al.	2013/0185229	A1	7/2013	Naga et al.
9,632,870	B2	4/2017	Bennett	2013/0191555	A1	7/2013	Liu
10,324,639	B2	6/2019	Seo	2013/0198459	A1	8/2013	Joshi et al.
10,567,406	B2	2/2020	Astigarraga et al.	2013/0205173	A1	8/2013	Yoneda
10,846,137	B2	11/2020	Vallala et al.	2013/0219164	A1	8/2013	Hamid
10,877,683	B2	12/2020	Wu et al.	2013/0227201	A1	8/2013	Talagala et al.
11,076,509	B2	7/2021	Alissa et al.	2013/0290607	A1	10/2013	Chang et al.
11,106,810	B2	8/2021	Natanzon et al.	2013/0311434	A1	11/2013	Jones
11,144,445	B1*	10/2021	Yu	2013/0311739	A1	11/2013	Kawakami et al.
11,194,707	B2	12/2021	Stalzer	2013/0318297	A1	11/2013	Jibbe et al.
2002/0013802	A1	1/2002	Mori et al.	2013/0332614	A1	12/2013	Brunk et al.
2002/0199073	A1	12/2002	Tamura et al.	2014/0020083	A1	1/2014	Fetik
2003/0145172	A1	7/2003	Galbraith et al.	2014/0074850	A1	3/2014	Noel et al.
2003/0191783	A1	10/2003	Wolczko et al.	2014/0082715	A1	3/2014	Grajek et al.
2003/0225961	A1	12/2003	Chow et al.	2014/0086146	A1	3/2014	Kim et al.
2004/0080985	A1	4/2004	Chang et al.	2014/0090009	A1	3/2014	Li et al.
2004/0111573	A1	6/2004	Garthwaite	2014/0096220	A1	4/2014	Pinto et al.
2004/0153844	A1	8/2004	Ghose et al.	2014/0101434	A1	4/2014	Senthurpandi et al.
				2014/0164774	A1	6/2014	Nord et al.
				2014/0173232	A1	6/2014	Reohr et al.
				2014/0195636	A1	7/2014	Karve et al.
				2014/0201512	A1	7/2014	Seethaler et al.

(56)

References Cited

U.S. PATENT DOCUMENTS

2014/0201541	A1	7/2014	Paul et al.
2014/0208155	A1	7/2014	Pan
2014/0215590	A1	7/2014	Brand
2014/0220561	A1	8/2014	Sukumar et al.
2014/0229654	A1	8/2014	Goss et al.
2014/0230017	A1	8/2014	Saib
2014/0258526	A1	9/2014	Le Sant et al.
2014/0282983	A1	9/2014	Ju et al.
2014/0285917	A1	9/2014	Cudak et al.
2014/0325262	A1	10/2014	Cooper et al.
2014/0351627	A1	11/2014	Best et al.
2014/0373104	A1	12/2014	Gaddam et al.
2014/0373126	A1	12/2014	Hussain et al.
2015/0026387	A1	1/2015	Sheredy et al.
2015/0074463	A1	3/2015	Jacoby et al.
2015/0089569	A1	3/2015	Sondhi et al.
2015/0095515	A1	4/2015	Krithivas et al.
2015/0113203	A1	4/2015	Dancho et al.
2015/0121137	A1	4/2015	McKnight et al.
2015/0134920	A1	5/2015	Anderson et al.
2015/0149822	A1	5/2015	Coronado et al.
2015/0154418	A1	6/2015	Redberg
2015/0193169	A1	7/2015	Sundaram et al.
2015/0378888	A1	12/2015	Zhang et al.
2016/0026397	A1	1/2016	Nishikido et al.
2016/0098323	A1	4/2016	Mutha et al.
2016/0182542	A1	6/2016	Staniford
2016/0232059	A1	8/2016	Khurange et al.
2016/0248631	A1	8/2016	Duchesneau
2016/0321140	A1	11/2016	Khurange et al.
2016/0342362	A1	11/2016	Vaddagiri et al.
2016/0350009	A1	12/2016	Cerreta et al.
2016/0352720	A1	12/2016	Hu et al.
2016/0352830	A1	12/2016	Borowiec et al.
2016/0352834	A1	12/2016	Borowiec et al.
2017/0262202	A1	9/2017	Seo
2018/0054454	A1	2/2018	Astigarraga et al.
2018/0081562	A1	3/2018	Vasudevan
2019/0220315	A1	7/2019	Vallala et al.
2020/0034560	A1	1/2020	Natanzon et al.
2020/0326871	A1	10/2020	Wu et al.
2021/0360833	A1	11/2021	Alissa et al.

FOREIGN PATENT DOCUMENTS

WO	2013071087	A1	5/2013
WO	2014110137	A1	7/2014
WO	2016015008	A1	1/2016
WO	2016190938	A1	12/2016
WO	2016195759	A1	12/2016
WO	2016195958	A1	12/2016
WO	2016195961	A1	12/2016

OTHER PUBLICATIONS

ETSI, "Network Function Virtualisation (NFV); Resiliency Requirements", ETSI GS NFCV-REL 001, V1.1.1, Jan. 2015, 82 pages, etsi.org (online), URL: www.etsi.org/deliver/etsi_gs/NFV-REL/001_099/001/01.01.01_60/gs_NFV-REL001v010101p.pdf.

Faith, "dictzip file format", GitHub.com (online), accessed Jul. 28, 2015, 1 page, URL: github.com/fidlej/idzip.

Google Search of "storage array define" performed by the Examiner on Nov. 4, 2015 for U.S. Appl. No. 14/725,278, Results limited to entries dated before 2012, 1 page.

Hota et al., "Capability-based Cryptographic Data Access Control in Cloud Computing", International Journal of Advanced Networking and Applications, col. 1, Issue 1, Aug. 2011, 10 pages, Eswar Publications, India.

Hu et al., "Container Marking: Combining Data Placement, Garbage Collection and Wear Levelling for Flash", 19th Annual IEEE International Symposium on Modelling, Analysis, and Simulation of Computer and Telecommunications Systems, Jul. 25-27, 2011, 11

pages, ISBN: 978-0-7695-4430-4, DOI: 10.1109/MASCOTS.2011.50.

Hwang et al., "RAID-x: A New Distributed Disk Array for I/O-Centric Cluster Computing", Proceedings of the Ninth International Symposium on High-performance Distributed Computing, Aug. 2000, pp. 279-286, the Ninth International Symposium on High-Performance Distributed Computing, IEEE Computer Society, Los Alamitos, CA.

International Search Report and Written Opinion, PCT/US2016/015006, Apr. 29, 2016, 12 pages.

International Search Report and Written Opinion, PCT/US2016/015008, May 4, 2016, 12 pages.

International Search Report and Written Opinion, PCT/US2016/016333, Jun. 8, 2016, 12 pages.

International Search Report and Written Opinion, PCT/US2016/020410, Jul. 8, 2016, 12 pages.

International Search Report and Written Opinion, PCT/US2016/032052, Aug. 30, 2016, 17 pages.

International Search Report and Written Opinion, PCT/US2016/032084, Jul. 18, 2016, 12 pages.

International Search Report and Written Opinion, PCT/US2016/035492, Aug. 17, 2016, 10 pages.

International Search Report and Written Opinion, PCT/US2016/036693, Aug. 29, 2016, 10 pages.

International Search Report and Written Opinion, PCT/US2016/038758, Oct. 7, 2016, 10 pages.

International Search Report and Written Opinion, PCT/US2016/040393, Sep. 22, 2016, 10 pages.

International Search Report and Written Opinion, PCT/US2016/044020, Sep. 30, 2016, 11 pages.

International Search Report and Written Opinion, PCT/US2016/044874, Oct. 7, 2016, 11 pages.

International Search Report and Written Opinion, PCT/US2016/044875, Oct. 5, 2016, 13 pages.

International Search Report and Written Opinion, PCT/US2016/044876, Oct. 21, 2016, 12 pages.

International Search Report and Written Opinion, PCT/US2016/044877, Sep. 29, 2016, 13 pages.

Kong, "Using PCI Express as the Primary System Interconnect in Multiroot Compute, Storage, Communications and Embedded Systems", White Paper, IDT.com (online), Aug. 28, 2008, 12 pages, URL: www.idt.com/document/whp/dt-pcie-multi-root-white-paper.

Li et al., "Access Control for the Services Oriented Architecture", Proceedings of the 2007 ACM Workshop on Secure Web Services (SWS '07), Nov. 2007, pp. 9-17, ACM New York, NY.

Matchcentre, "Manipulating Inequalities", Article [online], Apr. 2003, 2 pages, [retrieved on Feb. 7, 2019], Retrieved from the Internet: URL: http://www.mash.dept.shef.ac.uk/Resources/manipulating_inequalities.pdf.

Microsoft, "Hybrid for SharePoint Server 2013—Security Reference Architecture", Microsoft (online), Oct. 2014, 53 pages, URL: hybrid.office.com/img/Security_Reference_Architecture.pdf.

Microsoft, "Hybrid Identity Management", Microsoft (online), Apr. 2014, 2 pages, URL: download.microsoft.com/download/E/A/E/EAE57CD1-A80B-423C-96BB-142FAAC630B9/Hybrid_Identity_Datasheet.pdf.

Microsoft, "Hybrid Identity", Microsoft (online), Apr. 2014, 36 pages, URL: www.aka.ms/HybridIdentityWp.

Pcmag, "Storage Array Definition", Published May 10, 2013, URL: <http://web.archive.org/web/20130510121646/http://www.pcmag.com/encyclopedia/term/52091/storage-array>, 2 pages.

Stalzer, "FlashBlades: System Architecture and Applications", Proceedings of the 2nd Workshop on Architectures and Systems for Big Data, Jun. 2012, pp. 10-14, Association for Computing Machinery, New York, NY.

Storer et al., "Pergamum: Replacing Tape with Energy Efficient, Reliable, Disk-Based Archival Storage", FAST'08: Proceedings of the 6th USENIX Conference on File and Storage Technologies, Article No. 1, Feb. 2008, pp. 1-16, USENIX Association, Berkeley, CA.

(56)

References Cited

OTHER PUBLICATIONS

Storer et al., "Secure Data Deduplication", Proceedings of the 4th ACM International Workshop on Storage Security and Survivability (StorageSS'08), Oct. 2008, 10 pages, ACM New York, NY, USA, DOI: 10.1145/1456469.1456471.

Sweere, "Creating Storage Class Persistent Memory with NVDIMM", Published in Aug. 2013, Flash Memory Summit 2013, URL: http://www.flashmemorysummit.com/English/Collaterals/Proceedings/2013/20130814_T2_Sweere.pdf, 22 pages.

Techopedia, "What is a disk array", techopedia.com (online), Jan. 13, 2012, 1 page, URL: web.archive.org/web/20120113053358/http://www.techopedia.com/definition/1009/disk-array.

Webopedia, "What is a disk array", webopedia.com (online), May 26, 2011, 2 pages, URL: web.archive.org/web/20110526081214/http://www.webopedia.com/TERM/D/disk_array.html.

Wikipedia, "Convergent Encryption", Wikipedia.org (online), accessed Sep. 8, 2015, 2 pages, URL: en.wikipedia.org/wiki/Convergent_encryption.

* cited by examiner

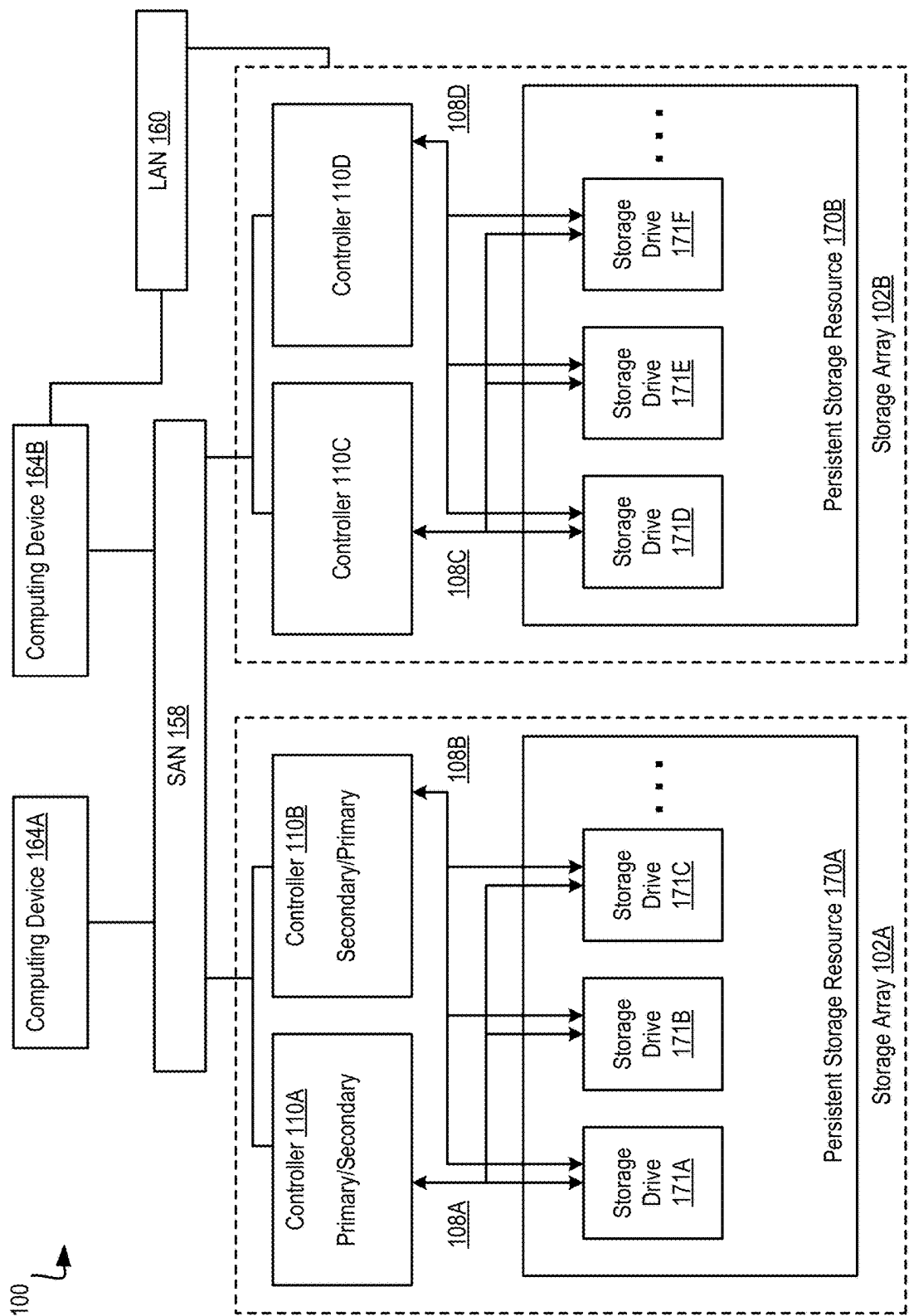


FIG. 1A

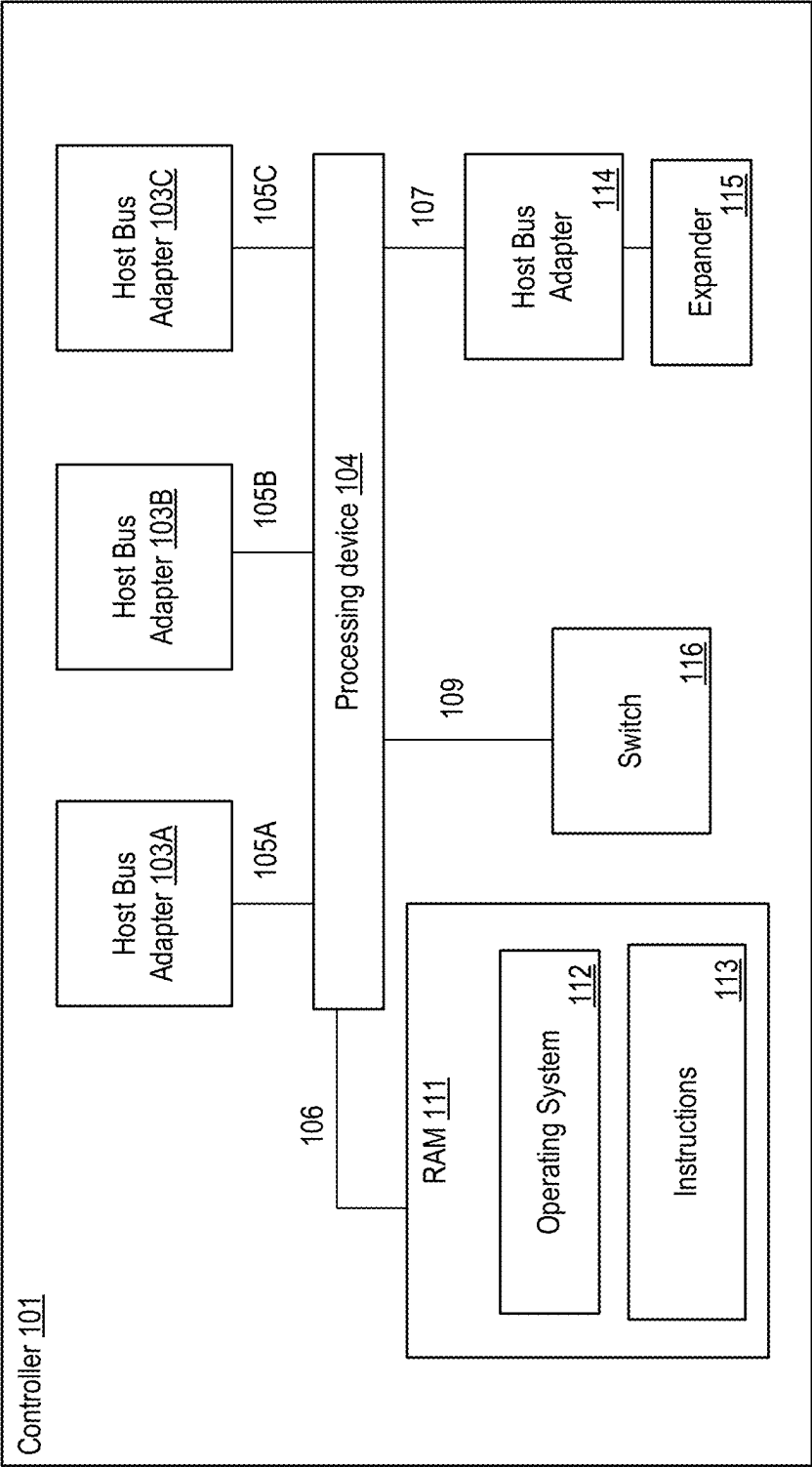


FIG. 1B

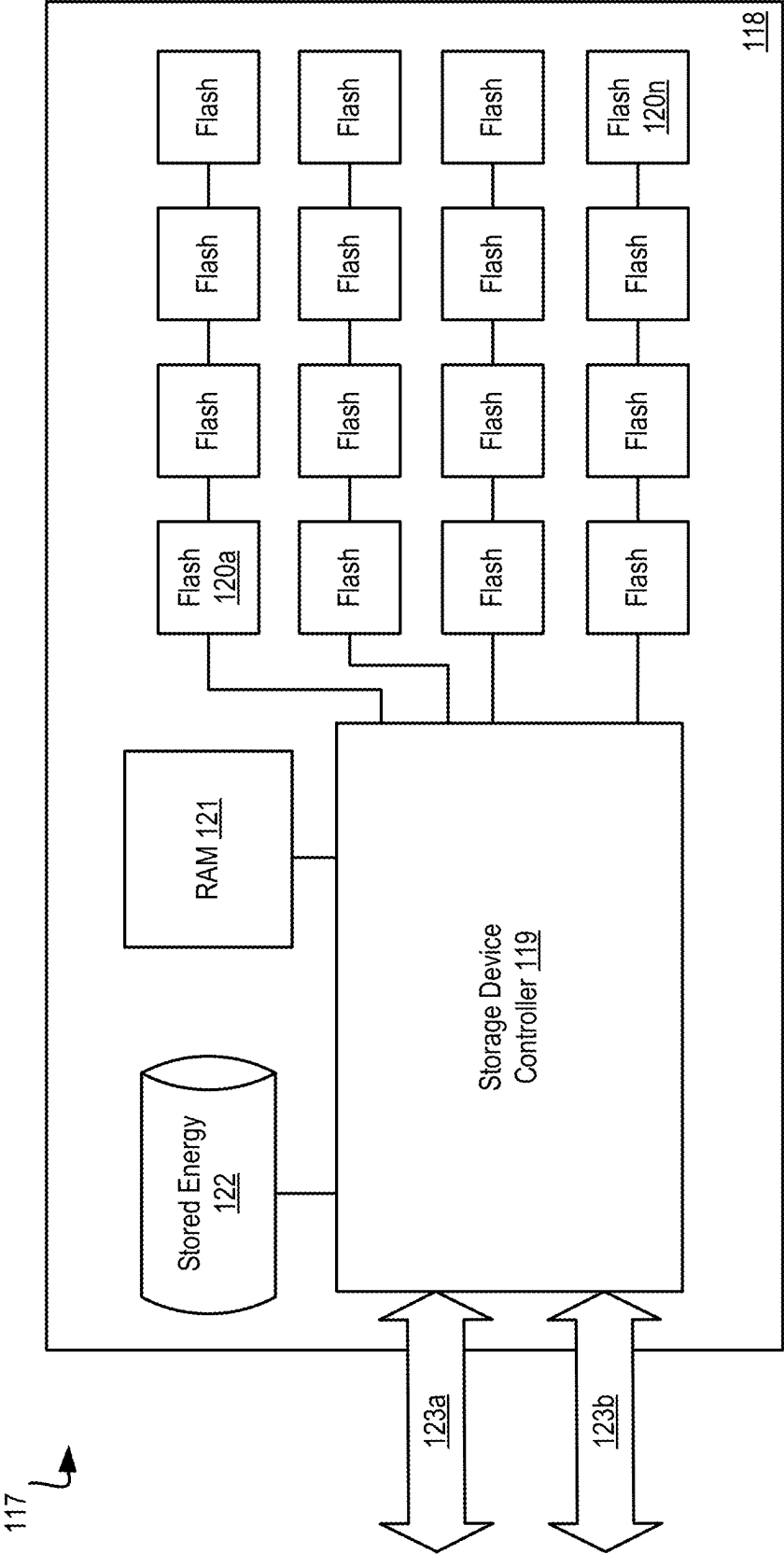
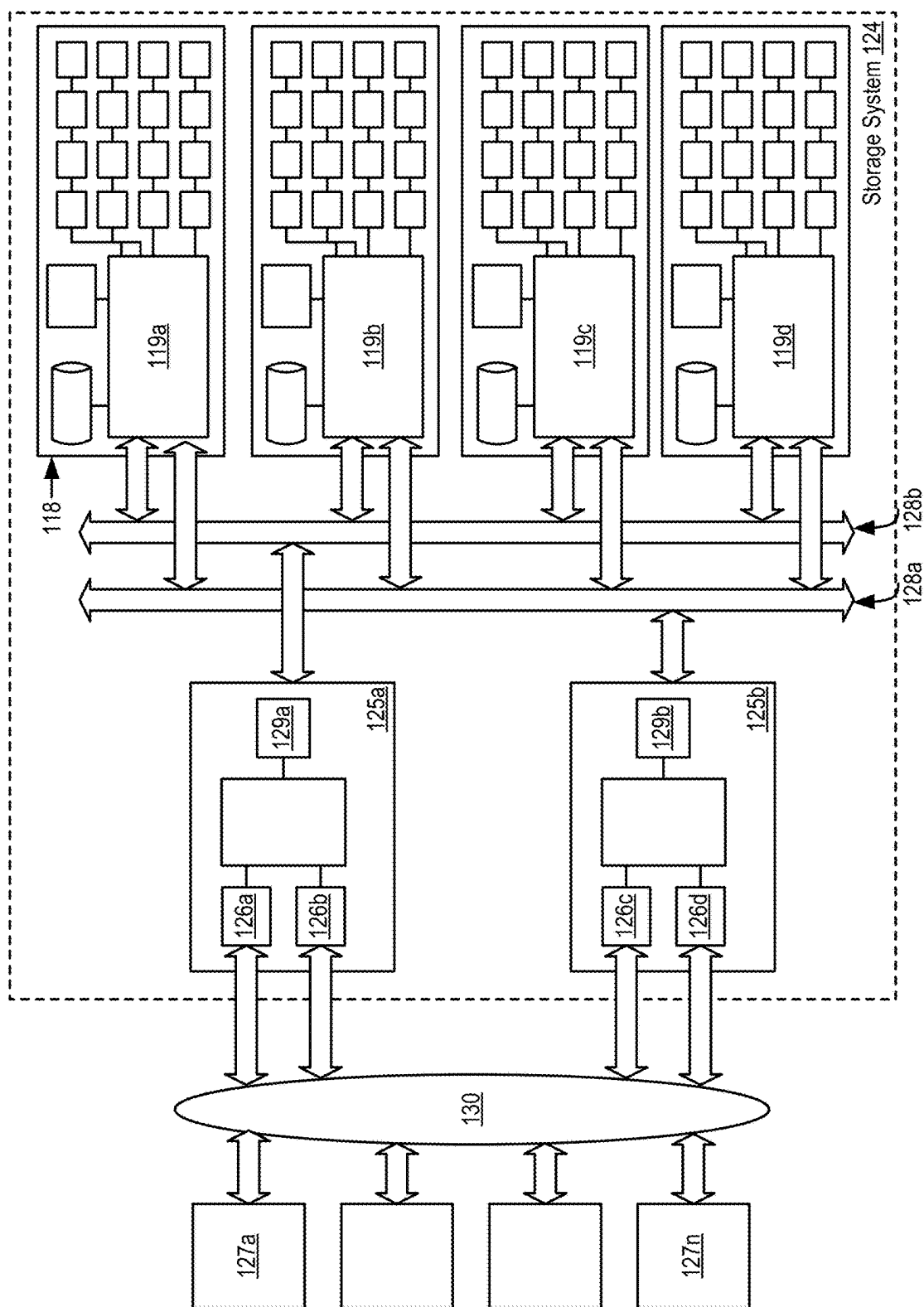


FIG. 1C



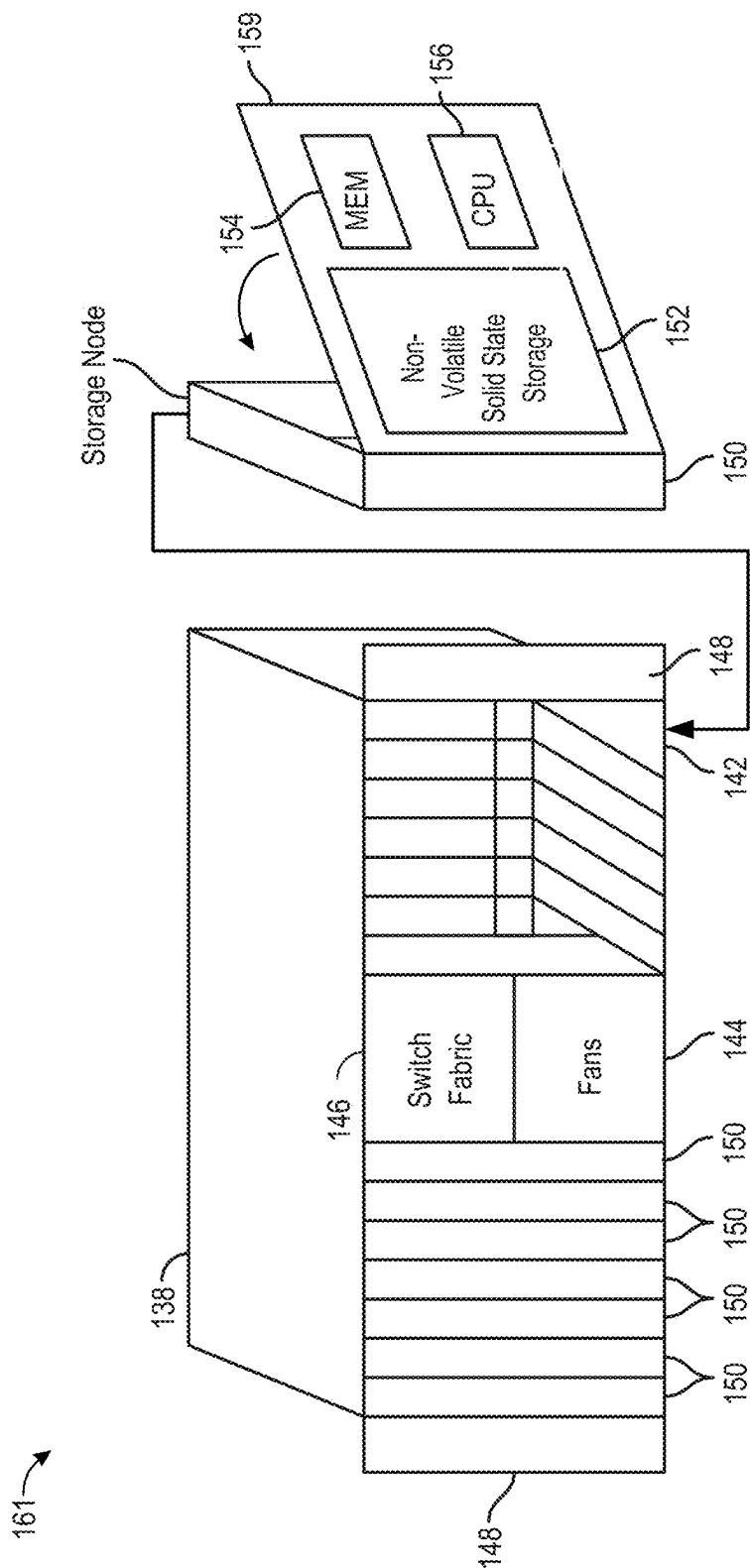


FIG. 2A

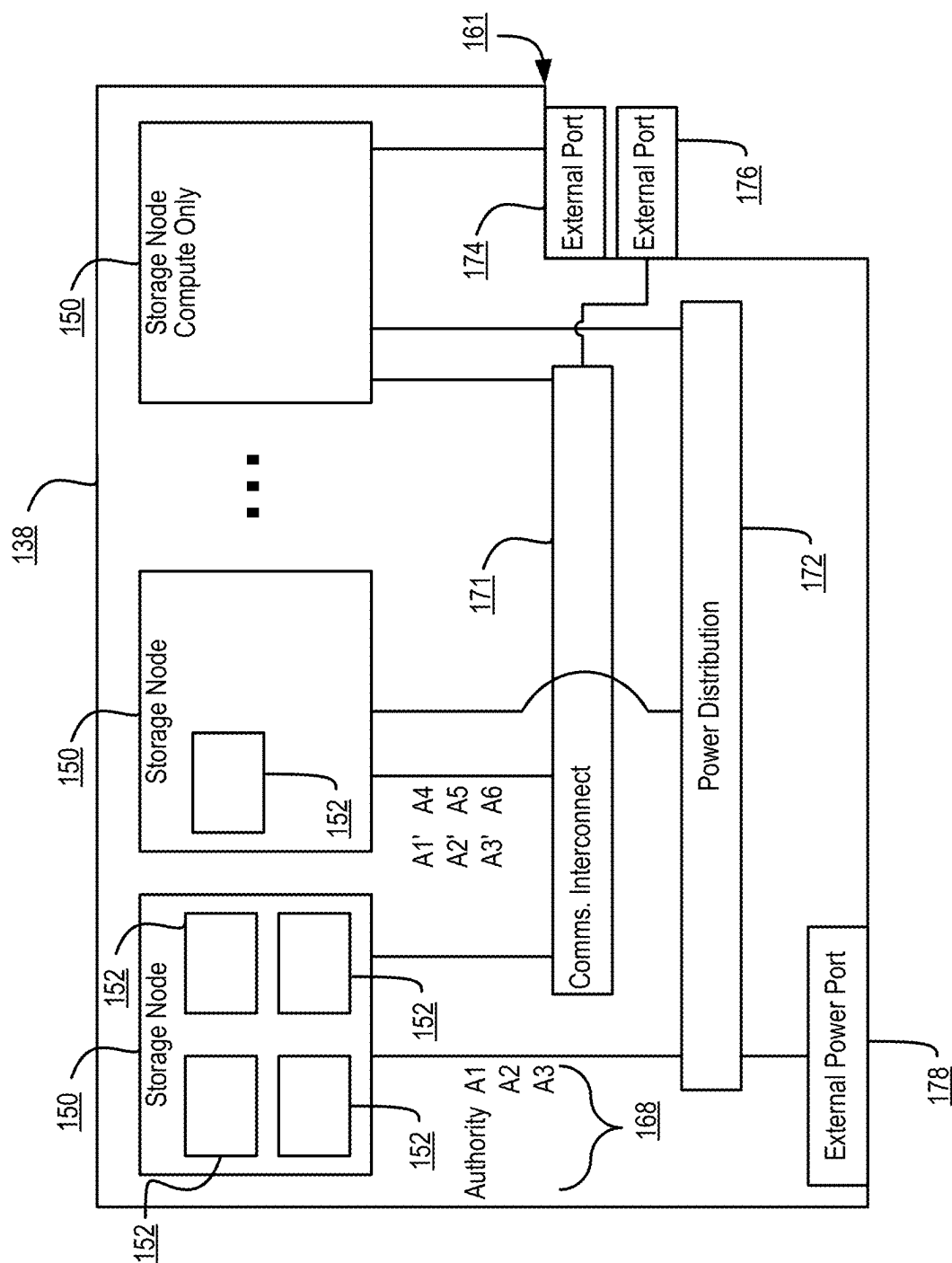


FIG. 2B

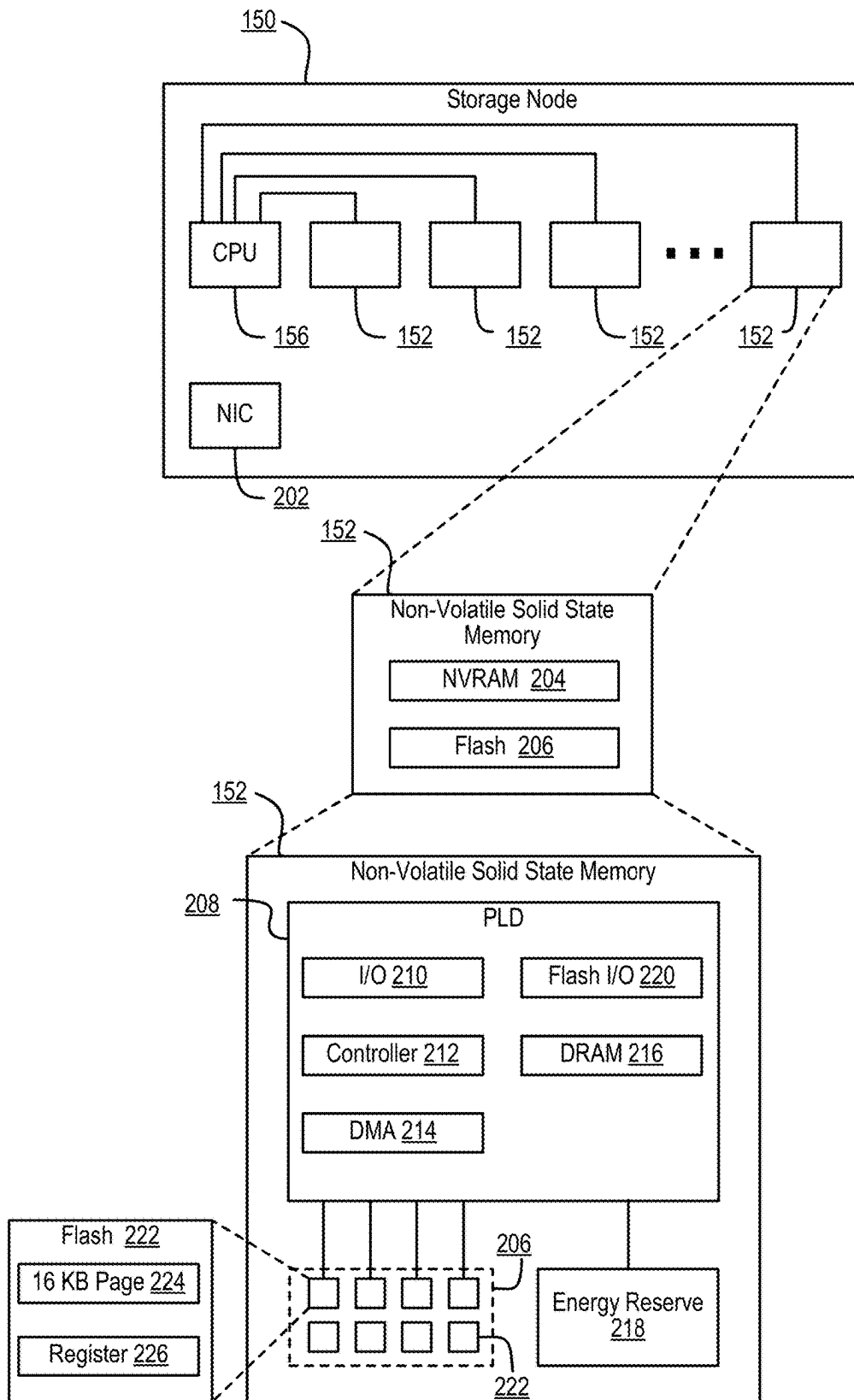


FIG. 2C

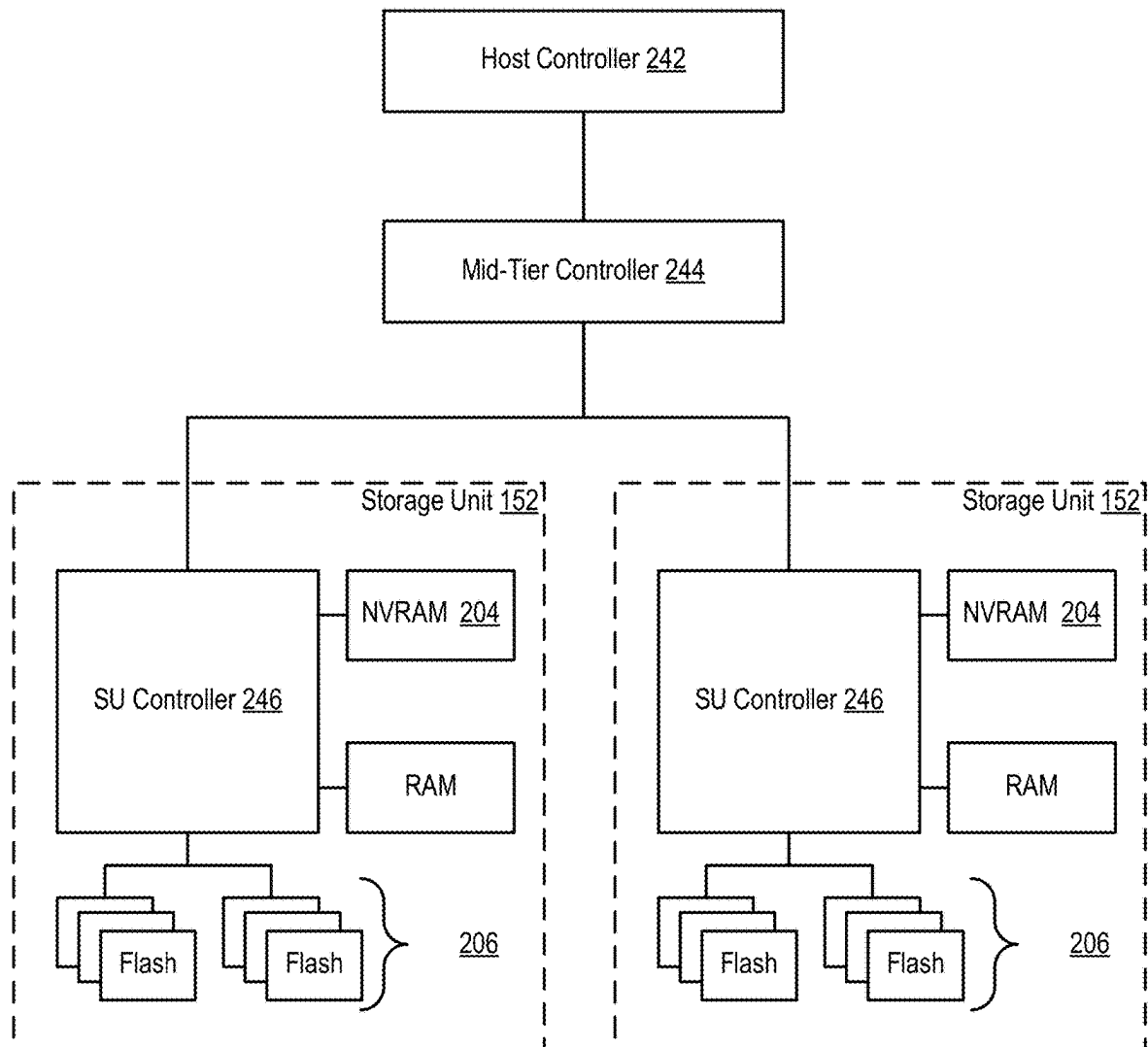
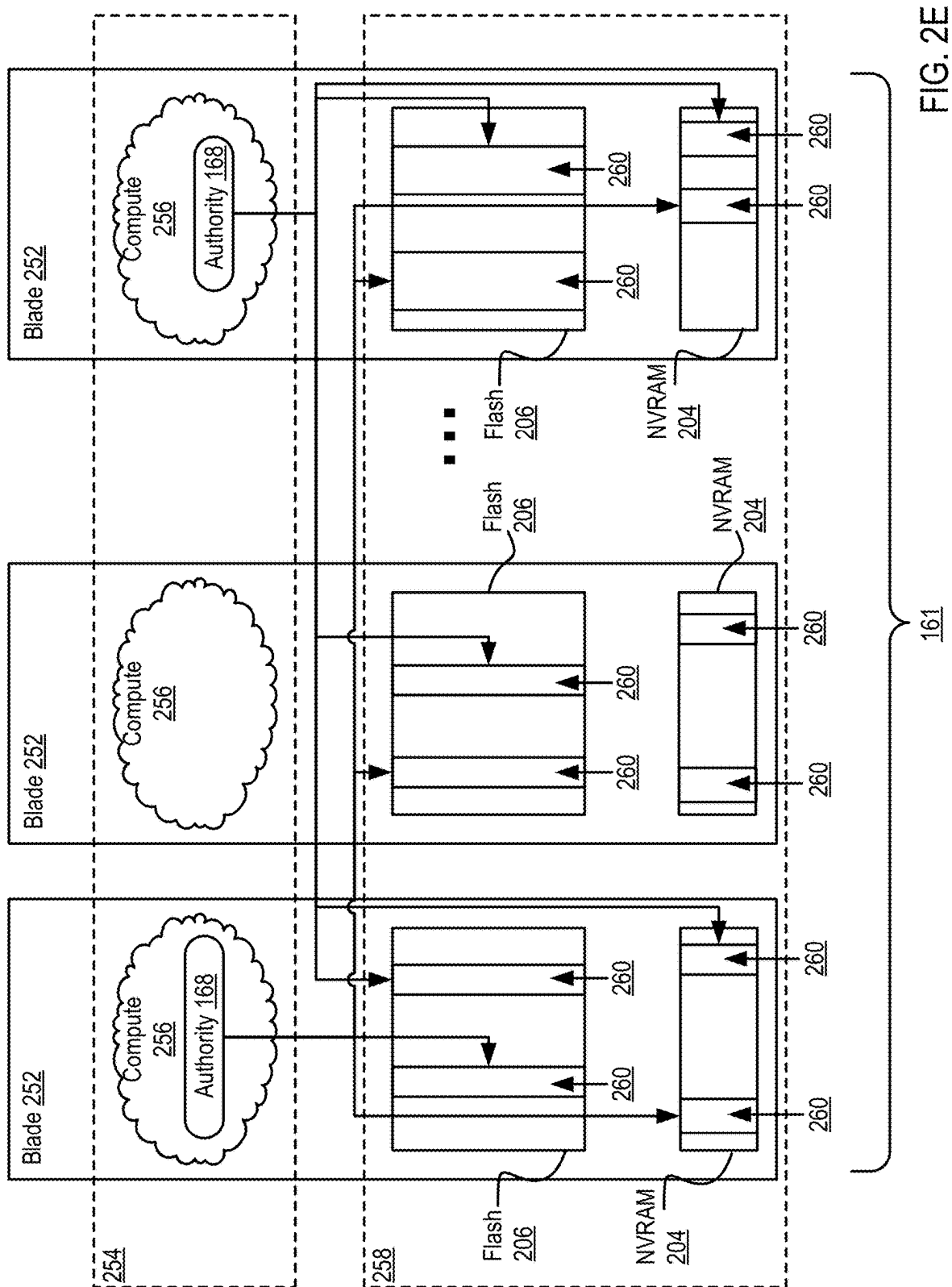


FIG. 2D



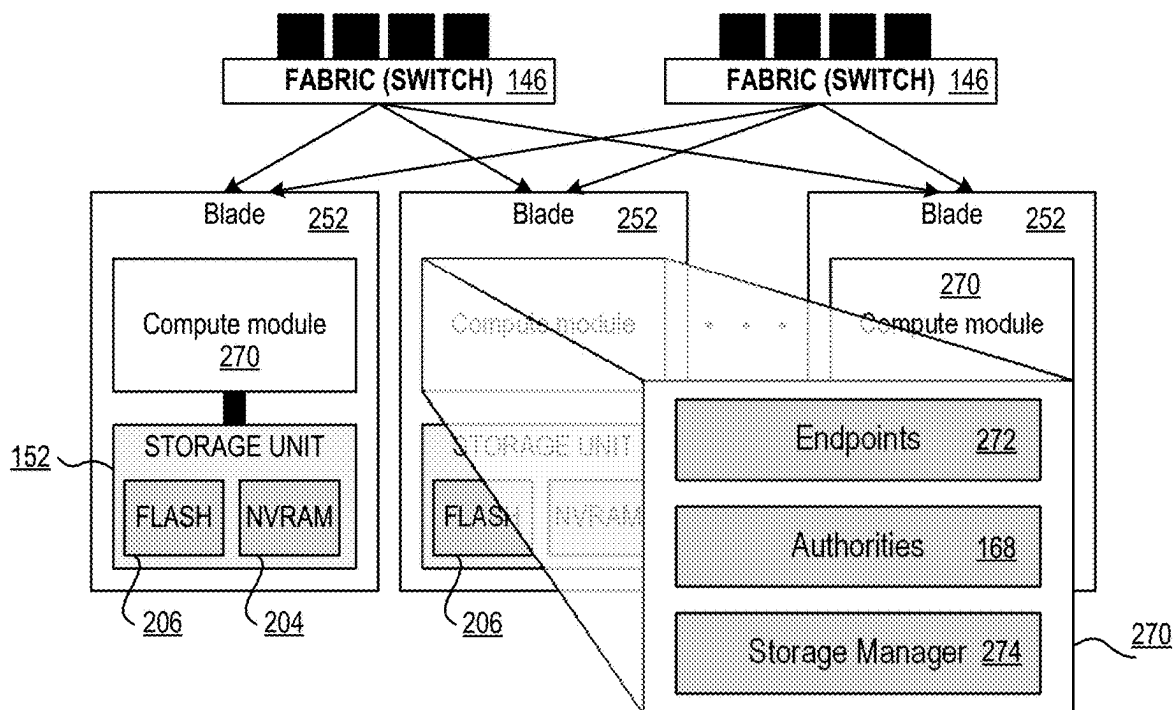


FIG. 2F

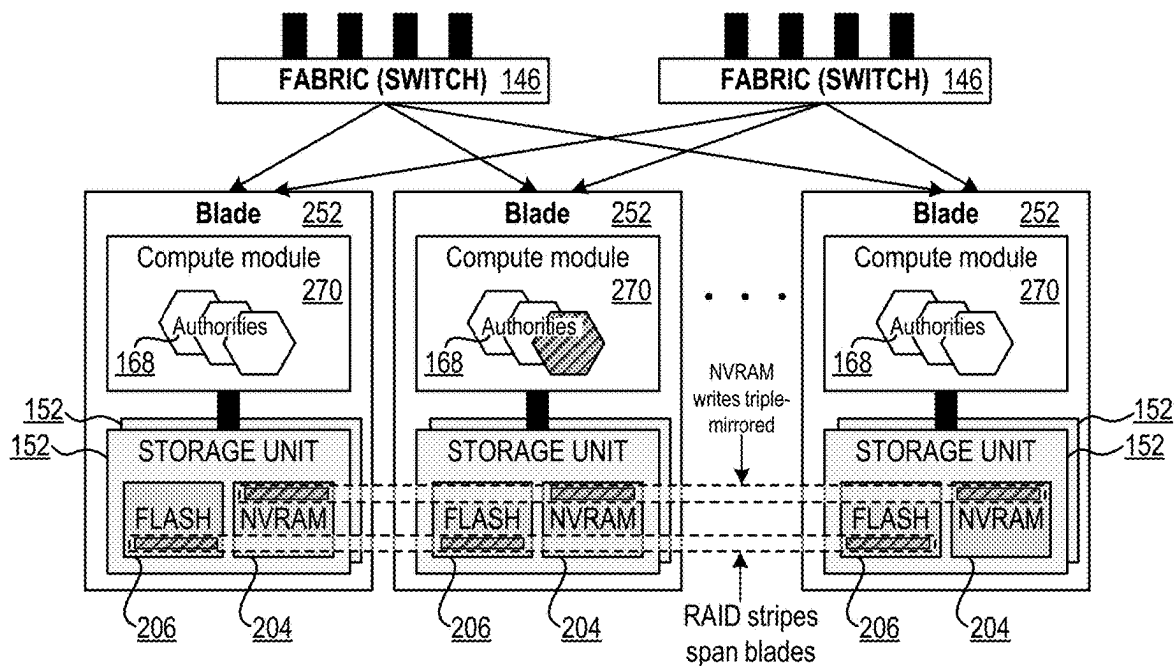


FIG. 2G

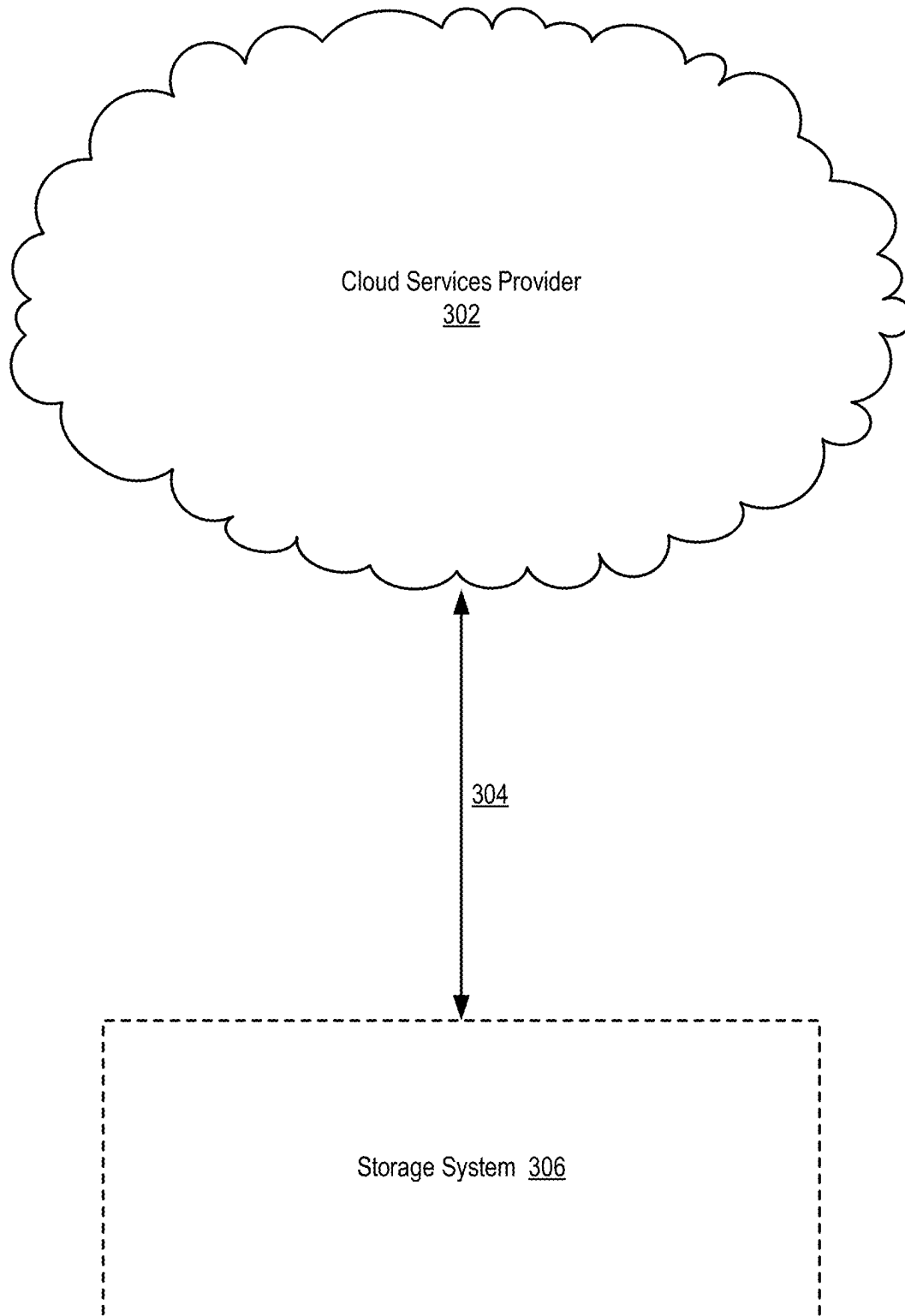


FIG. 3A

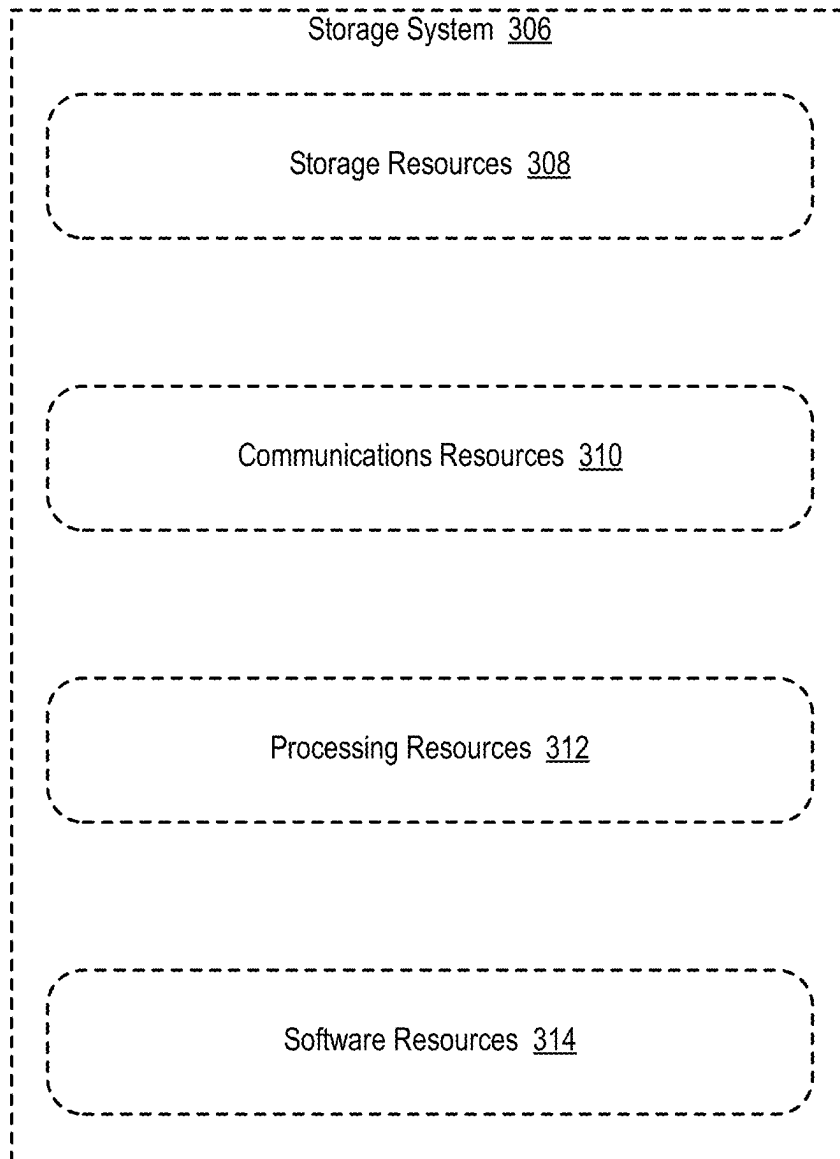


FIG. 3B

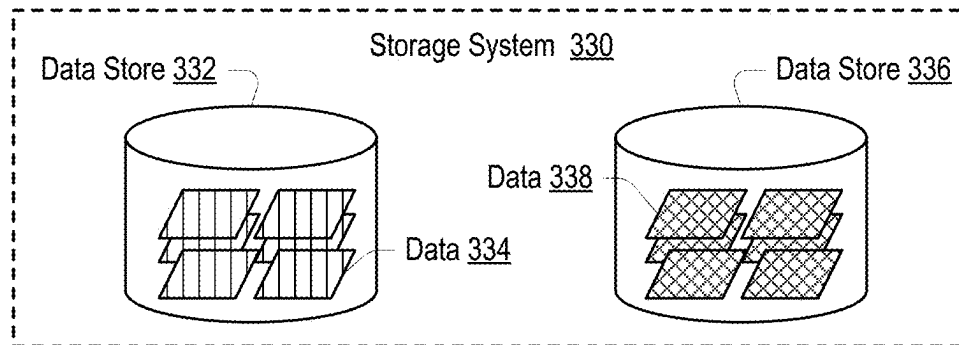
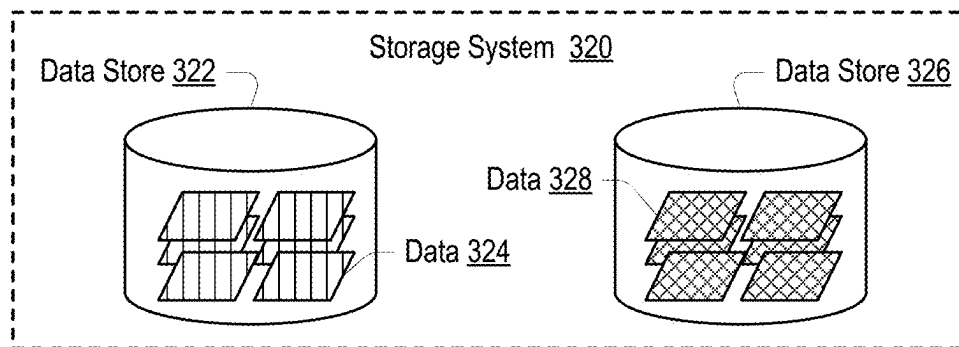
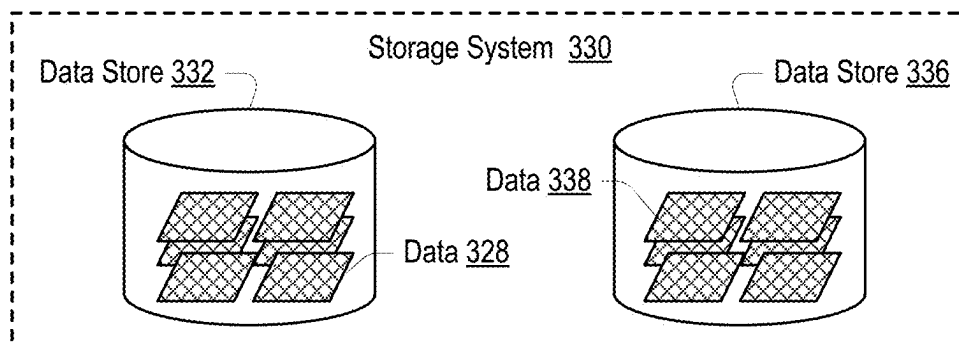
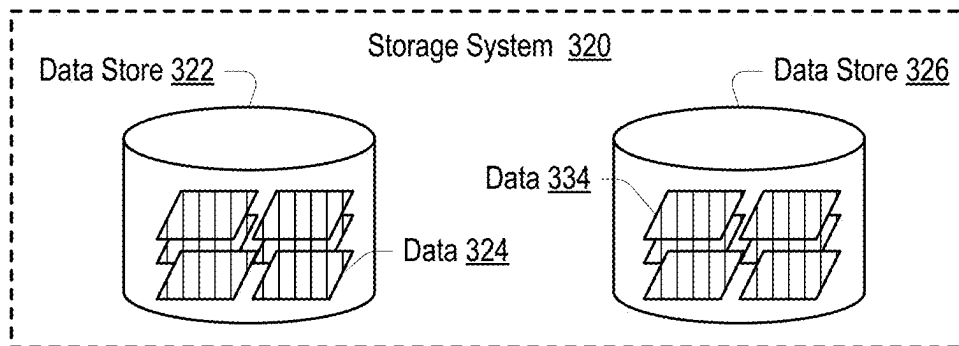
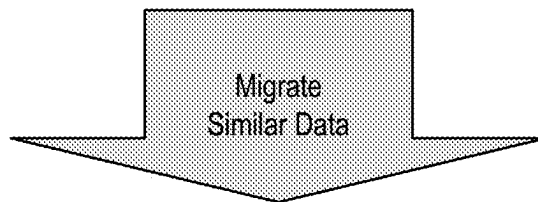


FIG. 3C



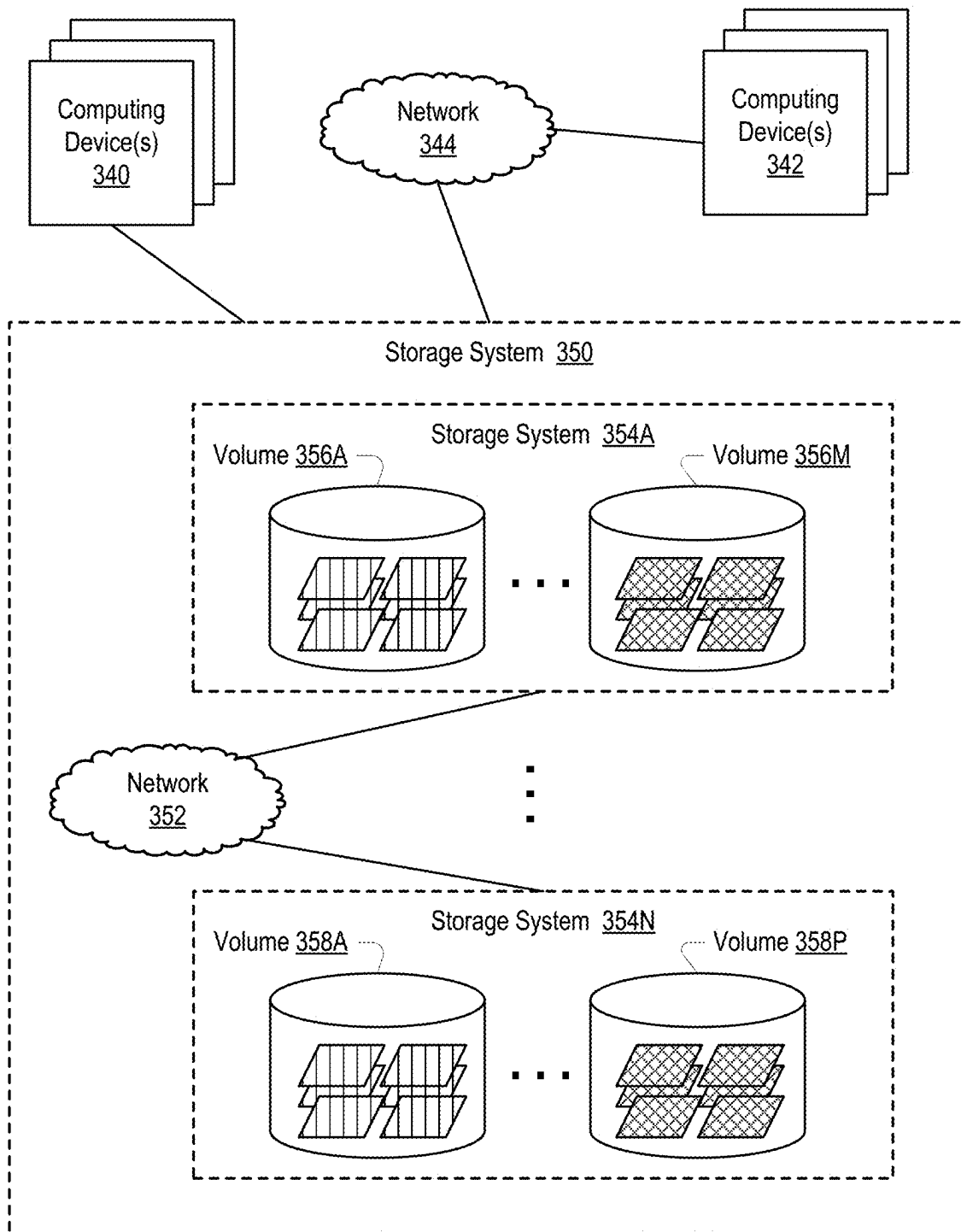


FIG. 3D

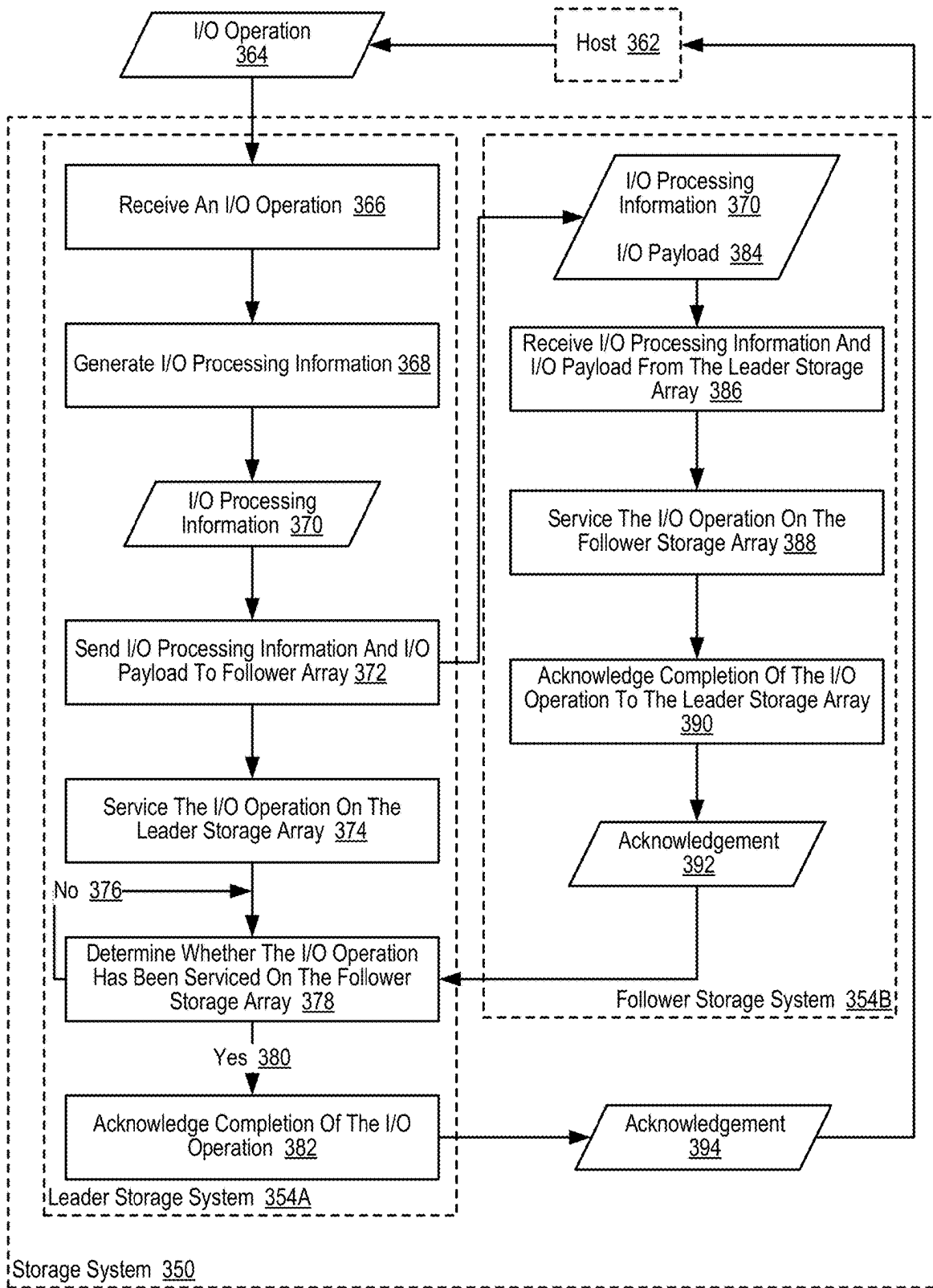
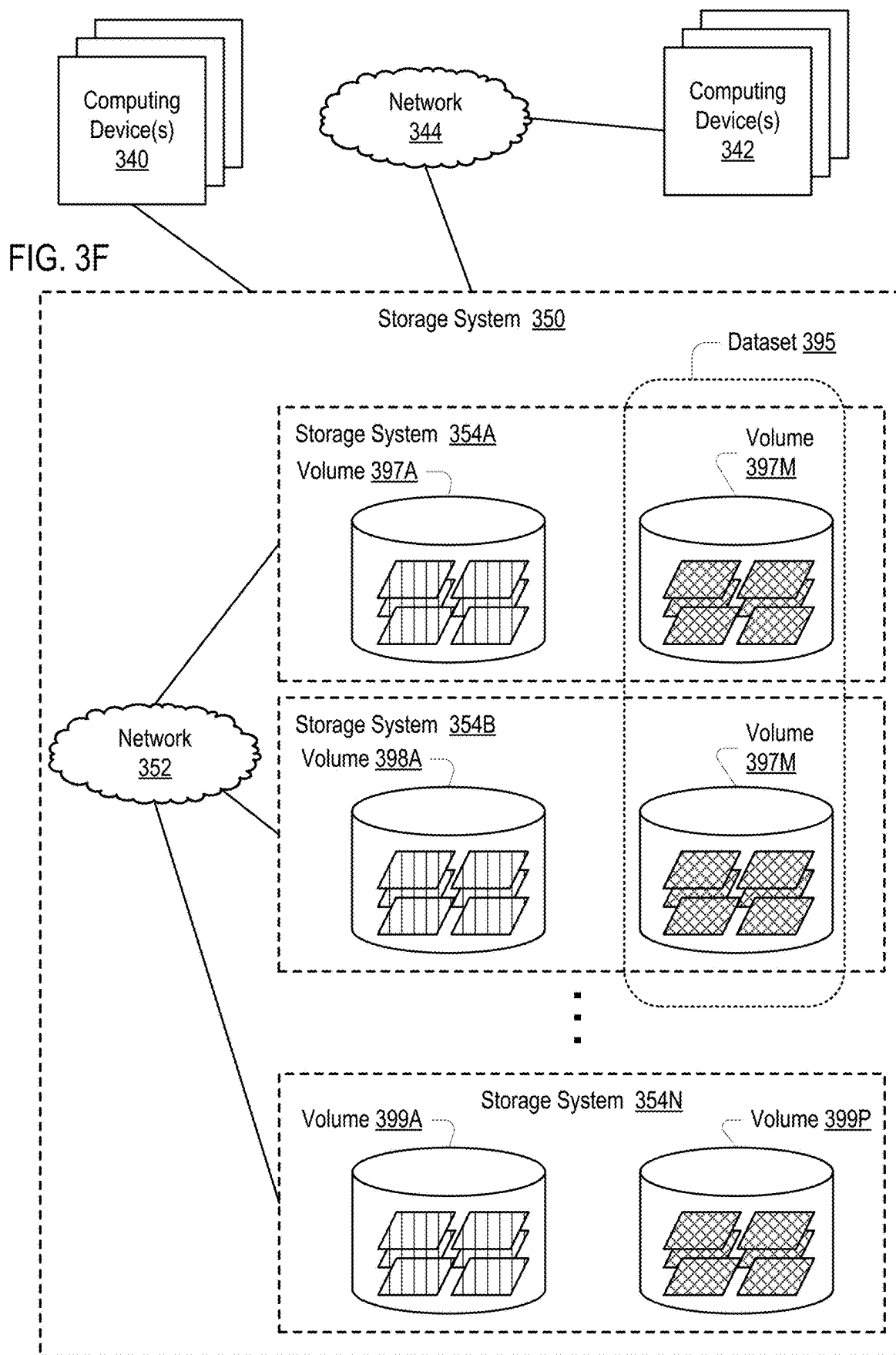


FIG. 3E



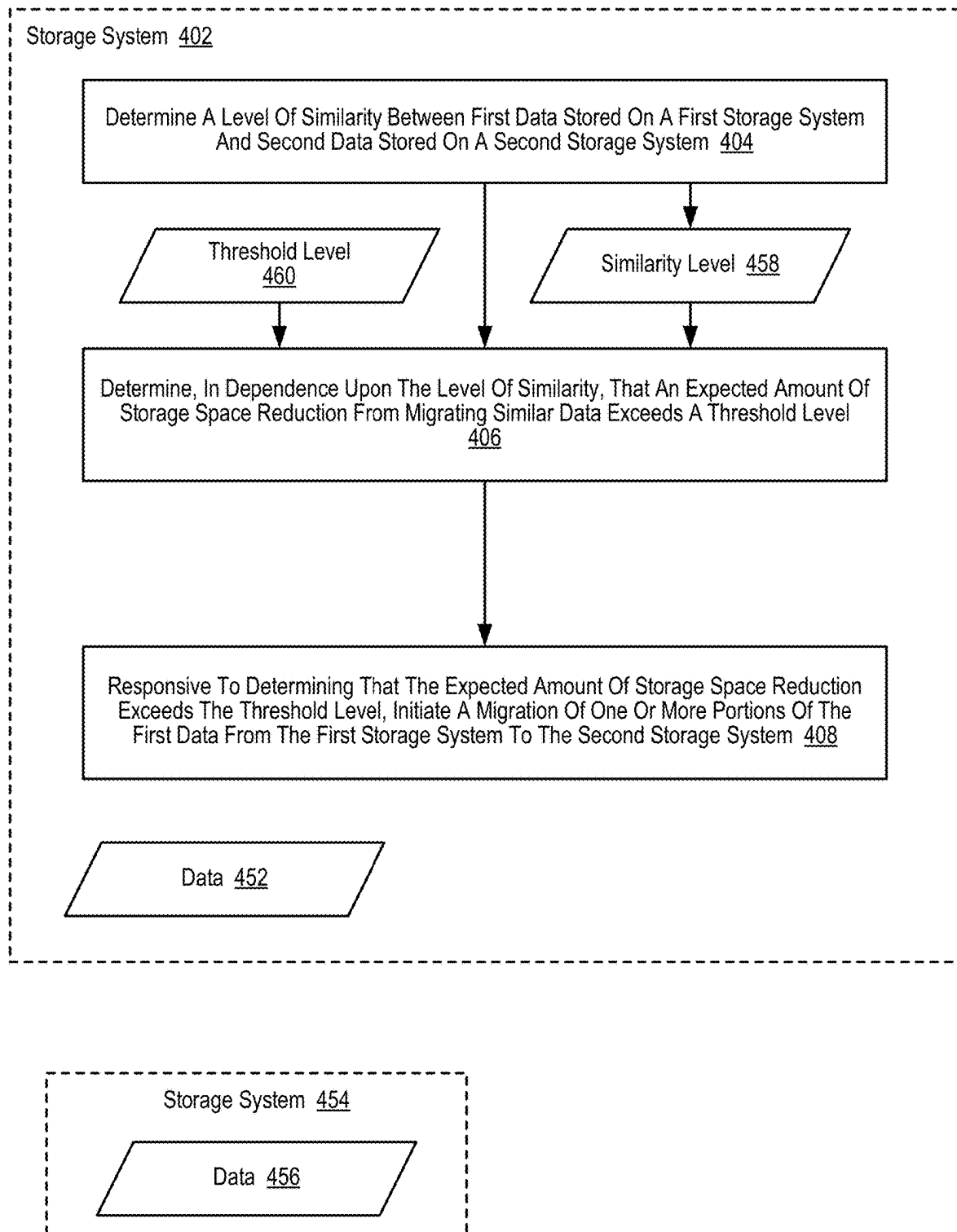


FIG. 4

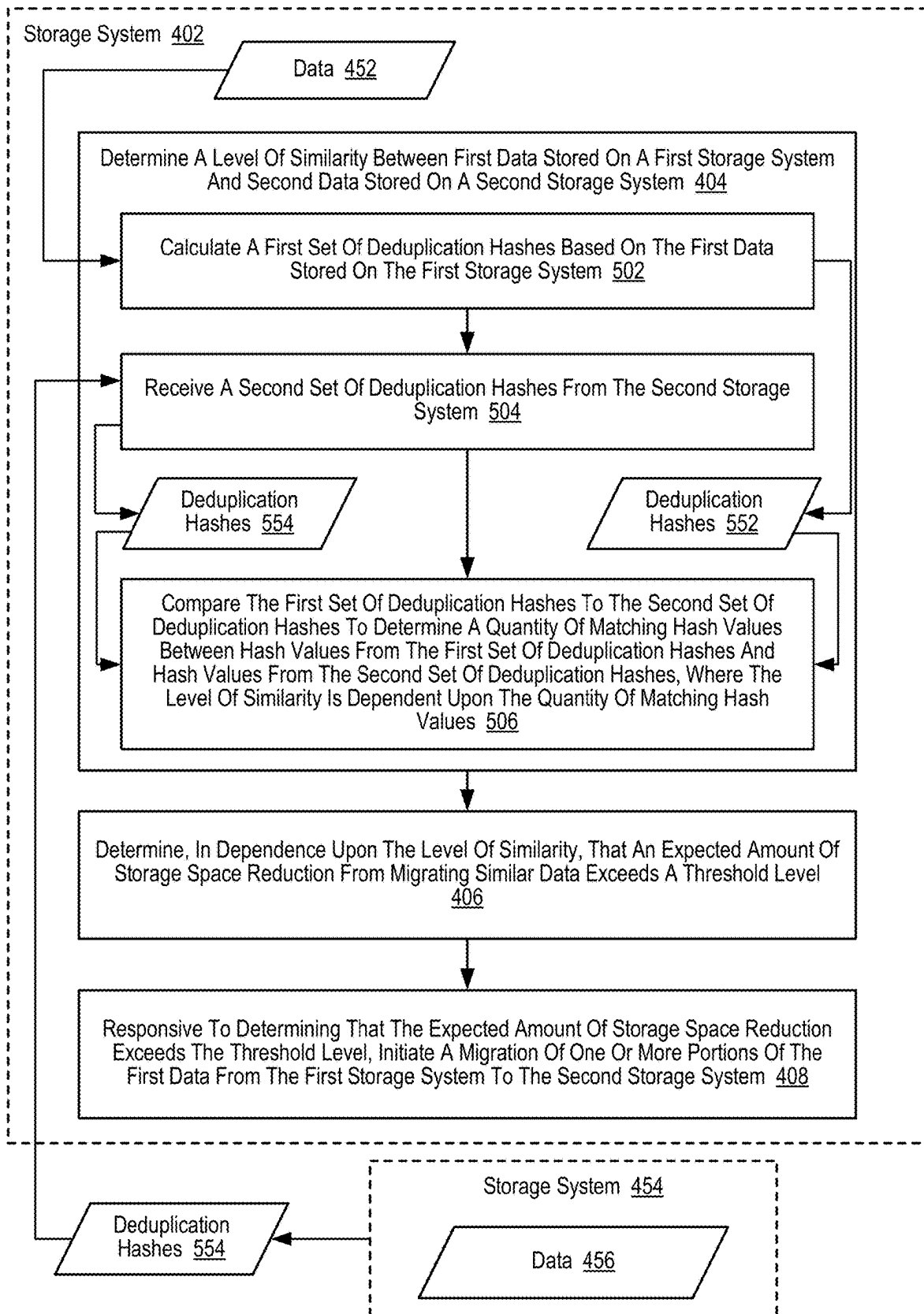


FIG. 5

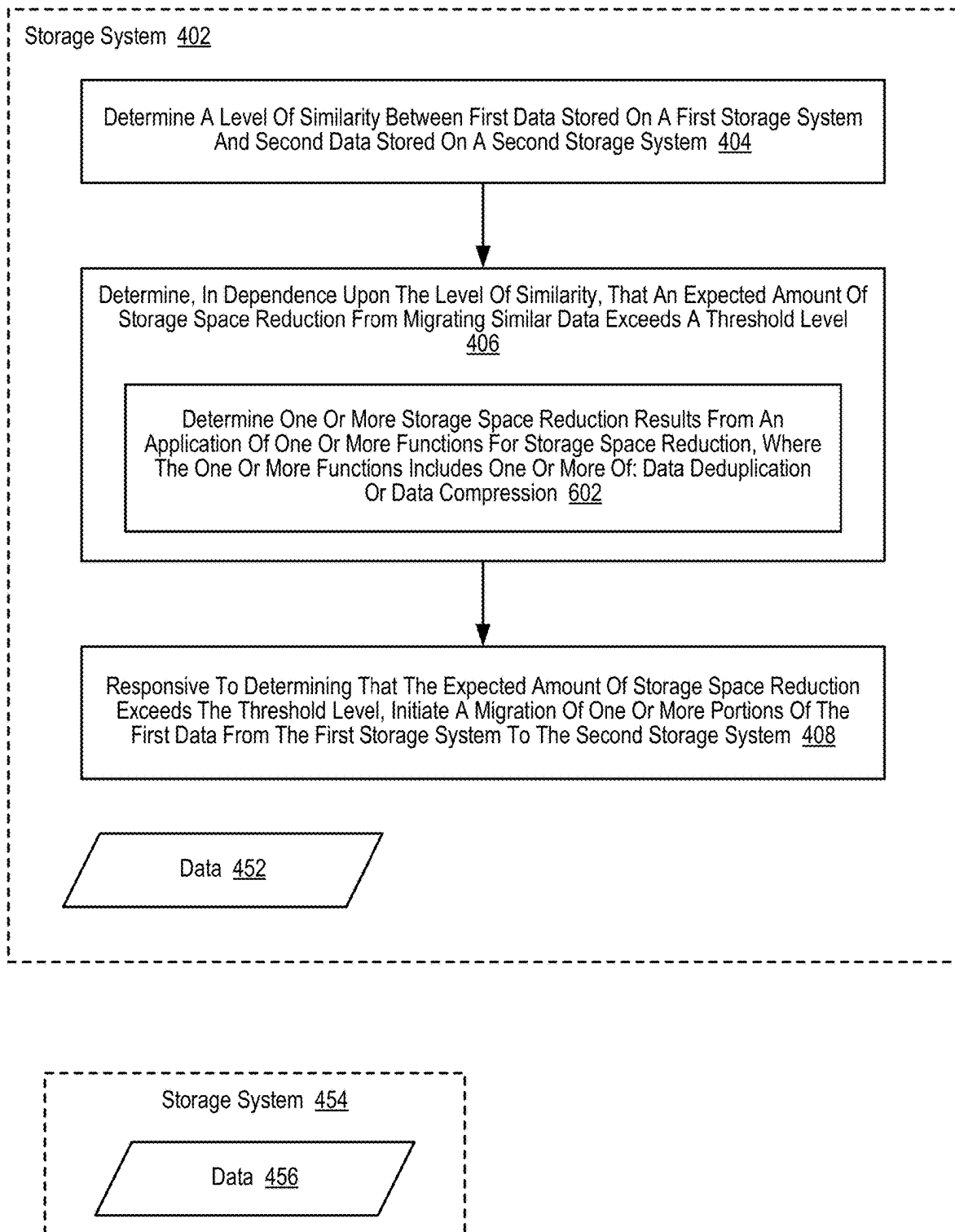


FIG. 6

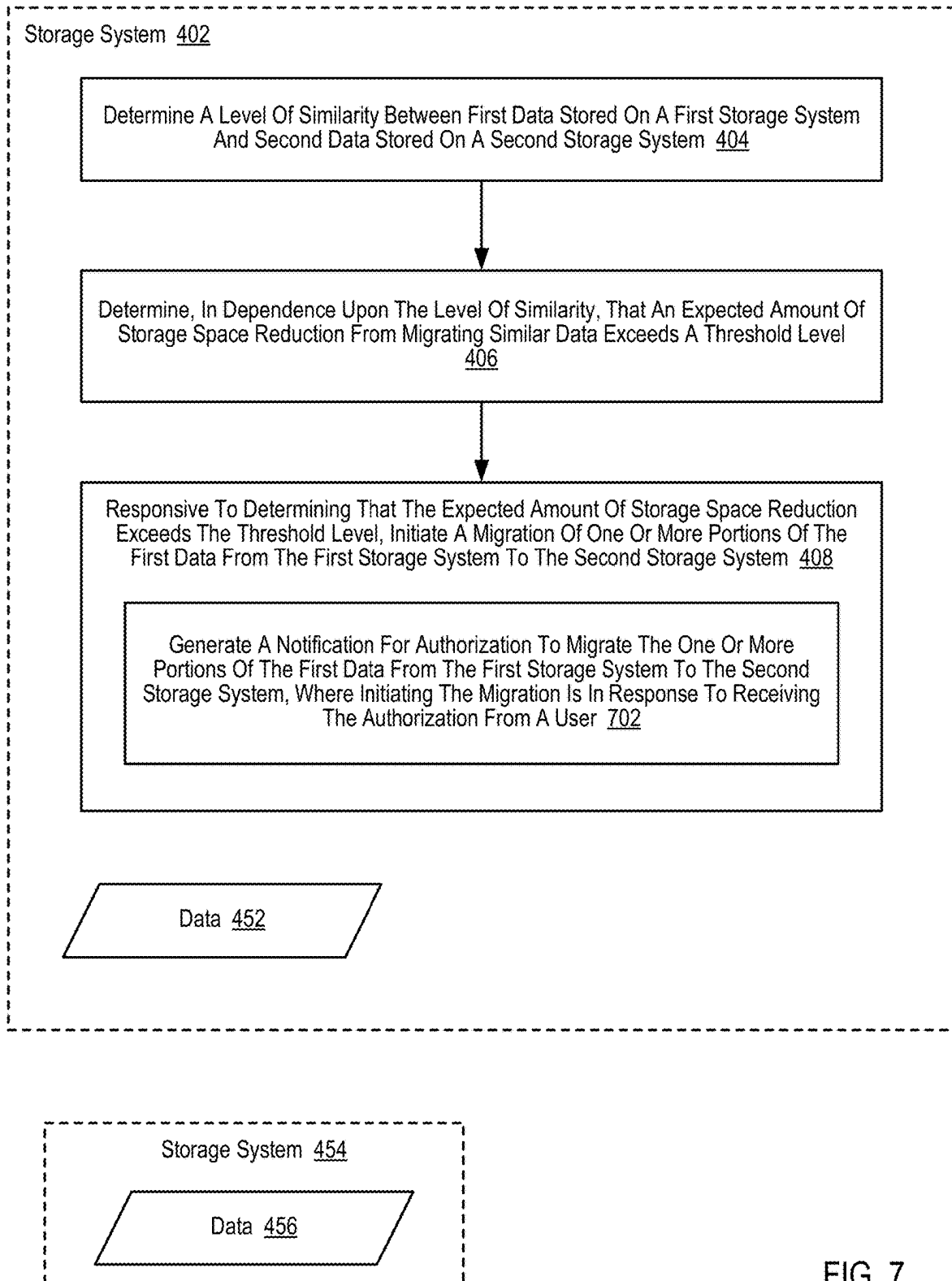


FIG. 7

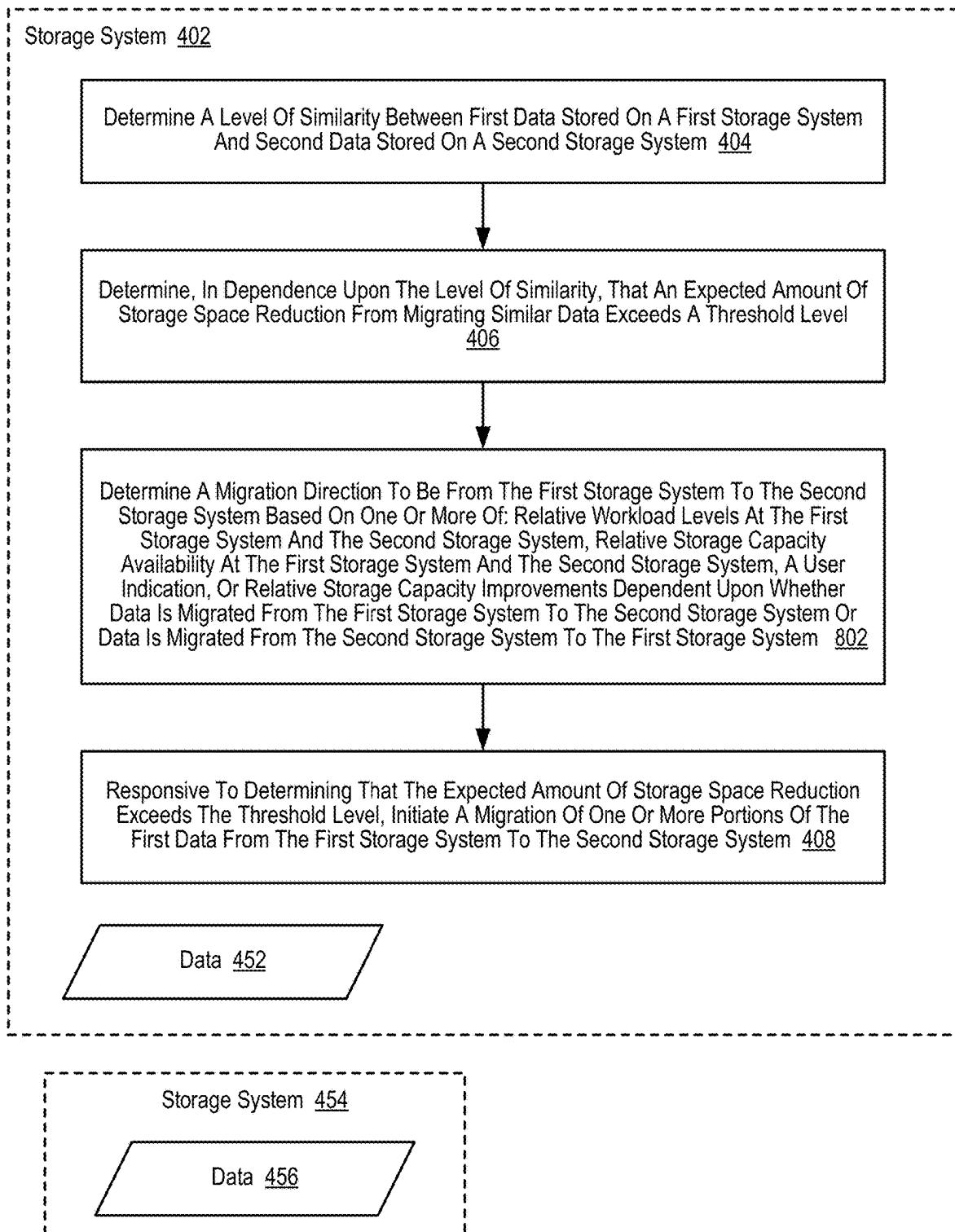


FIG. 8

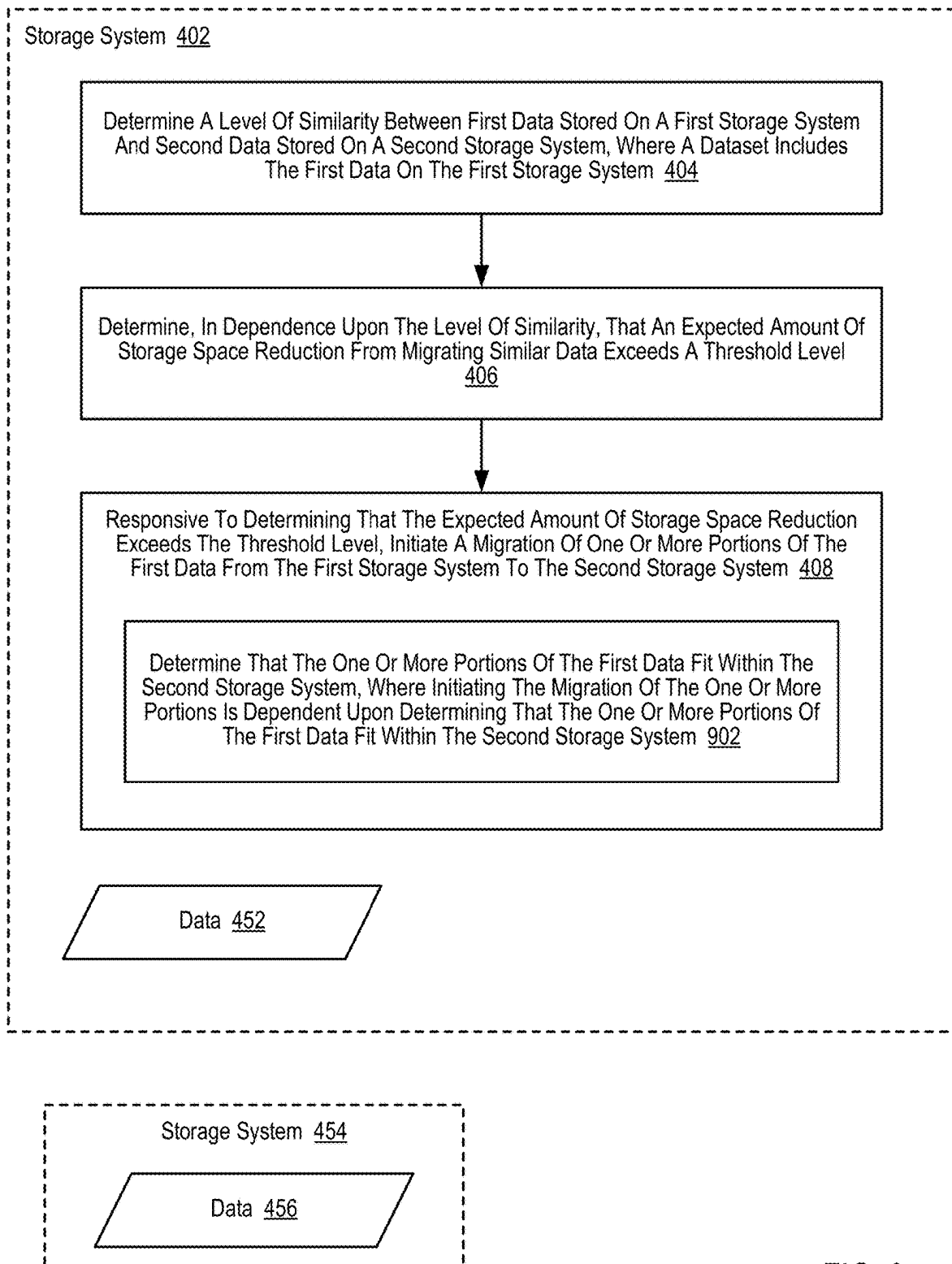


FIG. 9

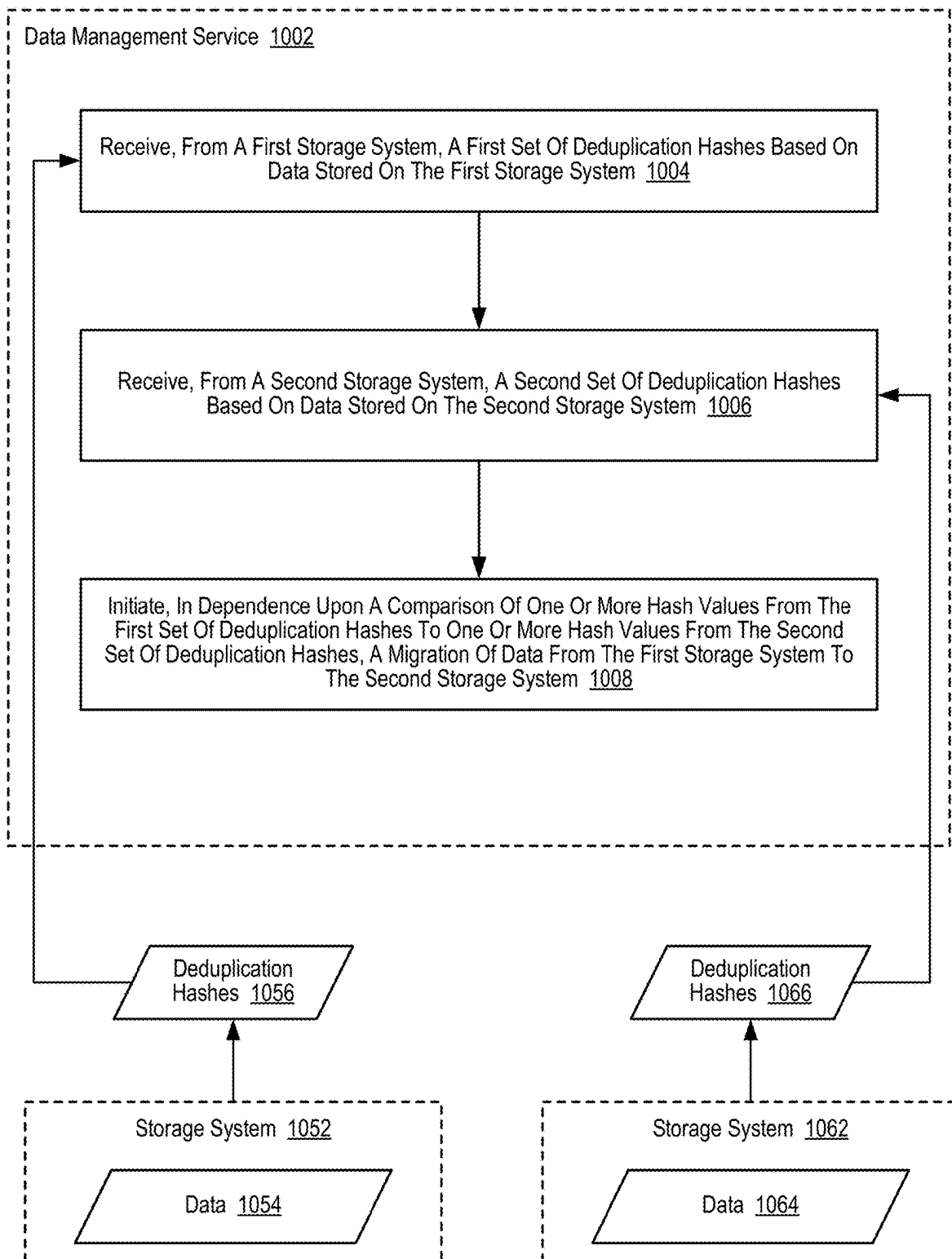


FIG. 10

1

MIGRATING SIMILAR DATA TO A SINGLE DATA REDUCTION POOL

CROSS REFERENCE TO RELATED APPLICATION

This is a continuation application for patent entitled to a filing date and claiming the benefit of earlier-filed U.S. patent application Ser. No. 15/642,305, filed Jul. 5, 2017, herein incorporated by reference in its entirety.

BRIEF DESCRIPTION OF DRAWINGS

FIG. 1A illustrates a first example system for data storage in accordance with some implementations.

FIG. 1B illustrates a second example system for data storage in accordance with some implementations.

FIG. 1C illustrates a third example system for data storage in accordance with some implementations.

FIG. 1D illustrates a fourth example system for data storage in accordance with some implementations.

FIG. 2A is a perspective view of a storage cluster with multiple storage nodes and internal storage coupled to each storage node to provide network attached storage, in accordance with some embodiments.

FIG. 2B is a block diagram showing an interconnect switch coupling multiple storage nodes in accordance with some embodiments.

FIG. 2C is a multiple level block diagram, showing contents of a storage node and contents of one of the non-volatile solid state storage units in accordance with some embodiments.

FIG. 2D shows a storage server environment, which uses embodiments of the storage nodes and storage units of some previous figures in accordance with some embodiments.

FIG. 2E is a blade hardware block diagram, showing a control plane, compute and storage planes, and authorities interacting with underlying physical resources, in accordance with some embodiments.

FIG. 2F depicts elasticity software layers in blades of a storage cluster, in accordance with some embodiments.

FIG. 2G depicts authorities and storage resources in blades of a storage cluster, in accordance with some embodiments.

FIG. 3A sets forth a diagram of a storage system that is coupled for data communications with a cloud services provider in accordance with some embodiments of the present disclosure.

FIG. 3B sets forth a diagram of a storage system in accordance with some embodiments of the present disclosure.

FIG. 3C sets forth a diagram of multiple storage systems in accordance with some embodiments of the present disclosure.

FIG. 3D sets forth a diagram of multiple storage systems in accordance with some embodiments of the present disclosure.

FIG. 3E sets forth a flow chart illustrating an example method for synchronous replication according to some embodiments of the present disclosure.

FIG. 3F sets forth a diagram of multiple storage systems in accordance with some embodiments of the present disclosure.

FIG. 4 sets forth a flow chart illustrating an example method for storage efficiency driven migration according to some embodiments of the present disclosure.

2

FIG. 5 sets forth a flow chart illustrating an example method for storage efficiency driven migration according to some embodiments of the present disclosure.

FIG. 6 sets forth a flow chart illustrating an example method for storage efficiency driven migration according to some embodiments of the present disclosure.

FIG. 7 sets forth a flow chart illustrating an example method for storage efficiency driven migration according to some embodiments of the present disclosure.

FIG. 8 sets forth a flow chart illustrating an example method for storage efficiency driven migration according to some embodiments of the present disclosure.

FIG. 9 sets forth a flow chart illustrating an example method for storage efficiency driven migration according to some embodiments of the present disclosure.

FIG. 10 sets forth a flow chart illustrating an example method for storage efficiency driven migration according to some embodiments of the present disclosure.

DESCRIPTION OF EMBODIMENTS

Example methods, apparatus, and products for storage efficiency driven migration in accordance with embodiments of the present disclosure are described with reference to the accompanying drawings, beginning with FIG. 1A. FIG. 1A illustrates an example system for data storage, in accordance with some implementations. System 100 (also referred to as “storage system” herein) includes numerous elements for purposes of illustration rather than limitation. It may be noted that system 100 may include the same, more, or fewer elements configured in the same or different manner in other implementations.

System 100 includes a number of computing devices 164A-B. Computing devices (also referred to as “client devices” herein) may be embodied, for example, a server in a data center, a workstation, a personal computer, a notebook, or the like. Computing devices 164A-B may be coupled for data communications to one or more storage arrays 102A-B through a storage area network (‘SAN’) 158 or a local area network (‘LAN’) 160.

The SAN 158 may be implemented with a variety of data communications fabrics, devices, and protocols. For example, the fabrics for SAN 158 may include Fibre Channel, Ethernet, Infiniband, Serial Attached Small Computer System Interface (‘SAS’), or the like. Data communications protocols for use with SAN 158 may include Advanced Technology Attachment (‘ATA’), Fibre Channel Protocol, Small Computer System Interface (‘SCSI’), Internet Small Computer System Interface (‘iSCSI’), HyperSCSI, Non-Volatile Memory Express (‘NVMe’) over Fabrics, or the like. It may be noted that SAN 158 is provided for illustration, rather than limitation. Other data communication couplings may be implemented between computing devices 164A-B and storage arrays 102A-B.

The LAN 160 may also be implemented with a variety of fabrics, devices, and protocols. For example, the fabrics for LAN 160 may include Ethernet (802.3), wireless (802.11), or the like. Data communication protocols for use in LAN 160 may include Transmission Control Protocol (‘TCP’), User Datagram Protocol (‘UDP’), Internet Protocol (‘IP’), HyperText Transfer Protocol (‘HTTP’), Wireless Access Protocol (‘WAP’), Handheld Device Transport Protocol (‘HDTTP’), Session Initiation Protocol (‘SIP’), Real Time Protocol (‘RTP’), or the like.

Storage arrays 102A-B may provide persistent data storage for the computing devices 164A-B. Storage array 102A may be contained in a chassis (not shown), and storage array

102B may be contained in another chassis (not shown), in implementations. Storage array 102A and 102B may include one or more storage array controllers 110 (also referred to as “controller” herein). A storage array controller 110 may be embodied as a module of automated computing machinery comprising computer hardware, computer software, or a combination of computer hardware and software. In some implementations, the storage array controllers 110 may be configured to carry out various storage tasks. Storage tasks may include writing data received from the computing devices 164A-B to storage array 102A-B, erasing data from storage array 102A-B, retrieving data from storage array 102A-B and providing data to computing devices 164A-B, monitoring and reporting of disk utilization and performance, performing redundancy operations, such as Redundant Array of Independent Drives (‘RAID’) or RAID-like data redundancy operations, compressing data, encrypting data, and so forth.

Storage array controller 110 may be implemented in a variety of ways, including as a Field Programmable Gate Array (‘FPGA’), a Programmable Logic Chip (PLC), an Application Specific Integrated Circuit (‘ASIC’), System-on-Chip (‘SOC’), or any computing device that includes discrete components such as a processing device, central processing unit, computer memory, or various adapters. Storage array controller 110 may include, for example, a data communications adapter configured to support communications via the SAN 158 or LAN 160. In some implementations, storage array controller 110 may be independently coupled to the LAN 160. In implementations, storage array controller 110 may include an I/O controller or the like that couples the storage array controller 110 for data communications, through a midplane (not shown), to a persistent storage resource 170A-B (also referred to as a “storage resource” herein). The persistent storage resource 170A-B may include any number of storage drives 171A-F (also referred to as “storage devices” herein) and any number of non-volatile Random Access Memory (‘NVRAM’) devices (not shown).

In some implementations, the NVRAM devices of a persistent storage resource 170A-B may be configured to receive, from the storage array controller 110, data to be stored in the storage drives 171A-F. In some examples, the data may originate from computing devices 164A-B. In some examples, writing data to the NVRAM device may be carried out more quickly than directly writing data to the storage drive 171A-F. In implementations, the storage array controller 110 may be configured to utilize the NVRAM devices as a quickly accessible buffer for data destined to be written to the storage drives 171A-F. Latency for write requests using NVRAM devices as a buffer may be improved relative to a system in which a storage array controller 110 writes data directly to the storage drives 171A-F. In some implementations, the NVRAM devices may be implemented with computer memory in the form of high bandwidth, low latency RAM. The NVRAM device is referred to as “non-volatile” because the NVRAM device may receive or include a unique power source that maintains the state of the RAM after main power loss to the NVRAM device. Such a power source may be a battery, one or more capacitors, or the like. In response to a power loss, the NVRAM device may be configured to write the contents of the RAM to a persistent storage, such as the storage drives 171A-F.

In implementations, storage drive 171A-F may refer to any device configured to record data persistently, where “persistently” or “persistent” refers to a device’s ability to

maintain recorded data after loss of power. In some implementations, storage drive 171A-F may correspond to non-disk storage media. For example, the storage drive 171A-F may be one or more solid-state drives (‘SSDs’), flash memory based storage, any type of solid-state non-volatile memory, or any other type of non-mechanical storage device. In other implementations, storage drive 171A-F may include mechanical or spinning hard disk, such as hard-disk drives (‘HDD’).

In some implementations, the storage array controllers 110 may be configured for offloading device management responsibilities from storage drive 171A-F in storage array 102A-B. For example, storage array controllers 110 may manage control information that may describe the state of one or more memory blocks in the storage drives 171A-F. The control information may indicate, for example, that a particular memory block has failed and should no longer be written to, that a particular memory block contains boot code for a storage array controller 110, the number of program-erase (‘P/E’) cycles that have been performed on a particular memory block, the age of data stored in a particular memory block, the type of data that is stored in a particular memory block, and so forth. In some implementations, the control information may be stored with an associated memory block as metadata. In other implementations, the control information for the storage drives 171A-F may be stored in one or more particular memory blocks of the storage drives 171A-F that are selected by the storage array controller 110. The selected memory blocks may be tagged with an identifier indicating that the selected memory block contains control information. The identifier may be utilized by the storage array controllers 110 in conjunction with storage drives 171A-F to quickly identify the memory blocks that contain control information. For example, the storage controllers 110 may issue a command to locate memory blocks that contain control information. It may be noted that control information may be so large that parts of the control information may be stored in multiple locations, that the control information may be stored in multiple locations for purposes of redundancy, for example, or that the control information may otherwise be distributed across multiple memory blocks in the storage drive 171A-F.

In implementations, storage array controllers 110 may offload device management responsibilities from storage drives 171A-F of storage array 102A-B by retrieving, from the storage drives 171A-F, control information describing the state of one or more memory blocks in the storage drives 171A-F. Retrieving the control information from the storage drives 171A-F may be carried out, for example, by the storage array controller 110 querying the storage drives 171A-F for the location of control information for a particular storage drive 171A-F. The storage drives 171A-F may be configured to execute instructions that enable the storage drive 171A-F to identify the location of the control information. The instructions may be executed by a controller (not shown) associated with or otherwise located on the storage drive 171A-F and may cause the storage drive 171A-F to scan a portion of each memory block to identify the memory blocks that store control information for the storage drives 171A-F. The storage drives 171A-F may respond by sending a response message to the storage array controller 110 that includes the location of control information for the storage drive 171A-F. Responsive to receiving the response message, storage array controllers 110 may issue a request to read data stored at the address associated with the location of control information for the storage drives 171A-F.

5

In other implementations, the storage array controllers **110** may further offload device management responsibilities from storage drives **171A-F** by performing, in response to receiving the control information, a storage drive management operation. A storage drive management operation may include, for example, an operation that is typically performed by the storage drive **171A-F** (e.g., the controller (not shown) associated with a particular storage drive **171A-F**). A storage drive management operation may include, for example, ensuring that data is not written to failed memory blocks within the storage drive **171A-F**, ensuring that data is written to memory blocks within the storage drive **171A-F** in such a way that adequate wear leveling is achieved, and so forth.

In implementations, storage array **102A-B** may implement two or more storage array controllers **110**. For example, storage array **102A** may include storage array controllers **110A** and storage array controllers **110B**. At a given instance, a single storage array controller **110** (e.g., storage array controller **110A**) of a storage system **100** may be designated with primary status (also referred to as “primary controller” herein), and other storage array controllers **110** (e.g., storage array controller **110A**) may be designated with secondary status (also referred to as “secondary controller” herein). The primary controller may have particular rights, such as permission to alter data in persistent storage resource **170A-B** (e.g., writing data to persistent storage resource **170A-B**). At least some of the rights of the primary controller may supersede the rights of the secondary controller. For instance, the secondary controller may not have permission to alter data in persistent storage resource **170A-B** when the primary controller has the right. The status of storage array controllers **110** may change. For example, storage array controller **110A** may be designated with secondary status, and storage array controller **110B** may be designated with primary status.

In some implementations, a primary controller, such as storage array controller **110A**, may serve as the primary controller for one or more storage arrays **102A-B**, and a second controller, such as storage array controller **110B**, may serve as the secondary controller for the one or more storage arrays **102A-B**. For example, storage array controller **110A** may be the primary controller for storage array **102A** and storage array **102B**, and storage array controller **110B** may be the secondary controller for storage array **102A** and **102B**. In some implementations, storage array controllers **110C** and **110D** (also referred to as “storage processing modules”) may neither have primary or secondary status. Storage array controllers **110C** and **110D**, implemented as storage processing modules, may act as a communication interface between the primary and secondary controllers (e.g., storage array controllers **110A** and **110B**, respectively) and storage array **102B**. For example, storage array controller **110A** of storage array **102A** may send a write request, via SAN **158**, to storage array **102B**. The write request may be received by both storage array controllers **110C** and **110D** of storage array **102B**. Storage array controllers **110C** and **110D** facilitate the communication, e.g., send the write request to the appropriate storage drive **171A-F**. It may be noted that in some implementations storage processing modules may be used to increase the number of storage drives controlled by the primary and secondary controllers.

In implementations, storage array controllers **110** are communicatively coupled, via a midplane (not shown), to one or more storage drives **171A-F** and to one or more NVRAM devices (not shown) that are included as part of a

6

storage array **102A-B**. The storage array controllers **110** may be coupled to the midplane via one or more data communications links and the midplane may be coupled to the storage drives **171A-F** and the NVRAM devices via one or more data communications links. The data communications links described herein are collectively illustrated by data communications links **108A-D** and may include a Peripheral Component Interconnect Express (‘PCIe’) bus, for example.

FIG. **1B** illustrates an example system for data storage, in accordance with some implementations. Storage array controller **101** illustrated in FIG. **1B** may be similar to the storage array controllers **110** described with respect to FIG. **1A**. In one example, storage array controller **101** may be similar to storage array controller **110A** or storage array controller **110B**. Storage array controller **101** includes numerous elements for purposes of illustration rather than limitation. It may be noted that storage array controller **101** may include the same, more, or fewer elements configured in the same or different manner in other implementations. It may be noted that elements of FIG. **1A** may be included below to help illustrate features of storage array controller **101**.

Storage array controller **101** may include one or more processing devices **104** and random access memory (‘RAM’) **111**. Processing device **104** (or controller **101**) represents one or more general-purpose processing devices such as a microprocessor, central processing unit, or the like. More particularly, the processing device **104** (or controller **101**) may be a complex instruction set computing (‘CISC’) microprocessor, reduced instruction set computing (‘RISC’) microprocessor, very long instruction word (‘VLIW’) microprocessor, or a processor implementing other instruction sets or processors implementing a combination of instruction sets. The processing device **104** (or controller **101**) may also be one or more special-purpose processing devices such as an application specific integrated circuit (‘ASIC’), a field programmable gate array (‘FPGA’), a digital signal processor (‘DSP’), network processor, or the like.

The processing device **104** may be connected to the RAM **111** via a data communications link **106**, which may be embodied as a high speed memory bus such as a Double-Data Rate 4 (‘DDR4’) bus. Stored in RAM **111** is an operating system **112**. In some implementations, instructions **113** are stored in RAM **111**. Instructions **113** may include computer program instructions for performing operations in a direct-mapped flash storage system. In one embodiment, a direct-mapped flash storage system is one that addresses data blocks within flash drives directly and without an address translation performed by the storage controllers of the flash drives.

In implementations, storage array controller **101** includes one or more host bus adapters **103A-C** that are coupled to the processing device **104** via a data communications link **105A-C**. In implementations, host bus adapters **103A-C** may be computer hardware that connects a host system (e.g., the storage array controller) to other network and storage arrays. In some examples, host bus adapters **103A-C** may be a Fibre Channel adapter that enables the storage array controller **101** to connect to a SAN, an Ethernet adapter that enables the storage array controller **101** to connect to a LAN, or the like. Host bus adapters **103A-C** may be coupled to the processing device **104** via a data communications link **105A-C** such as, for example, a PCIe bus.

In implementations, storage array controller **101** may include a host bus adapter **114** that is coupled to an expander **115**. The expander **115** may be used to attach a host system to a larger number of storage drives. The expander **115** may,

for example, be a SAS expander utilized to enable the host bus adapter **114** to attach to storage drives in an implementation where the host bus adapter **114** is embodied as a SAS controller.

In implementations, storage array controller **101** may include a switch **116** coupled to the processing device **104** via a data communications link **109**. The switch **116** may be a computer hardware device that can create multiple endpoints out of a single endpoint, thereby enabling multiple devices to share a single endpoint. The switch **116** may, for example, be a PCIe switch that is coupled to a PCIe bus (e.g., data communications link **109**) and presents multiple PCIe connection points to the midplane.

In implementations, storage array controller **101** includes a data communications link **107** for coupling the storage array controller **101** to other storage array controllers. In some examples, data communications link **107** may be a QuickPath Interconnect (QPI) interconnect.

A traditional storage system that uses traditional flash drives may implement a process across the flash drives that are part of the traditional storage system. For example, a higher level process of the storage system may initiate and control a process across the flash drives. However, a flash drive of the traditional storage system may include its own storage controller that also performs the process. Thus, for the traditional storage system, a higher level process (e.g., initiated by the storage system) and a lower level process (e.g., initiated by a storage controller of the storage system) may both be performed.

To resolve various deficiencies of a traditional storage system, operations may be performed by higher level processes and not by the lower level processes. For example, the flash storage system may include flash drives that do not include storage controllers that provide the process. Thus, the operating system of the flash storage system itself may initiate and control the process. This may be accomplished by a direct-mapped flash storage system that addresses data blocks within the flash drives directly and without an address translation performed by the storage controllers of the flash drives.

The operating system of the flash storage system may identify and maintain a list of allocation units across multiple flash drives of the flash storage system. The allocation units may be entire erase blocks or multiple erase blocks. The operating system may maintain a map or address range that directly maps addresses to erase blocks of the flash drives of the flash storage system.

Direct mapping to the erase blocks of the flash drives may be used to rewrite data and erase data. For example, the operations may be performed on one or more allocation units that include a first data and a second data where the first data is to be retained and the second data is no longer being used by the flash storage system. The operating system may initiate the process to write the first data to new locations within other allocation units and erasing the second data and marking the allocation units as being available for use for subsequent data. Thus, the process may only be performed by the higher level operating system of the flash storage system without an additional lower level process being performed by controllers of the flash drives.

Advantages of the process being performed only by the operating system of the flash storage system include increased reliability of the flash drives of the flash storage system as unnecessary or redundant write operations are not being performed during the process. One possible point of novelty here is the concept of initiating and controlling the process at the operating system of the flash storage system.

In addition, the process can be controlled by the operating system across multiple flash drives. This is in contrast to the process being performed by a storage controller of a flash drive.

A storage system can consist of two storage array controllers that share a set of drives for failover purposes, or it could consist of a single storage array controller that provides a storage service that utilizes multiple drives, or it could consist of a distributed network of storage array controllers each with some number of drives or some amount of Flash storage where the storage array controllers in the network collaborate to provide a complete storage service and collaborate on various aspects of a storage service including storage allocation and garbage collection.

FIG. 1C illustrates a third example system **117** for data storage in accordance with some implementations. System **117** (also referred to as “storage system” herein) includes numerous elements for purposes of illustration rather than limitation. It may be noted that system **117** may include the same, more, or fewer elements configured in the same or different manner in other implementations.

In one embodiment, system **117** includes a dual Peripheral Component Interconnect (‘PCI’) flash storage device **118** with separately addressable fast write storage. System **117** may include a storage controller **119**. In one embodiment, storage controller **119** may be a CPU, ASIC, FPGA, or any other circuitry that may implement control structures necessary according to the present disclosure. In one embodiment, system **117** includes flash memory devices (e.g., including flash memory devices **120a-n**), operatively coupled to various channels of the storage device controller **119**. Flash memory devices **120a-n**, may be presented to the controller **119** as an addressable collection of Flash pages, erase blocks, and/or control elements sufficient to allow the storage device controller **119** to program and retrieve various aspects of the Flash. In one embodiment, storage device controller **119** may perform operations on flash memory devices **120A-N** including storing and retrieving data content of pages, arranging and erasing any blocks, tracking statistics related to the use and reuse of Flash memory pages, erase blocks, and cells, tracking and predicting error codes and faults within the Flash memory, controlling voltage levels associated with programming and retrieving contents of Flash cells, etc.

In one embodiment, system **117** may include RAM **121** to store separately addressable fast-write data. In one embodiment, RAM **121** may be one or more separate discrete devices. In another embodiment, RAM **121** may be integrated into storage device controller **119** or multiple storage device controllers. The RAM **121** may be utilized for other purposes as well, such as temporary program memory for a processing device (e.g., a CPU) in the storage device controller **119**.

In one embodiment, system **119** may include a stored energy device **122**, such as a rechargeable battery or a capacitor. Stored energy device **122** may store energy sufficient to power the storage device controller **119**, some amount of the RAM (e.g., RAM **121**), and some amount of Flash memory (e.g., Flash memory **120a-120n**) for sufficient time to write the contents of RAM to Flash memory. In one embodiment, storage device controller **119** may write the contents of RAM to Flash Memory if the storage device controller detects loss of external power.

In one embodiment, system **117** includes two data communications links **123a**, **123b**. In one embodiment, data communications links **123a**, **123b** may be PCI interfaces. In another embodiment, data communications links **123a**, **123b**

may be based on other communications standards (e.g., HyperTransport, InfiniBand, etc.). Data communications links **123a**, **123b** may be based on non-volatile memory express ('NVMe') or NVMe over fabrics ('NVMe-f') specifications that allow external connection to the storage device controller **119** from other components in the storage system **117**. It should be noted that data communications links may be interchangeably referred to herein as PCI buses for convenience.

System **117** may also include an external power source (not shown), which may be provided over one or both data communications links **123a**, **123b**, or which may be provided separately. An alternative embodiment includes a separate Flash memory (not shown) dedicated for use in storing the content of RAM **121**. The storage device controller **119** may present a logical device over a PCI bus which may include an addressable fast-write logical device, or a distinct part of the logical address space of the storage device **118**, which may be presented as PCI memory or as persistent storage. In one embodiment, operations to store into the device are directed into the RAM **121**. On power failure, the storage device controller **119** may write stored content associated with the addressable fast-write logical storage to Flash memory (e.g., Flash memory **120a-n**) for long-term persistent storage.

In one embodiment, the logical device may include some presentation of some or all of the content of the Flash memory devices **120a-n**, where that presentation allows a storage system including a storage device **118** (e.g., storage system **117**) to directly address Flash memory pages and directly reprogram erase blocks from storage system components that are external to the storage device through the PCI bus. The presentation may also allow one or more of the external components to control and retrieve other aspects of the Flash memory including some or all of: tracking statistics related to use and reuse of Flash memory pages, erase blocks, and cells across all the Flash memory devices; tracking and predicting error codes and faults within and across the Flash memory devices; controlling voltage levels associated with programming and retrieving contents of Flash cells; etc.

In one embodiment, the stored energy device **122** may be sufficient to ensure completion of in-progress operations to the Flash memory devices **107a-120n** stored energy device **122** may power storage device controller **119** and associated Flash memory devices (e.g., **120a-n**) for those operations, as well as for the storing of fast-write RAM to Flash memory. Stored energy device **122** may be used to store accumulated statistics and other parameters kept and tracked by the Flash memory devices **120a-n** and/or the storage device controller **119**. Separate capacitors or stored energy devices (such as smaller capacitors near or embedded within the Flash memory devices themselves) may be used for some or all of the operations described herein.

Various schemes may be used to track and optimize the life span of the stored energy component, such as adjusting voltage levels over time, partially discharging the storage energy device **122** to measure corresponding discharge characteristics, etc. If the available energy decreases over time, the effective available capacity of the addressable fast-write storage may be decreased to ensure that it can be written safely based on the currently available stored energy.

FIG. 1D illustrates a third example system **124** for data storage in accordance with some implementations. In one embodiment, system **124** includes storage controllers **125a**, **125b**. In one embodiment, storage controllers **125a**, **125b** are operatively coupled to Dual PCI storage devices **119a**,

119b and **119c**, **119d**, respectively. Storage controllers **125a**, **125b** may be operatively coupled (e.g., via a storage network **130**) to some number of host computers **127a-n**.

In one embodiment, two storage controllers (e.g., **125a** and **125b**) provide storage services, such as a SCS) block storage array, a file server, an object server, a database or data analytics service, etc. The storage controllers **125a**, **125b** may provide services through some number of network interfaces (e.g., **126a-d**) to host computers **127a-n** outside of the storage system **124**. Storage controllers **125a**, **125b** may provide integrated services or an application entirely within the storage system **124**, forming a converged storage and compute system. The storage controllers **125a**, **125b** may utilize the fast write memory within or across storage devices **119a-d** to journal in progress operations to ensure the operations are not lost on a power failure, storage controller removal, storage controller or storage system shutdown, or some fault of one or more software or hardware components within the storage system **124**.

In one embodiment, controllers **125a**, **125b** operate as PCI masters to one or the other PCI buses **128a**, **128b**. In another embodiment, **128a** and **128b** may be based on other communications standards (e.g., HyperTransport, InfiniBand, etc.). Other storage system embodiments may operate storage controllers **125a**, **125b** as multi-masters for both PCI buses **128a**, **128b**. Alternately, a PCI/NVMe/NVMe-f switching infrastructure or fabric may connect multiple storage controllers. Some storage system embodiments may allow storage devices to communicate with each other directly rather than communicating only with storage controllers. In one embodiment, a storage device controller **119a** may be operable under direction from a storage controller **125a** to synthesize and transfer data to be stored into Flash memory devices from data that has been stored in RAM (e.g., RAM **121** of FIG. 1C). For example, a recalculated version of RAM content may be transferred after a storage controller has determined that an operation has fully committed across the storage system, or when fast-write memory on the device has reached a certain used capacity, or after a certain amount of time, to ensure improve safety of the data or to release addressable fast-write capacity for reuse. This mechanism may be used, for example, to avoid a second transfer over a bus (e.g., **128a**, **128b**) from the storage controllers **125a**, **125b**. In one embodiment, a recalculation may include compressing data, attaching indexing or other metadata, combining multiple data segments together, performing erasure code calculations, etc.

In one embodiment, under direction from a storage controller **125a**, **125b**, a storage device controller **119a**, **119b** may be operable to calculate and transfer data to other storage devices from data stored in RAM (e.g., RAM **121** of FIG. 1C) without involvement of the storage controllers **125a**, **125b**. This operation may be used to mirror data stored in one controller **125a** to another controller **125b**, or it could be used to offload compression, data aggregation, and/or erasure coding calculations and transfers to storage devices to reduce load on storage controllers or the storage controller interface **129a**, **129b** to the PCI bus **128a**, **128b**.

A storage device controller **119** may include mechanisms for implementing high availability primitives for use by other parts of a storage system external to the Dual PCI storage device **118**. For example, reservation or exclusion primitives may be provided so that, in a storage system with two storage controllers providing a highly available storage service, one storage controller may prevent the other storage controller from accessing or continuing to access the storage device. This could be used, for example, in cases where one

11

controller detects that the other controller is not functioning properly or where the interconnect between the two storage controllers may itself not be functioning properly.

In one embodiment, a storage system for use with Dual PCI direct mapped storage devices with separately address-
able fast write storage includes systems that manage erase
blocks or groups of erase blocks as allocation units for
storing data on behalf of the storage service, or for storing
metadata (e.g., indexes, logs, etc.) associated with the storage
service, or for proper management of the storage system
itself. Flash pages, which may be a few kilobytes in size,
may be written as data arrives or as the storage system is to
persist data for long intervals of time (e.g., above a defined
threshold of time). To commit data more quickly, or to
reduce the number of writes to the Flash memory devices,
the storage controllers may first write data into the separ-
ately addressable fast write storage on one more storage
devices.

In one embodiment, the storage controllers **125a**, **125b**
may initiate the use of erase blocks within and across storage
devices (e.g., **118**) in accordance with an age and expected
remaining lifespan of the storage devices, or based on other
statistics. The storage controllers **125a**, **125b** may initiate
garbage collection and data migration data between storage
devices in accordance with pages that are no longer needed
as well as to manage Flash page and erase block lifespans
and to manage overall system performance.

In one embodiment, the storage system **124** may utilize
mirroring and/or erasure coding schemes as part of storing
data into addressable fast write storage and/or as part of
writing data into allocation units associated with erase
blocks. Erasure codes may be used across storage devices, as
well as within erase blocks or allocation units, or within and
across Flash memory devices on a single storage device, to
provide redundancy against single or multiple storage device
failures or to protect against internal corruptions of Flash
memory pages resulting from Flash memory operations or
from degradation of Flash memory cells. Mirroring and
erasure coding at various levels may be used to recover from
multiple types of failures that occur separately or in com-
bination.

The embodiments depicted with reference to FIGS. 2A-G
illustrate a storage cluster that stores user data, such as user
data originating from one or more user or client systems or
other sources external to the storage cluster. The storage
cluster distributes user data across storage nodes housed
within a chassis, or across multiple chassis, using erasure
coding and redundant copies of metadata. Erasure coding
refers to a method of data protection or reconstruction in
which data is stored across a set of different locations, such
as disks, storage nodes or geographic locations. Flash
memory is one type of solid-state memory that may be
integrated with the embodiments, although the embodiments
may be extended to other types of solid-state memory or
other storage medium, including non-solid state memory.
Control of storage locations and workloads are distributed
across the storage locations in a clustered peer-to-peer
system. Tasks such as mediating communications between
the various storage nodes, detecting when a storage node has
become unavailable, and balancing I/Os (inputs and outputs)
across the various storage nodes, are all handled on a
distributed basis. Data is laid out or distributed across
multiple storage nodes in data fragments or stripes that
support data recovery in some embodiments. Ownership of
data can be reassigned within a cluster, independent of input
and output patterns. This architecture described in more
detail below allows a storage node in the cluster to fail, with

12

the system remaining operational, since the data can be
reconstructed from other storage nodes and thus remain
available for input and output operations. In various embodi-
ments, a storage node may be referred to as a cluster node,
a blade, or a server.

The storage cluster may be contained within a chassis,
i.e., an enclosure housing one or more storage nodes. A
mechanism to provide power to each storage node, such as
a power distribution bus, and a communication mechanism,
such as a communication bus that enables communication
between the storage nodes are included within the chassis.
The storage cluster can run as an independent system in one
location according to some embodiments. In one embodi-
ment, a chassis contains at least two instances of both the
power distribution and the communication bus which may
be enabled or disabled independently. The internal commu-
nication bus may be an Ethernet bus, however, other tech-
nologies such as PCIe, InfiniBand, and others, are equally
suitable. The chassis provides a port for an external com-
munication bus for enabling communication between mul-
tiple chassis, directly or through a switch, and with client
systems. The external communication may use a technology
such as Ethernet, InfiniBand, Fibre Channel, etc. In some
embodiments, the external communication bus uses different
communication bus technologies for inter-chassis and client
communication. If a switch is deployed within or between
chassis, the switch may act as a translation between multiple
protocols or technologies. When multiple chassis are con-
nected to define a storage cluster, the storage cluster may be
accessed by a client using either proprietary interfaces or
standard interfaces such as network file system ('NFS'),
common internet file system ('CIFS'), small computer sys-
tem interface ('SCSI') or hypertext transfer protocol
(('HTTP')). Translation from the client protocol may occur at
the switch, chassis external communication bus or within
each storage node. In some embodiments, multiple chassis
may be coupled or connected to each other through an
aggregator switch. A portion and/or all of the coupled or
connected chassis may be designated as a storage cluster. As
discussed above, each chassis can have multiple blades, each
blade has a media access control ('MAC') address, but the
storage cluster is presented to an external network as having
a single cluster IP address and a single MAC address in some
embodiments.

Each storage node may be one or more storage servers and
each storage server is connected to one or more non-volatile
solid state memory units, which may be referred to as
storage units or storage devices. One embodiment includes
a single storage server in each storage node and between one
to eight non-volatile solid state memory units, however this
one example is not meant to be limiting. The storage server
may include a processor, DRAM and interfaces for the
internal communication bus and power distribution for each
of the power buses. Inside the storage node, the interfaces
and storage unit share a communication bus, e.g., PCI
Express, in some embodiments. The non-volatile solid state
memory units may directly access the internal communica-
tion bus interface through a storage node communication
bus, or request the storage node to access the bus interface.
The non-volatile solid state memory unit contains an embed-
ded CPU, solid state storage controller, and a quantity of
solid state mass storage, e.g., between 2-32 terabytes ('TB')
in some embodiments. An embedded volatile storage
medium, such as DRAM, and an energy reserve apparatus
are included in the non-volatile solid state memory unit. In
some embodiments, the energy reserve apparatus is a capaci-
tor, super-capacitor, or battery that enables transferring a

13

subset of DRAM contents to a stable storage medium in the case of power loss. In some embodiments, the non-volatile solid state memory unit is constructed with a storage class memory, such as phase change or magnetoresistive random access memory ('MRAM') that substitutes for DRAM and enables a reduced power hold-up apparatus.

One of many features of the storage nodes and non-volatile solid state storage is the ability to proactively rebuild data in a storage cluster. The storage nodes and non-volatile solid state storage can determine when a storage node or non-volatile solid state storage in the storage cluster is unreachable, independent of whether there is an attempt to read data involving that storage node or non-volatile solid state storage. The storage nodes and non-volatile solid state storage then cooperate to recover and rebuild the data in at least partially new locations. This constitutes a proactive rebuild, in that the system rebuilds data without waiting until the data is needed for a read access initiated from a client system employing the storage cluster. These and further details of the storage memory and operation thereof are discussed below.

FIG. 2A is a perspective view of a storage cluster 161, with multiple storage nodes 150 and internal solid-state memory coupled to each storage node to provide network attached storage or storage area network, in accordance with some embodiments. A network attached storage, storage area network, or a storage cluster, or other storage memory, could include one or more storage clusters 161, each having one or more storage nodes 150, in a flexible and reconfigurable arrangement of both the physical components and the amount of storage memory provided thereby. The storage cluster 161 is designed to fit in a rack, and one or more racks can be set up and populated as desired for the storage memory. The storage cluster 161 has a chassis 138 having multiple slots 142. It should be appreciated that chassis 138 may be referred to as a housing, enclosure, or rack unit. In one embodiment, the chassis 138 has fourteen slots 142, although other numbers of slots are readily devised. For example, some embodiments have four slots, eight slots, sixteen slots, thirty-two slots, or other suitable number of slots. Each slot 142 can accommodate one storage node 150 in some embodiments. Chassis 138 includes flaps 148 that can be utilized to mount the chassis 138 on a rack. Fans 144 provide air circulation for cooling of the storage nodes 150 and components thereof, although other cooling components could be used, or an embodiment could be devised without cooling components. A switch fabric 146 couples storage nodes 150 within chassis 138 together and to a network for communication to the memory. In an embodiment depicted in herein, the slots 142 to the left of the switch fabric 146 and fans 144 are shown occupied by storage nodes 150, while the slots 142 to the right of the switch fabric 146 and fans 144 are empty and available for insertion of storage node 150 for illustrative purposes. This configuration is one example, and one or more storage nodes 150 could occupy the slots 142 in various further arrangements. The storage node arrangements need not be sequential or adjacent in some embodiments. Storage nodes 150 are hot pluggable, meaning that a storage node 150 can be inserted into a slot 142 in the chassis 138, or removed from a slot 142, without stopping or powering down the system. Upon insertion or removal of storage node 150 from slot 142, the system automatically reconfigures in order to recognize and adapt to the change. Reconfiguration, in some embodiments, includes restoring redundancy and/or rebalancing data or load.

14

Each storage node 150 can have multiple components. In the embodiment shown here, the storage node 150 includes a printed circuit board 159 populated by a CPU 156, i.e., processor, a memory 154 coupled to the CPU 156, and a non-volatile solid state storage 152 coupled to the CPU 156, although other mountings and/or components could be used in further embodiments. The memory 154 has instructions which are executed by the CPU 156 and/or data operated on by the CPU 156. As further explained below, the non-volatile solid state storage 152 includes flash or, in further embodiments, other types of solid-state memory.

Referring to FIG. 2A, storage cluster 161 is scalable, meaning that storage capacity with non-uniform storage sizes is readily added, as described above. One or more storage nodes 150 can be plugged into or removed from each chassis and the storage cluster self-configures in some embodiments. Plug-in storage nodes 150, whether installed in a chassis as delivered or later added, can have different sizes. For example, in one embodiment a storage node 150 can have any multiple of 4 TB, e.g., 8 TB, 12 TB, 16 TB, 32 TB, etc. In further embodiments, a storage node 150 could have any multiple of other storage amounts or capacities. Storage capacity of each storage node 150 is broadcast, and influences decisions of how to stripe the data. For maximum storage efficiency, an embodiment can self-configure as wide as possible in the stripe, subject to a predetermined requirement of continued operation with loss of up to one, or up to two, non-volatile solid state storage units 152 or storage nodes 150 within the chassis.

FIG. 2B is a block diagram showing a communications interconnect 171A-F and power distribution bus 172 coupling multiple storage nodes 150. Referring back to FIG. 2A, the communications interconnect 171A-F can be included in or implemented with the switch fabric 146 in some embodiments. Where multiple storage clusters 161 occupy a rack, the communications interconnect 171A-F can be included in or implemented with a top of rack switch, in some embodiments. As illustrated in FIG. 2B, storage cluster 161 is enclosed within a single chassis 138. External port 176 is coupled to storage nodes 150 through communications interconnect 171A-F, while external port 174 is coupled directly to a storage node. External power port 178 is coupled to power distribution bus 172. Storage nodes 150 may include varying amounts and differing capacities of non-volatile solid state storage 152 as described with reference to FIG. 2A. In addition, one or more storage nodes 150 may be a compute only storage node as illustrated in FIG. 2B. Authorities 168 are implemented on the non-volatile solid state storages 152, for example as lists or other data structures stored in memory. In some embodiments the authorities are stored within the non-volatile solid state storage 152 and supported by software executing on a controller or other processor of the non-volatile solid state storage 152. In a further embodiment, authorities 168 are implemented on the storage nodes 150, for example as lists or other data structures stored in the memory 154 and supported by software executing on the CPU 156 of the storage node 150. Authorities 168 control how and where data is stored in the non-volatile solid state storages 152 in some embodiments. This control assists in determining which type of erasure coding scheme is applied to the data, and which storage nodes 150 have which portions of the data. Each authority 168 may be assigned to a non-volatile solid state storage 152. Each authority may control a range of inode numbers, segment numbers, or other data identifiers which are

15

assigned to data by a file system, by the storage nodes **150**, or by the non-volatile solid state storage **152**, in various embodiments.

Every piece of data, and every piece of metadata, has redundancy in the system in some embodiments. In addition, every piece of data and every piece of metadata has an owner, which may be referred to as an authority. If that authority is unreachable, for example through failure of a storage node, there is a plan of succession for how to find that data or that metadata. In various embodiments, there are redundant copies of authorities **168**. Authorities **168** have a relationship to storage nodes **150** and non-volatile solid state storage **152** in some embodiments. Each authority **168**, covering a range of data segment numbers or other identifiers of the data, may be assigned to a specific non-volatile solid state storage **152**. In some embodiments the authorities **168** for all of such ranges are distributed over the non-volatile solid state storages **152** of a storage cluster. Each storage node **150** has a network port that provides access to the non-volatile solid state storage(s) **152** of that storage node **150**. Data can be stored in a segment, which is associated with a segment number and that segment number is an indirection for a configuration of a RAID (redundant array of independent disks) stripe in some embodiments. The assignment and use of the authorities **168** thus establishes an indirection to data. Indirection may be referred to as the ability to reference data indirectly, in this case via an authority **168**, in accordance with some embodiments. A segment identifies a set of non-volatile solid state storage **152** and a local identifier into the set of non-volatile solid state storage **152** that may contain data. In some embodiments, the local identifier is an offset into the device and may be reused sequentially by multiple segments. In other embodiments the local identifier is unique for a specific segment and never reused. The offsets in the non-volatile solid state storage **152** are applied to locating data for writing to or reading from the non-volatile solid state storage **152** (in the form of a RAID stripe). Data is striped across multiple units of non-volatile solid state storage **152**, which may include or be different from the non-volatile solid state storage **152** having the authority **168** for a particular data segment.

If there is a change in where a particular segment of data is located, e.g., during a data move or a data reconstruction, the authority **168** for that data segment should be consulted, at that non-volatile solid state storage **152** or storage node **150** having that authority **168**. In order to locate a particular piece of data, embodiments calculate a hash value for a data segment or apply an inode number or a data segment number. The output of this operation points to a non-volatile solid state storage **152** having the authority **168** for that particular piece of data. In some embodiments there are two stages to this operation. The first stage maps an entity identifier (ID), e.g., a segment number, inode number, or directory number to an authority identifier. This mapping may include a calculation such as a hash or a bit mask. The second stage is mapping the authority identifier to a particular non-volatile solid state storage **152**, which may be done through an explicit mapping. The operation is repeatable, so that when the calculation is performed, the result of the calculation repeatably and reliably points to a particular non-volatile solid state storage **152** having that authority **168**. The operation may include the set of reachable storage nodes as input. If the set of reachable non-volatile solid state storage units changes the optimal set changes. In some embodiments, the persisted value is the current assignment (which is always true) and the calculated value is the target

16

assignment the cluster will attempt to reconfigure towards. This calculation may be used to determine the optimal non-volatile solid state storage **152** for an authority in the presence of a set of non-volatile solid state storage **152** that are reachable and constitute the same cluster. The calculation also determines an ordered set of peer non-volatile solid state storage **152** that will also record the authority to non-volatile solid state storage mapping so that the authority may be determined even if the assigned non-volatile solid state storage is unreachable. A duplicate or substitute authority **168** may be consulted if a specific authority **168** is unavailable in some embodiments.

With reference to FIGS. 2A and 2B, two of the many tasks of the CPU **156** on a storage node **150** are to break up write data, and reassemble read data. When the system has determined that data is to be written, the authority **168** for that data is located as above. When the segment ID for data is already determined the request to write is forwarded to the non-volatile solid state storage **152** currently determined to be the host of the authority **168** determined from the segment. The host CPU **156** of the storage node **150**, on which the non-volatile solid state storage **152** and corresponding authority **168** reside, then breaks up or shards the data and transmits the data out to various non-volatile solid state storage **152**. The transmitted data is written as a data stripe in accordance with an erasure coding scheme. In some embodiments, data is requested to be pulled, and in other embodiments, data is pushed. In reverse, when data is read, the authority **168** for the segment ID containing the data is located as described above. The host CPU **156** of the storage node **150** on which the non-volatile solid state storage **152** and corresponding authority **168** reside requests the data from the non-volatile solid state storage and corresponding storage nodes pointed to by the authority. In some embodiments the data is read from flash storage as a data stripe. The host CPU **156** of storage node **150** then reassembles the read data, correcting any errors (if present) according to the appropriate erasure coding scheme, and forwards the reassembled data to the network. In further embodiments, some or all of these tasks can be handled in the non-volatile solid state storage **152**. In some embodiments, the segment host requests the data be sent to storage node **150** by requesting pages from storage and then sending the data to the storage node making the original request.

In some systems, for example in UNIX-style file systems, data is handled with an index node or inode, which specifies a data structure that represents an object in a file system. The object could be a file or a directory, for example. Metadata may accompany the object, as attributes such as permission data and a creation timestamp, among other attributes. A segment number could be assigned to all or a portion of such an object in a file system. In other systems, data segments are handled with a segment number assigned elsewhere. For purposes of discussion, the unit of distribution is an entity, and an entity can be a file, a directory or a segment. That is, entities are units of data or metadata stored by a storage system. Entities are grouped into sets called authorities. Each authority has an authority owner, which is a storage node that has the exclusive right to update the entities in the authority. In other words, a storage node contains the authority, and that the authority, in turn, contains entities.

A segment is a logical container of data in accordance with some embodiments. A segment is an address space between medium address space and physical flash locations, i.e., the data segment number, are in this address space. Segments may also contain meta-data, which enable data redundancy to be restored (rewritten to different flash loca-

tions or devices) without the involvement of higher level software. In one embodiment, an internal format of a segment contains client data and medium mappings to determine the position of that data. Each data segment is protected, e.g., from memory and other failures, by breaking the segment into a number of data and parity shards, where applicable. The data and parity shards are distributed, i.e., striped, across non-volatile solid state storage **152** coupled to the host CPUs **156** (See FIGS. **2E** and **2G**) in accordance with an erasure coding scheme. Usage of the term segments refers to the container and its place in the address space of segments in some embodiments. Usage of the term stripe refers to the same set of shards as a segment and includes how the shards are distributed along with redundancy or parity information in accordance with some embodiments.

A series of address-space transformations takes place across an entire storage system. At the top are the directory entries (file names) which link to an inode. Inodes point into medium address space, where data is logically stored. Medium addresses may be mapped through a series of indirect mediums to spread the load of large files, or implement data services like deduplication or snapshots. Medium addresses may be mapped through a series of indirect mediums to spread the load of large files, or implement data services like deduplication or snapshots. Segment addresses are then translated into physical flash locations. Physical flash locations have an address range bounded by the amount of flash in the system in accordance with some embodiments. Medium addresses and segment addresses are logical containers, and in some embodiments use a 128 bit or larger identifier so as to be practically infinite, with a likelihood of reuse calculated as longer than the expected life of the system. Addresses from logical containers are allocated in a hierarchical fashion in some embodiments. Initially, each non-volatile solid state storage unit **152** may be assigned a range of address space. Within this assigned range, the non-volatile solid state storage **152** is able to allocate addresses without synchronization with other non-volatile solid state storage **152**.

Data and metadata is stored by a set of underlying storage layouts that are optimized for varying workload patterns and storage devices. These layouts incorporate multiple redundancy schemes, compression formats and index algorithms. Some of these layouts store information about authorities and authority masters, while others store file metadata and file data. The redundancy schemes include error correction codes that tolerate corrupted bits within a single storage device (such as a NAND flash chip), erasure codes that tolerate the failure of multiple storage nodes, and replication schemes that tolerate data center or regional failures. In some embodiments, low density parity check ('LDPC') code is used within a single storage unit. Reed-Solomon encoding is used within a storage cluster, and mirroring is used within a storage grid in some embodiments. Metadata may be stored using an ordered log structured index (such as a Log Structured Merge Tree), and large data may not be stored in a log structured layout.

In order to maintain consistency across multiple copies of an entity, the storage nodes agree implicitly on two things through calculations: (1) the authority that contains the entity, and (2) the storage node that contains the authority. The assignment of entities to authorities can be done by pseudo randomly assigning entities to authorities, by splitting entities into ranges based upon an externally produced key, or by placing a single entity into each authority. Examples of pseudorandom schemes are linear hashing and the Replication Under Scalable Hashing ('RUSH') family of

hashes, including Controlled Replication Under Scalable Hashing ('CRUSH'). In some embodiments, pseudo-random assignment is utilized only for assigning authorities to nodes because the set of nodes can change. The set of authorities cannot change so any subjective function may be applied in these embodiments. Some placement schemes automatically place authorities on storage nodes, while other placement schemes rely on an explicit mapping of authorities to storage nodes. In some embodiments, a pseudorandom scheme is utilized to map from each authority to a set of candidate authority owners. A pseudorandom data distribution function related to CRUSH may assign authorities to storage nodes and create a list of where the authorities are assigned. Each storage node has a copy of the pseudorandom data distribution function, and can arrive at the same calculation for distributing, and later finding or locating an authority. Each of the pseudorandom schemes requires the reachable set of storage nodes as input in some embodiments in order to conclude the same target nodes. Once an entity has been placed in an authority, the entity may be stored on physical devices so that no expected failure will lead to unexpected data loss. In some embodiments, rebalancing algorithms attempt to store the copies of all entities within an authority in the same layout and on the same set of machines.

Examples of expected failures include device failures, stolen machines, datacenter fires, and regional disasters, such as nuclear or geological events. Different failures lead to different levels of acceptable data loss. In some embodiments, a stolen storage node impacts neither the security nor the reliability of the system, while depending on system configuration, a regional event could lead to no loss of data, a few seconds or minutes of lost updates, or even complete data loss.

In the embodiments, the placement of data for storage redundancy is independent of the placement of authorities for data consistency. In some embodiments, storage nodes that contain authorities do not contain any persistent storage. Instead, the storage nodes are connected to non-volatile solid state storage units that do not contain authorities. The communications interconnect between storage nodes and non-volatile solid state storage units consists of multiple communication technologies and has non-uniform performance and fault tolerance characteristics. In some embodiments, as mentioned above, non-volatile solid state storage units are connected to storage nodes via PCI express, storage nodes are connected together within a single chassis using Ethernet backplane, and chassis are connected together to form a storage cluster. Storage clusters are connected to clients using Ethernet or fiber channel in some embodiments. If multiple storage clusters are configured into a storage grid, the multiple storage clusters are connected using the Internet or other long-distance networking links, such as a "metro scale" link or private link that does not traverse the internet.

Authority owners have the exclusive right to modify entities, to migrate entities from one non-volatile solid state storage unit to another non-volatile solid state storage unit, and to add and remove copies of entities. This allows for maintaining the redundancy of the underlying data. When an authority owner fails, is going to be decommissioned, or is overloaded, the authority is transferred to a new storage node. Transient failures make it non-trivial to ensure that all non-faulty machines agree upon the new authority location. The ambiguity that arises due to transient failures can be achieved automatically by a consensus protocol such as Paxos, hot-warm failover schemes, via manual intervention

by a remote system administrator, or by a local hardware administrator (such as by physically removing the failed machine from the cluster, or pressing a button on the failed machine). In some embodiments, a consensus protocol is used, and failover is automatic. If too many failures or replication events occur in too short a time period, the system goes into a self-preservation mode and halts replication and data movement activities until an administrator intervenes in accordance with some embodiments.

As authorities are transferred between storage nodes and authority owners update entities in their authorities, the system transfers messages between the storage nodes and non-volatile solid state storage units. With regard to persistent messages, messages that have different purposes are of different types. Depending on the type of the message, the system maintains different ordering and durability guarantees. As the persistent messages are being processed, the messages are temporarily stored in multiple durable and non-durable storage hardware technologies. In some embodiments, messages are stored in RAM, NVRAM and on NAND flash devices, and a variety of protocols are used in order to make efficient use of each storage medium. Latency-sensitive client requests may be persisted in replicated NVRAM, and then later NAND, while background rebalancing operations are persisted directly to NAND.

Persistent messages are persistently stored prior to being transmitted. This allows the system to continue to serve client requests despite failures and component replacement. Although many hardware components contain unique identifiers that are visible to system administrators, manufacturer, hardware supply chain and ongoing monitoring quality control infrastructure, applications running on top of the infrastructure address virtualize addresses. These virtualized addresses do not change over the lifetime of the storage system, regardless of component failures and replacements. This allows each component of the storage system to be replaced over time without reconfiguration or disruptions of client request processing, i.e., the system supports non-disruptive upgrades.

In some embodiments, the virtualized addresses are stored with sufficient redundancy. A continuous monitoring system correlates hardware and software status and the hardware identifiers. This allows detection and prediction of failures due to faulty components and manufacturing details. The monitoring system also enables the proactive transfer of authorities and entities away from impacted devices before failure occurs by removing the component from the critical path in some embodiments.

FIG. 2C is a multiple level block diagram, showing contents of a storage node 150 and contents of a non-volatile solid state storage 152 of the storage node 150. Data is communicated to and from the storage node 150 by a network interface controller ('NIC') 202 in some embodiments. Each storage node 150 has a CPU 156, and one or more non-volatile solid state storage 152, as discussed above. Moving down one level in FIG. 2C, each non-volatile solid state storage 152 has a relatively fast non-volatile solid state memory, such as nonvolatile random access memory ('NVRAM') 204, and flash memory 206. In some embodiments, NVRAM 204 may be a component that does not require program/erase cycles (DRAM, MRAM, PCM), and can be a memory that can support being written vastly more often than the memory is read from. Moving down another level in FIG. 2C, the NVRAM 204 is implemented in one embodiment as high speed volatile memory, such as dynamic random access memory (DRAM) 216, backed up by energy reserve 218. Energy reserve 218 provides suffi-

cient electrical power to keep the DRAM 216 powered long enough for contents to be transferred to the flash memory 206 in the event of power failure. In some embodiments, energy reserve 218 is a capacitor, super-capacitor, battery, or other device, that supplies a suitable supply of energy sufficient to enable the transfer of the contents of DRAM 216 to a stable storage medium in the case of power loss. The flash memory 206 is implemented as multiple flash dies 222, which may be referred to as packages of flash dies 222 or an array of flash dies 222. It should be appreciated that the flash dies 222 could be packaged in any number of ways, with a single die per package, multiple dies per package (i.e. multichip packages), in hybrid packages, as bare dies on a printed circuit board or other substrate, as encapsulated dies, etc. In the embodiment shown, the non-volatile solid state storage 152 has a controller 212 or other processor, and an input output (I/O) port 210 coupled to the controller 212. I/O port 210 is coupled to the CPU 156 and/or the network interface controller 202 of the flash storage node 150. Flash input output (I/O) port 220 is coupled to the flash dies 222, and a direct memory access unit (DMA) 214 is coupled to the controller 212, the DRAM 216 and the flash dies 222. In the embodiment shown, the I/O port 210, controller 212, DMA unit 214 and flash I/O port 220 are implemented on a programmable logic device ('PLD') 208, e.g., a field programmable gate array (FPGA). In this embodiment, each flash die 222 has pages, organized as sixteen kB (kilobyte) pages 224, and a register 226 through which data can be written to or read from the flash die 222. In further embodiments, other types of solid-state memory are used in place of, or in addition to flash memory illustrated within flash die 222.

Storage clusters 161, in various embodiments as disclosed herein, can be contrasted with storage arrays in general. The storage nodes 150 are part of a collection that creates the storage cluster 161. Each storage node 150 owns a slice of data and computing required to provide the data. Multiple storage nodes 150 cooperate to store and retrieve the data. Storage memory or storage devices, as used in storage arrays in general, are less involved with processing and manipulating the data. Storage memory or storage devices in a storage array receive commands to read, write, or erase data. The storage memory or storage devices in a storage array are not aware of a larger system in which they are embedded, or what the data means. Storage memory or storage devices in storage arrays can include various types of storage memory, such as RAM, solid state drives, hard disk drives, etc. The storage units 152 described herein have multiple interfaces active simultaneously and serving multiple purposes. In some embodiments, some of the functionality of a storage node 150 is shifted into a storage unit 152, transforming the storage unit 152 into a combination of storage unit 152 and storage node 150. Placing computing (relative to storage data) into the storage unit 152 places this computing closer to the data itself. The various system embodiments have a hierarchy of storage node layers with different capabilities. By contrast, in a storage array, a controller owns and knows everything about all of the data that the controller manages in a shelf or storage devices. In a storage cluster 161, as described herein, multiple controllers in multiple storage units 152 and/or storage nodes 150 cooperate in various ways (e.g., for erasure coding, data sharding, metadata communication and redundancy, storage capacity expansion or contraction, data recovery, and so on).

FIG. 2D shows a storage server environment, which uses embodiments of the storage nodes 150 and storage units 152 of FIGS. 2A-C. In this version, each storage unit 152 has a

21

processor such as controller **212** (see FIG. 2C), an FPGA (field programmable gate array), flash memory **206**, and NVRAM **204** (which is super-capacitor backed DRAM **216**, see FIGS. 2B and 2C) on a PCIe (peripheral component interconnect express) board in a chassis **138** (see FIG. 2A). The storage unit **152** may be implemented as a single board containing storage, and may be the largest tolerable failure domain inside the chassis. In some embodiments, up to two storage units **152** may fail and the device will continue with no data loss.

The physical storage is divided into named regions based on application usage in some embodiments. The NVRAM **204** is a contiguous block of reserved memory in the storage unit **152** DRAM **216**, and is backed by NAND flash. NVRAM **204** is logically divided into multiple memory regions written for two as spool (e.g., spool_region). Space within the NVRAM **204** spools is managed by each authority **168** independently. Each device provides an amount of storage space to each authority **168**. That authority **168** further manages lifetimes and allocations within that space. Examples of a spool include distributed transactions or notions. When the primary power to a storage unit **152** fails, onboard super-capacitors provide a short duration of power hold up. During this holdup interval, the contents of the NVRAM **204** are flushed to flash memory **206**. On the next power-on, the contents of the NVRAM **204** are recovered from the flash memory **206**.

As for the storage unit controller, the responsibility of the logical “controller” is distributed across each of the blades containing authorities **168**. This distribution of logical control is shown in FIG. 2D as a host controller **242**, mid-tier controller **244** and storage unit controller(s) **246**. Management of the control plane and the storage plane are treated independently, although parts may be physically co-located on the same blade. Each authority **168** effectively serves as an independent controller. Each authority **168** provides its own data and metadata structures, its own background workers, and maintains its own lifecycle.

FIG. 2E is a blade **252** hardware block diagram, showing a control plane **254**, compute and storage planes **256**, **258**, and authorities **168** interacting with underlying physical resources, using embodiments of the storage nodes **150** and storage units **152** of FIGS. 2A-C in the storage server environment of FIG. 2D. The control plane **254** is partitioned into a number of authorities **168** which can use the compute resources in the compute plane **256** to run on any of the blades **252**. The storage plane **258** is partitioned into a set of devices, each of which provides access to flash **206** and NVRAM **204** resources.

In the compute and storage planes **256**, **258** of FIG. 2E, the authorities **168** interact with the underlying physical resources (i.e., devices). From the point of view of an authority **168**, its resources are striped over all of the physical devices. From the point of view of a device, it provides resources to all authorities **168**, irrespective of where the authorities happen to run. Each authority **168** has allocated or has been allocated one or more partitions **260** of storage memory in the storage units **152**, e.g. partitions **260** in flash memory **206** and NVRAM **204**. Each authority **168** uses those allocated partitions **260** that belong to it, for writing or reading user data. Authorities can be associated with differing amounts of physical storage of the system. For example, one authority **168** could have a larger number of partitions **260** or larger sized partitions **260** in one or more storage units **152** than one or more other authorities **168**.

FIG. 2F depicts elasticity software layers in blades **252** of a storage cluster, in accordance with some embodiments. In

22

the elasticity structure, elasticity software is symmetric, i.e., each blade's compute module **270** runs the three identical layers of processes depicted in FIG. 2F. Storage managers **274** execute read and write requests from other blades **252** for data and metadata stored in local storage unit **152** NVRAM **204** and flash **206**. Authorities **168** fulfill client requests by issuing the necessary reads and writes to the blades **252** on whose storage units **152** the corresponding data or metadata resides. Endpoints **272** parse client connection requests received from switch fabric **146** supervisory software, relay the client connection requests to the authorities **168** responsible for fulfillment, and relay the authorities' **168** responses to clients. The symmetric three-layer structure enables the storage system's high degree of concurrency. Elasticity scales out efficiently and reliably in these embodiments. In addition, elasticity implements a unique scale-out technique that balances work evenly across all resources regardless of client access pattern, and maximizes concurrency by eliminating much of the need for inter-blade coordination that typically occurs with conventional distributed locking.

Still referring to FIG. 2F, authorities **168** running in the compute modules **270** of a blade **252** perform the internal operations required to fulfill client requests. One feature of elasticity is that authorities **168** are stateless, i.e., they cache active data and metadata in their own blades' **252** DRAMs for fast access, but the authorities store every update in their NVRAM **204** partitions on three separate blades **252** until the update has been written to flash **206**. All the storage system writes to NVRAM **204** are in triplicate to partitions on three separate blades **252** in some embodiments. With triple-mirrored NVRAM **204** and persistent storage protected by parity and Reed-Solomon RAID checksums, the storage system can survive concurrent failure of two blades **252** with no loss of data, metadata, or access to either.

Because authorities **168** are stateless, they can migrate between blades **252**. Each authority **168** has a unique identifier. NVRAM **204** and flash **206** partitions are associated with authorities' **168** identifiers, not with the blades **252** on which they are running in some. Thus, when an authority **168** migrates, the authority **168** continues to manage the same storage partitions from its new location. When a new blade **252** is installed in an embodiment of the storage cluster, the system automatically rebalances load by: partitioning the new blade's **252** storage for use by the system's authorities **168**, migrating selected authorities **168** to the new blade **252**, starting endpoints **272** on the new blade **252** and including them in the switch fabric's **146** client connection distribution algorithm.

From their new locations, migrated authorities **168** persist the contents of their NVRAM **204** partitions on flash **206**, process read and write requests from other authorities **168**, and fulfill the client requests that endpoints **272** direct to them. Similarly, if a blade **252** fails or is removed, the system redistributes its authorities **168** among the system's remaining blades **252**. The redistributed authorities **168** continue to perform their original functions from their new locations.

FIG. 2G depicts authorities **168** and storage resources in blades **252** of a storage cluster, in accordance with some embodiments. Each authority **168** is exclusively responsible for a partition of the flash **206** and NVRAM **204** on each blade **252**. The authority **168** manages the content and integrity of its partitions independently of other authorities **168**. Authorities **168** compress incoming data and preserve it temporarily in their NVRAM **204** partitions, and then consolidate, RAID-protect, and persist the data in segments

23

of the storage in their flash **206** partitions. As the authorities **168** write data to flash **206**, storage managers **274** perform the necessary flash translation to optimize write performance and maximize media longevity. In the background, authorities **168** “garbage collect,” or reclaim space occupied by data that clients have made obsolete by overwriting the data. It should be appreciated that since authorities’ **168** partitions are disjoint, there is no need for distributed locking to execute client and writes or to perform background functions.

The embodiments described herein may utilize various software, communication and/or networking protocols. In addition, the configuration of the hardware and/or software may be adjusted to accommodate various protocols. For example, the embodiments may utilize Active Directory, which is a database based system that provides authentication, directory, policy, and other services in a WINDOWS’ environment. In these embodiments, LDAP (Lightweight Directory Access Protocol) is one example application protocol for querying and modifying items in directory service providers such as Active Directory. In some embodiments, a network lock manager (‘NLM’) is utilized as a facility that works in cooperation with the Network File System (‘NFS’) to provide a System V style of advisory file and record locking over a network. The Server Message Block (‘SMB’) protocol, one version of which is also known as Common Internet File System (‘CIFS’), may be integrated with the storage systems discussed herein. SMP operates as an application-layer network protocol typically used for providing shared access to files, printers, and serial ports and miscellaneous communications between nodes on a network. SMB also provides an authenticated inter-process communication mechanism. AMAZON™ S3 (Simple Storage Service) is a web service offered by Amazon Web Services, and the systems described herein may interface with Amazon S3 through web services interfaces (REST (representational state transfer), SOAP (simple object access protocol), and BitTorrent). A RESTful API (application programming interface) breaks down a transaction to create a series of small modules. Each module addresses a particular underlying part of the transaction. The control or permissions provided with these embodiments, especially for object data, may include utilization of an access control list (‘ACL’). The ACL is a list of permissions attached to an object and the ACL specifies which users or system processes are granted access to objects, as well as what operations are allowed on given objects. The systems may utilize Internet Protocol version 6 (‘IPv6’), as well as IPv4, for the communications protocol that provides an identification and location system for computers on networks and routes traffic across the Internet. The routing of packets between networked systems may include Equal-cost multi-path routing (‘ECMP’), which is a routing strategy where next-hop packet forwarding to a single destination can occur over multiple “best paths” which tie for top place in routing metric calculations. Multi-path routing can be used in conjunction with most routing protocols, because it is a per-hop decision limited to a single router. The software may support Multi-tenancy, which is an architecture in which a single instance of a software application serves multiple customers. Each customer may be referred to as a tenant. Tenants may be given the ability to customize some parts of the application, but may not customize the application’s code, in some embodiments. The embodiments may maintain audit logs. An audit log is a document that records an event in a computing system. In addition to documenting what resources were accessed, audit log entries typically include destination and

24

source addresses, a timestamp, and user login information for compliance with various regulations. The embodiments may support various key management policies, such as encryption key rotation. In addition, the system may support dynamic root passwords or some variation dynamically changing passwords.

FIG. 3A sets forth a diagram of a storage system **306** that is coupled for data communications with a cloud services provider **302** in accordance with some embodiments of the present disclosure. Although depicted in less detail, the storage system **306** depicted in FIG. 3A may be similar to the storage systems described above with reference to FIGS. 1A-1D and FIGS. 2A-2G. In some embodiments, the storage system **306** depicted in FIG. 3A may be embodied as a storage system that includes imbalanced active/active controllers, as a storage system that includes balanced active/active controllers, as a storage system that includes active/active controllers where less than all of each controller’s resources are utilized such that each controller has reserve resources that may be used to support failover, as a storage system that includes fully active/active controllers, as a storage system that includes dataset-segregated controllers, as a storage system that includes dual-layer architectures with front-end controllers and back-end integrated storage controllers, as a storage system that includes scale-out clusters of dual-controller arrays, as well as combinations of such embodiments.

In the example depicted in FIG. 3A, the storage system **306** is coupled to the cloud services provider **302** via a data communications link **304**. The data communications link **304** may be embodied as a dedicated data communications link, as a data communications pathway that is provided through the use of one or data communications networks such as a wide area network (‘WAN’) or local area network (‘LAN’), or as some other mechanism capable of transporting digital information between the storage system **306** and the cloud services provider **302**. Such a data communications link **304** may be fully wired, fully wireless, or some aggregation of wired and wireless data communications pathways. In such an example, digital information may be exchanged between the storage system **306** and the cloud services provider **302** via the data communications link **304** using one or more data communications protocols. For example, digital information may be exchanged between the storage system **306** and the cloud services provider **302** via the data communications link **304** using the handheld device transfer protocol (‘HDTP’), hypertext transfer protocol (‘HTTP’), internet protocol (‘IP’), real-time transfer protocol (‘RTP’), transmission control protocol (‘TCP’), user datagram protocol (‘UDP’), wireless application protocol (‘WAP’), or other protocol.

The cloud services provider **302** depicted in FIG. 3A may be embodied, for example, as a system and computing environment that provides services to users of the cloud services provider **302** through the sharing of computing resources via the data communications link **304**. The cloud services provider **302** may provide on-demand access to a shared pool of configurable computing resources such as computer networks, servers, storage, applications and services, and so on. The shared pool of configurable resources may be rapidly provisioned and released to a user of the cloud services provider **302** with minimal management effort. Generally, the user of the cloud services provider **302** is unaware of the exact computing resources utilized by the cloud services provider **302** to provide the services. Although in many cases such a cloud services provider **302** may be accessible via the Internet, readers of skill in the art

will recognize that any system that abstracts the use of shared resources to provide services to a user through any data communications link may be considered a cloud services provider 302.

In the example depicted in FIG. 3A, the cloud services provider 302 may be configured to provide a variety of services to the storage system 306 and users of the storage system 306 through the implementation of various service models. For example, the cloud services provider 302 may be configured to provide services to the storage system 306 and users of the storage system 306 through the implementation of an infrastructure as a service ('IaaS') service model where the cloud services provider 302 offers computing infrastructure such as virtual machines and other resources as a service to subscribers. In addition, the cloud services provider 302 may be configured to provide services to the storage system 306 and users of the storage system 306 through the implementation of a platform as a service ('PaaS') service model where the cloud services provider 302 offers a development environment to application developers. Such a development environment may include, for example, an operating system, programming-language execution environment, database, web server, or other components that may be utilized by application developers to develop and run software solutions on a cloud platform. Furthermore, the cloud services provider 302 may be configured to provide services to the storage system 306 and users of the storage system 306 through the implementation of a software as a service ('SaaS') service model where the cloud services provider 302 offers application software, databases, as well as the platforms that are used to run the applications to the storage system 306 and users of the storage system 306, providing the storage system 306 and users of the storage system 306 with on-demand software and eliminating the need to install and run the application on local computers, which may simplify maintenance and support of the application. The cloud services provider 302 may be further configured to provide services to the storage system 306 and users of the storage system 306 through the implementation of an authentication as a service ('AaaS') service model where the cloud services provider 302 offers authentication services that can be used to secure access to applications, data sources, or other resources. The cloud services provider 302 may also be configured to provide services to the storage system 306 and users of the storage system 306 through the implementation of a storage as a service model where the cloud services provider 302 offers access to its storage infrastructure for use by the storage system 306 and users of the storage system 306. Readers will appreciate that the cloud services provider 302 may be configured to provide additional services to the storage system 306 and users of the storage system 306 through the implementation of additional service models, as the service models described above are included only for explanatory purposes and in no way represent a limitation of the services that may be offered by the cloud services provider 302 or a limitation as to the service models that may be implemented by the cloud services provider 302.

In the example depicted in FIG. 3A, the cloud services provider 302 may be embodied, for example, as a private cloud, as a public cloud, or as a combination of a private cloud and public cloud. In an embodiment in which the cloud services provider 302 is embodied as a private cloud, the cloud services provider 302 may be dedicated to providing services to a single organization rather than providing services to multiple organizations. In an embodiment where the cloud services provider 302 is embodied as a public

cloud, the cloud services provider 302 may provide services to multiple organizations. Public cloud and private cloud deployment models may differ and may come with various advantages and disadvantages. For example, because a public cloud deployment involves the sharing of a computing infrastructure across different organization, such a deployment may not be ideal for organizations with security concerns, mission-critical workloads, uptime requirements demands, and so on. While a private cloud deployment can address some of these issues, a private cloud deployment may require on-premises staff to manage the private cloud. In still alternative embodiments, the cloud services provider 302 may be embodied as a mix of a private and public cloud services with a hybrid cloud deployment.

Although not explicitly depicted in FIG. 3A, readers will appreciate that additional hardware components and additional software components may be necessary to facilitate the delivery of cloud services to the storage system 306 and users of the storage system 306. For example, the storage system 306 may be coupled to (or even include) a cloud storage gateway. Such a cloud storage gateway may be embodied, for example, as hardware-based or software-based appliance that is located on premise with the storage system 306. Such a cloud storage gateway may operate as a bridge between local applications that are executing on the storage array 306 and remote, cloud-based storage that is utilized by the storage array 306. Through the use of a cloud storage gateway, organizations may move primary iSCSI or NAS to the cloud services provider 302, thereby enabling the organization to save space on their on-premises storage systems. Such a cloud storage gateway may be configured to emulate a disk array, a block-based device, a file server, or other storage system that can translate the SCSI commands, file server commands, or other appropriate command into REST-space protocols that facilitate communications with the cloud services provider 302.

In order to enable the storage system 306 and users of the storage system 306 to make use of the services provided by the cloud services provider 302, a cloud migration process may take place during which data, applications, or other elements from an organization's local systems (or even from another cloud environment) are moved to the cloud services provider 302. In order to successfully migrate data, applications, or other elements to the cloud services provider's 302 environment, middleware such as a cloud migration tool may be utilized to bridge gaps between the cloud services provider's 302 environment and an organization's environment. Such cloud migration tools may also be configured to address potentially high network costs and long transfer times associated with migrating large volumes of data to the cloud services provider 302, as well as addressing security concerns associated with sensitive data to the cloud services provider 302 over data communications networks. In order to further enable the storage system 306 and users of the storage system 306 to make use of the services provided by the cloud services provider 302, a cloud orchestrator may also be used to arrange and coordinate automated tasks in pursuit of creating a consolidated process or workflow. Such a cloud orchestrator may perform tasks such as configuring various components, whether those components are cloud components or on-premises components, as well as managing the interconnections between such components. The cloud orchestrator can simplify the inter-component communication and connections to ensure that links are correctly configured and maintained.

In the example depicted in FIG. 3A, and as described briefly above, the cloud services provider 302 may be

configured to provide services to the storage system **306** and users of the storage system **306** through the usage of a SaaS service model where the cloud services provider **302** offers application software, databases, as well as the platforms that are used to run the applications to the storage system **306** and users of the storage system **306**, providing the storage system **306** and users of the storage system **306** with on-demand software and eliminating the need to install and run the application on local computers, which may simplify maintenance and support of the application. Such applications may take many forms in accordance with various embodiments of the present disclosure. For example, the cloud services provider **302** may be configured to provide access to data analytics applications to the storage system **306** and users of the storage system **306**. Such data analytics applications may be configured, for example, to receive telemetry data phoned home by the storage system **306**. Such telemetry data may describe various operating characteristics of the storage system **306** and may be analyzed, for example, to determine the health of the storage system **306**, to identify workloads that are executing on the storage system **306**, to predict when the storage system **306** will run out of various resources, to recommend configuration changes, hardware or software upgrades, workflow migrations, or other actions that may improve the operation of the storage system **306**.

The cloud services provider **302** may also be configured to provide access to virtualized computing environments to the storage system **306** and users of the storage system **306**. Such virtualized computing environments may be embodied, for example, as a virtual machine or other virtualized computer hardware platforms, virtual storage devices, virtualized computer network resources, and so on. Examples of such virtualized environments can include virtual machines that are created to emulate an actual computer, virtualized desktop environments that separate a logical desktop from a physical machine, virtualized file systems that allow uniform access to different types of concrete file systems, and many others.

For further explanation, FIG. 3B sets forth a diagram of a storage system **306** in accordance with some embodiments of the present disclosure. Although depicted in less detail, the storage system **306** depicted in FIG. 3B may be similar to the storage systems described above with reference to FIGS. 1A-1D and FIGS. 2A-2G as the storage system may include many of the components described above.

The storage system **306** depicted in FIG. 3B may include storage resources **308**, which may be embodied in many forms. For example, in some embodiments the storage resources **308** can include nano-RAM or another form of nonvolatile random access memory that utilizes carbon nanotubes deposited on a substrate. In some embodiments, the storage resources **308** may include 3D crosspoint non-volatile memory in which bit storage is based on a change of bulk resistance, in conjunction with a stackable cross-gridded data access array. In some embodiments, the storage resources **308** may include flash memory, including single-level cell ('SLC') NAND flash, multi-level cell ('MLC') NAND flash, triple-level cell ('TLC') NAND flash, quad-level cell ('QLC') NAND flash, and others. In some embodiments, the storage resources **308** may include non-volatile magnetoresistive random-access memory ('MRAM'), including spin transfer torque ('STT') MRAM, in which data is stored through the use of magnetic storage elements. In some embodiments, the example storage resources **308** may include non-volatile phase-change memory ('PCM') that may have the ability to hold multiple bits in a single cell

as cells can achieve a number of distinct intermediary states. In some embodiments, the storage resources **308** may include quantum memory that allows for the storage and retrieval of photonic quantum information. In some embodiments, the example storage resources **308** may include resistive random-access memory ('ReRAM') in which data is stored by changing the resistance across a dielectric solid-state material. In some embodiments, the storage resources **308** may include storage class memory ('SCM') in which solid-state nonvolatile memory may be manufactured at a high density using some combination of sub-lithographic patterning techniques, multiple bits per cell, multiple layers of devices, and so on. Readers will appreciate that other forms of computer memories and storage devices may be utilized by the storage systems described above, including DRAM, SRAM, EEPROM, universal memory, and many others. The storage resources **308** depicted in FIG. 3A may be embodied in a variety of form factors, including but not limited to, dual in-line memory modules ('DIMMs'), non-volatile dual in-line memory modules ('NVDIMMs'), M.2, U.2, and others.

The example storage system **306** depicted in FIG. 3B may implement a variety of storage architectures. For example, storage systems in accordance with some embodiments of the present disclosure may utilize block storage where data is stored in blocks, and each block essentially acts as an individual hard drive. Storage systems in accordance with some embodiments of the present disclosure may utilize object storage, where data is managed as objects. Each object may include the data itself, a variable amount of metadata, and a globally unique identifier, where object storage can be implemented at multiple levels (e.g., device level, system level, interface level). Storage systems in accordance with some embodiments of the present disclosure utilize file storage in which data is stored in a hierarchical structure. Such data may be saved in files and folders, and presented to both the system storing it and the system retrieving it in the same format.

The example storage system **306** depicted in FIG. 3B may be embodied as a storage system in which additional storage resources can be added through the use of a scale-up model, additional storage resources can be added through the use of a scale-out model, or through some combination thereof. In a scale-up model, additional storage may be added by adding additional storage devices. In a scale-out model, however, additional storage nodes may be added to a cluster of storage nodes, where such storage nodes can include additional processing resources, additional networking resources, and so on.

The storage system **306** depicted in FIG. 3B also includes communications resources **310** that may be useful in facilitating data communications between components within the storage system **306**, as well as data communications between the storage system **306** and computing devices that are outside of the storage system **306**. The communications resources **310** may be configured to utilize a variety of different protocols and data communication fabrics to facilitate data communications between components within the storage systems as well as computing devices that are outside of the storage system. For example, the communications resources **310** can include fibre channel ('FC') technologies such as FC fabrics and FC protocols that can transport SCSI commands over FC networks. The communications resources **310** can also include FC over ethernet ('FCoE') technologies through which FC frames are encapsulated and transmitted over Ethernet networks. The communications resources **310** can also include InfiniBand

(‘IB’) technologies in which a switched fabric topology is utilized to facilitate transmissions between channel adapters. The communications resources **310** can also include NVM Express (‘NVMe’) technologies and NVMe over fabrics (‘NVMeoF’) technologies through which non-volatile storage media attached via a PCI express (‘PCIe’) bus may be accessed. The communications resources **310** can also include mechanisms for accessing storage resources **308** within the storage system **306** utilizing serial attached SCSI (‘SAS’), serial ATA (‘SATA’) bus interfaces for connecting storage resources **308** within the storage system **306** to host bus adapters within the storage system **306**, internet small computer systems interface (‘iSCSI’) technologies to provide block-level access to storage resources **308** within the storage system **306**, and other communications resources that that may be useful in facilitating data communications between components within the storage system **306**, as well as data communications between the storage system **306** and computing devices that are outside of the storage system **306**.

The storage system **306** depicted in FIG. 3B also includes processing resources **312** that may be useful in useful in executing computer program instructions and performing other computational tasks within the storage system **306**. The processing resources **312** may include one or more application-specific integrated circuits (‘ASICs’) that are customized for some particular purpose as well as one or more central processing units (‘CPUs’). The processing resources **312** may also include one or more digital signal processors (‘DSPs’), one or more field-programmable gate arrays (‘FPGAs’), one or more systems on a chip (‘SoCs’), or other form of processing resources **312**. The storage system **306** may utilize the storage resources **312** to perform a variety of tasks including, but not limited to, supporting the execution of software resources **314** that will be described in greater detail below.

The storage system **306** depicted in FIG. 3B also includes software resources **314** that, when executed by processing resources **312** within the storage system **306**, may perform various tasks. The software resources **314** may include, for example, one or more modules of computer program instructions that when executed by processing resources **312** within the storage system **306** are useful in carrying out various data protection techniques to preserve the integrity of data that is stored within the storage systems. Readers will appreciate that such data protection techniques may be carried out, for example, by system software executing on computer hardware within the storage system, by a cloud services provider, or in other ways. Such data protection techniques can include, for example, data archiving techniques that cause data that is no longer actively used to be moved to a separate storage device or separate storage system for long-term retention, data backup techniques through which data stored in the storage system may be copied and stored in a distinct location to avoid data loss in the event of equipment failure or some other form of catastrophe with the storage system, data replication techniques through which data stored in the storage system is replicated to another storage system such that the data may be accessible via multiple storage systems, data snapshotting techniques through which the state of data within the storage system is captured at various points in time, data and database cloning techniques through which duplicate copies of data and databases may be created, and other data protection techniques. Through the use of such data protection techniques, business continuity and disaster recovery

objectives may be met as a failure of the storage system may not result in the loss of data stored in the storage system.

The software resources **314** may also include software that is useful in implementing software-defined storage (‘SDS’). In such an example, the software resources **314** may include one or more modules of computer program instructions that, when executed, are useful in policy-based provisioning and management of data storage that is independent of the underlying hardware. Such software resources **314** may be useful in implementing storage virtualization to separate the storage hardware from the software that manages the storage hardware.

The software resources **314** may also include software that is useful in facilitating and optimizing I/O operations that are directed to the storage resources **308** in the storage system **306**. For example, the software resources **314** may include software modules that perform carry out various data reduction techniques such as, for example, data compression, data deduplication, and others. The software resources **314** may include software modules that intelligently group together I/O operations to facilitate better usage of the underlying storage resource **308**, software modules that perform data migration operations to migrate from within a storage system, as well as software modules that perform other functions. Such software resources **314** may be embodied as one or more software containers or in many other ways.

Readers will appreciate that the various components depicted in FIG. 3B may be grouped into one or more optimized computing packages as converged infrastructures. Such converged infrastructures may include pools of computers, storage and networking resources that can be shared by multiple applications and managed in a collective manner using policy-driven processes. Such converged infrastructures may minimize compatibility issues between various components within the storage system **306** while also reducing various costs associated with the establishment and operation of the storage system **306**. Such converged infrastructures may be implemented with a converged infrastructure reference architecture, with standalone appliances, with a software driven hyper-converged approach, or in other ways.

Readers will appreciate that the storage system **306** depicted in FIG. 3B may be useful for supporting various types of software applications. For example, the storage system **306** may be useful in supporting artificial intelligence applications, database applications, DevOps projects, electronic design automation tools, event-driven software applications, high performance computing applications, simulation applications, high-speed data capture and analysis applications, machine learning applications, media production applications, media serving applications, picture archiving and communication systems (‘PACS’) applications, software development applications, and many other types of applications by providing storage resources to such applications.

For further explanation, FIG. 3C sets forth diagrams of storage systems **320** and **330** implementing storage efficiency driven migration in accordance with some embodiments of the present disclosure. Storage systems **320** and **330** may each implement data storage features described above with reference to FIGS. 1A-1D, 2A-2G, 3A, and 3B, as storage systems **320** and **330** may include some or all of the components described above—in addition to components, modules, or logic for implementing features of storage efficiency driven data migration.

31

In some computer environments, different storage systems may service different types of workloads, where similar workload types may generate similar type data. Further, in some cases, data of a similar type may be reduced through deduplication or through compression or through some other technique better, or with higher resulting storage space efficiency, than data of dissimilar types. For example, storage system 320 may service one or more workloads that generate one type of data, depicted as data 324 stored within data store 322, and one or more workloads that generate a different type of data, depicted as data 328 stored within data store 326. As one example, data 324 may be video data and data 326 may be web analytics data within a database, and the data store may be a virtual storage volume, a physical storage device, or some other type of data store.

Similarly, storage system 330 may service one or more workloads that generate one type of data, depicted as data 334 stored within data store 332, and one or more workloads that generate a different type of data, depicted as data 338 stored within data store 336. In this example, the type of data for data 334 is similar to the type of data for data 324, and the type of data for data 338 is similar to the type of data for data 328.

Further in this example, because of the lack of similarity between data types for data 324 and data 328 within storage system 320, an application of a technique to reduce storage capacity applied across all data stored within all data stores of storage system 320 may have limited or marginal results. However, if a technique to reduce storage capacity were applied to data 324 and data 334, then because of a similarity in the types of data for data 324 and data 334, the technique to reduce storage capacity would produce greater reductions in storage capacity than if the technique to reduce storage capacity were applied to data 324 and data 328. Therefore, if data 324 were migrated to storage system 330 or if data 334 were migrated to storage system 320, then the combined footprint of storage space used by both data 324 and 334 would be smaller than if no migration were performed. Similarly, if data 328 were migrated to storage system 330 or if data 338 were migrated to storage system 320, then the combined footprint of storage space used by both data 328 and 338 would be smaller than if no migration were performed. While in this example all of data 334 may be migrated to improve storage space usage, in other examples, one or more portions of data 334 may be migrated, where the one or more portions may be sufficient to satisfy a threshold level of storage space usage reduction. In other words, in some examples, less than the entire amount of similar data may be migrated, and the reasons may include limited bandwidth, limited computing resources to perform the migration, or an expected increase in resource utilization that may be hampered by a migration of a large quantity of data.

Because of the improvements to the reduction to storage use that would result from considering data 324 and data 334 together when applying a technique for reducing storage capacity, where data 324 and data 334 are stored, respectively, on different storage systems 320 and 330, if data 334 is migrated to storage system 320, or if data 324 is migrated to storage system 330, then the total storage requirements for data 324 and data 334 are smaller than if data 324 and data 334 are stored, as they presently are, on different storage systems. The bottom half of FIG. 3C depicts the state of storage system 320 and storage system 330 subsequent to migrating data 334 from storage system 330 to storage system 320 and migrating data 328 from storage system 320 to storage system 330. Further, in general, data selected to be

32

migrated may be condensed, compressed, or deduplicated prior to migration, thereby reducing a quantity of data being transmitted over a network or communication link.

As described further below, in some examples, detection of a possibility for reducing overall storage usage across multiple systems based on a migration of similar data may be described within a notification provided to a user. For example, a data management console may include a user interface that may display an alert, or some other user interface element describing benefits to migrating data between storage systems, where the benefits may include reduced overall storage usage, load balancing improvements, or other implications from migrating data. In this example, a user may also be notified of resources that may be consumed in migrating the data, or of a timeline or schedule to complete the migration, where a user may then authorize the migration of data or not authorize the migration of data. In short, a user may simply click a button to initiate a migration of similar data to reduce overall storage capacity usage—without deleting data or making data less accessible.

In different embodiments, described in greater detail below, storage system 320 may generate or calculate a set of deduplication hashes for data stored within each of the data stores within storage system 320. Further, storage system 320 may determine an expected amount of storage capacity reduction based on the calculated set of deduplication hashes if deduplication were applied to data 324 and data 328. To determine whether or not a data migration would result in reduced storage capacity, storage system 320 may receive or request a set of deduplication hashes from one or more other storage systems, which in this example includes storage system 330. Given the one or more sets of deduplication hashes from other systems, storage system 320 may compare the set of deduplication hashes for locally stored data against the one or more sets of deduplication hashes for data stored on the one or more other storage systems. In this way, storage system 320 may determine which combinations of data using a deduplication technique correspond to the greatest data storage reductions, or some reduction greater than a threshold amount of data storage reduction, and determine one or more migrations of data to be able to apply the deduplication technique to a given combination of data.

In other examples, instead of, or in addition to, a storage system determining to migrate data, individual storage systems may communicate with a data management service, where the data management service provides a user interface for managing data stored across the multiple storage systems, and where the data management service communicates with one or more managed storage systems to determine whether or not a migration of data would result in an overall reduction in data storage usage. In one example, the data management service may be implemented as a cloud-based service, where the storage systems may either be cloud-based, physically or virtually, or implemented within a local computing environment, physically or virtually.

While in this example, it is beneficial to swap data 328 for data 334 or swap data 324 for data 338—in other words, two migrations would be performed to achieve a swap—in other examples, a single migration of data may be identified, where the single migration of data itself is sufficient to achieve a reduction of storage space usage. In other examples, where a greater number of storage systems are present, there may be multiple migrations of data from multiple sources of storage systems onto a target storage

system to achieve reductions is storage space based on improving the performance of a data reduction technique as applied to similar data.

In this way, storage efficiency driven migration may provide an advantage of a smaller overall storage usage across multiple storage systems after migration of similar data, where the multiple storage systems may be distinct or independent from one another.

For further explanation, FIG. 3D sets forth diagrams of storage systems 350 and 354A-354N implementing storage efficiency driven migration in accordance with some embodiments of the present disclosure. Storage systems 350 and 354A-354N may each implement data storage features described above with reference to FIGS. 1A-1D, 2A-2G, and 3A-3C, as storage systems 320 and 330 may include some or all of the components described above—in addition to components, modules, or logic for implementing features of storage efficiency driven data migration.

Storage system 350 may appear to a client device to be a single storage system providing a single unified storage space, where storage system 350 may invisibly include multiple storage systems that may be directly connected to each other or connected over a network. In other words, the individual storage systems 354A-354N are scaled out to appear as a larger, cohesive storage system 350. Further, any given storage system 354A-354N may include a respective storage array, where an I/O operation received by storage system 350 may be relayed to any given storage system 354A-354N. As depicted in FIG. 3D, storage system 350 may include multiple storage systems 354A-354N, where each of the multiple storage systems 354A-354N may include a distinct and independent storage array that may communicate over network 352. Further, for each storage system 354A-354N, each corresponding storage array may further provide a user access to one or more data stores or data volumes, depicted as volumes 356A-356M within storage system 354A and volumes 358A-358P within storage system 354N. In this example, computing device(s) 340 may communicate directly to storage system 350, or computing device(s) 342 may communicate with storage system 350 over network 344.

In this example, it may be that any given volume stored among storage system 354A-354N within storage system 350 is stored on a single storage system. As such, storage efficiency driven migration may be implemented between any two or more storage systems within storage system 350 similarly to the implementation, described above with reference to FIG. 3C, of storage efficiency driven migration between storage system 320 and storage system 330. In short, the scope of data reduction techniques may be an individual storage system, and therefore, if data that is outside the scope of a particular storage system because the data is stored on a different storage system is moved from the different storage system to the particular storage system, the overall data storage used by the migrated data and the data on the particular storage system—subsequent to application of one or more data reduction techniques on the migrated data and the data on the particular storage—is less than prior to the migration. As described above, in some cases, an entire volume may be migrated, and in other cases, less than an entire volume may be migrated.

Further, in the course of servicing I/O operations, storage system 350 may migrate volumes for various reasons, including balancing network activity, workload balancing, among other reasons described below with reference to FIG. 9. In this context, storage system 350 may determine that migrating a volume 356A from storage system 354A to

storage system 354N may improve workload balance or network traffic. However, if storage system 350 determines that such a migration would result in greater use of storage space resulting from a data reduction technique being unable to generate similar data reduction results, then storage system 350 may determine that the cost of greater storage space usage—as measured across the multiple storage systems—outweighs the benefits of a move for other reasons, and consequently, storage system 350 may abort the data migration or determine not to migrate the data. In a converse scenario, storage system 350 may, according to implementations of storage efficiency driven migration, determine to prioritize migration of data between storage systems due to storage capacity constraints even if a consequence is poorer performance with regard to other system metrics, such as network latency or I/O operations handled per unit time, or some other metric. In other words, because storage capacity may be reaching a maximum capacity or critically low capacity, other system activities for improving other metrics are deprioritized relative to storage efficiency driven migration to avoid running out of storage capacity.

In other examples, storage system 350, instead of migrating data subsequent to storage, may select a storage system among storage systems 354A-354N in which to store incoming I/O operation data based at least in part on relative improvements in storage space utilization from applying a data reduction technique to the incoming I/O operation data with previously stored data.

For further explanation, FIG. 3E sets forth a flow chart illustrating an example method for synchronizing a plurality of storage systems 354A and 354B according to some embodiments of the present disclosure. The example method depicted in FIG. 3E includes two storage systems 354A and 354B, labeled as a leader storage system 354A and a follower storage system 354B. Although depicted in less detail, the storage systems 354A and 354B depicted in FIG. 3E may be similar to the storage arrays and storage systems described above with reference to FIGS. 1-3D, the storage clusters described above with reference to FIGS. 1-3D, or in some other way.

The designation of one storage system as the ‘leader’ and another storage system as the ‘follower’ may refer to the respective relationships of each storage system for the purposes of synchronously replicating data across the storage systems. In such an example, and as will be described in greater detail below, the leader storage system 354A may be responsible for performing some processing of an incoming I/O operation and passing such information along to the follower storage system 354B, acknowledging the completion of a synchronously replicated I/O operation to the issuer of the I/O operation, or performing other tasks that are not required of the follower storage system 354B. The leader storage system 354A may be responsible for performing tasks that are not required of the follower storage system 354B for all incoming I/O operations or, alternatively, the leader-follower relationship may be specific to only a subset of the I/O operations that are received by either storage system. For example, the leader-follower relationship may be specific to I/O operations that are directed towards a first volume, a first group of volumes, a first group of logical addresses, a first group of physical addresses, or some other logical or physical delineator. In such a way, a first storage system may serve as the leader storage system for I/O operations directed to a first set of volumes (or other delineator) while a second storage system may serve as the leader storage system for I/O operations directed to a second set of volumes (or other delineator). The example method

35

depicted in FIG. 3E depicts an embodiment where synchronizing a plurality of storage systems 354A and 354B occurs in response to the receipt of an I/O operation by the leader storage system 354A, although synchronizing a plurality of storage systems 354A and 354B may also be carried out in response to the receipt of an I/O operation by the follower storage system 354B, as will be described in greater detail below.

In this example method for synchronizing a plurality of storage systems, data corresponding to I/O operations that are directed towards a particular volume, set of volumes, or other some other delineator, may be considered a dataset that is synchronously replicated across the plurality of storage systems. As described below with reference to FIG. 3F, a dataset that is synchronously replicated across storage systems may be expanded to include additional storage systems among which data is synchronously replicated, where the additional storage systems are then considered together when implementing storage efficiency driven migration.

The example method depicted in FIG. 3E includes receiving (366), by the leader storage system 354A, an I/O operation 364. The I/O operation 364 may be embodied, for example, as a request to write data to a location within the storage system 350, as a request to copy data from a first location within the storage system 350 to a second location within the storage system 350, as a request to take a snapshot of a portion of the storage system 350 and store such a snapshot within the storage system 350, or as some other operation that results in a change to some portion of storage resources within the storage system 350. In the example method depicted in FIG. 3E, the I/O operation 364 is issued by a host 362 that may be embodied, for example, as an application that is executing on a virtual machine, as an application that is executing on a computing device that is connected to the storage system 350, or as some other entity configured to access the storage system 350.

The example method depicted in FIG. 3E also includes generating (368), by the leader storage system 354A, I/O processing information 370. The I/O processing information 370 may be embodied, for example, as system-level information that is used to describe an I/O operation that is to be performed by a storage system. The leader storage system 354A may generate (368) I/O processing information 370 by processing the I/O operation 364 just enough to figure out what should happen in order to service the I/O operation 364. For example, the leader storage system 354A may determine whether some ordering of the execution of the I/O operation 364 relative to other I/O operations is required to produce an equivalent result on each storage system. Consider an example in which the I/O operation 364 is embodied as a request to take a snapshot of a particular volume that is replicated on each storage system 354A and 354B. In such an example, if three write operations (write A, write B, write C) that were directed to the particular volume have been serviced by the leader storage system 354A, the follower storage system 354B will also need to service the same three write operations before servicing the I/O operation 364 (i.e., the request to take a snapshot of the particular volume that is replicated on each storage system 354A and 354B in order for the snapshot taken by the leader storage system 354A to match the snapshot taken by the follower storage system 354B. In such an example, the I/O processing information 370 generated (368) by the leader storage system 354A may include information indicating that the follower storage system 354B should complete the execution of write A, write B, and write C before servicing the I/O operation 364.

36

The example method depicted in FIG. 3E also includes sending (372), from the leader storage system 354A to the follower storage system 354B, the I/O processing information 370 along with any I/O payload 384 that is associated with the I/O operation 364. The I/O payload 384 may be embodied, for example, as data that is to be written to storage within the storage system 350 when the I/O operation 364 is embodied as a request to write data to the storage system 350. In such an example, because the I/O operation was received (366) by the leader storage system 354A, the follower storage system 354B has not received the I/O payload 384 associated with the I/O operation 364. In the example method depicted in FIG. 3E, the I/O processing information 370 and the I/O payload 384 that is associated with the I/O operation 364 may be sent (372) from the leader storage system 354A to the follower storage system 354B via a data communications network that couples the leader storage system 354A to the follower storage system 354B, via a data communications link, for example a dedicated data communications link, that couples the leader storage system 354A to the follower storage system 354B, or via some other mechanism.

The example method depicted in FIG. 3E also includes receiving (386), by the follower storage system 354B, I/O processing information 370 and I/O payload 384 from the leader storage system 354A. The follower storage system 354B may receive (386) the I/O processing information 370 and I/O payload 384 from the leader storage system 354A, for example, via one or more messages that are sent from the leader storage system 354A to the follower storage system 354B. The one or more messages may be sent from the leader storage system 354A to the follower storage system 354B via one or more dedicated data communications links between the two storage systems 354A and 354B, by the leader storage system 354A writing the message to a predetermined memory location (e.g., the location of a queue) on the follower storage system 354B using RDMA or a similar mechanism, or in other ways.

The example method depicted in FIG. 3E also includes servicing (388) the I/O operation 364 on the follower storage system 354B. In the example method depicted in FIG. 3E, the follower storage system 354B may service (388) the I/O operation 364 by completing the I/O operation 364 using the I/O processing information 370 and the I/O payload 384 that was received from the leader storage system 354A. In such an example, the I/O operation 364 may be considered to have been completed and successfully serviced, for example, when the I/O payload 384 has been committed to persistent storage within the follower storage system 354B.

The example method depicted in FIG. 3E also includes acknowledging (390), by the follower storage system 354B, completion of the I/O operation 364 to the leader storage system 354A. The follower storage system 354B may acknowledge (390) completion of the I/O operation 364 to the leader storage system 354A, for example, through the use of one or more messages sent from the follower storage system 354B to the leader storage system 354A. Such messages may include, for example, information identifying the particular I/O operation 364 that was completed as well as any additional information useful in acknowledging (390) the completion of the I/O operation 364 by the follower storage system 354B. In the example method depicted in FIG. 3E, acknowledging (390) completion of the I/O operation 364 to the leader storage system 354A is illustrated by the follower storage system 354B issuing an acknowledgment 392 message to the leader storage system 354A.

37

The example method depicted in FIG. 3E also includes servicing (374), by the leader storage system 354A, the I/O operation 364 on the leader storage system 354A. In the example method depicted in FIG. 3E, the leader storage system 354A may service (374) the I/O operation 364 by completing the I/O operation 364 using the I/O processing information 370 and the I/O payload 384. In such an example, the I/O operation 364 may be considered to have been completed and successfully serviced, for example, when the I/O payload 384 has been committed to persistent storage within the leader storage system 354A.

The example method depicted in FIG. 3E also includes determining (378), by the leader storage system 354A, whether the I/O operation 364 has been serviced on the follower storage system 354B. The leader storage system 354A may determine (378) whether the I/O operation 364 has been serviced on the follower storage system 354B, for example, by determining whether the leader storage system 354A has received an acknowledgment message or other message from the follower storage system 354B indicating that the follower storage system 354B has completed servicing the I/O operation 364. In such an example, if the leader storage system 354A affirmatively (380) determines that the I/O operation 364 has been serviced by the follower storage system 354B, the leader storage system 354A may proceed by acknowledging (382) the completion of the I/O operation 364 to the host 362 that initiated the I/O operation 364. If the leader storage system 354A determines that the I/O operation 364 has not (376) been serviced by the follower storage system 354B, however, the leader storage system 354A may not yet acknowledge (382) the completion of the I/O operation 364 to the host 362 that initiated the I/O operation 364 as the storage system 350 may only acknowledge (382) the completion of the I/O operation 364 to the host 362 that initiated the I/O operation 364 when the I/O operation 364 has been successfully serviced on both storage systems 354A and 354B. In the example method depicted in FIG. 3E, acknowledging (382) completion of the I/O operation 364 may be carried out through the use of one or more acknowledgement 394 messages that are sent from the leader storage system 354A to the host 362 or via some other appropriate mechanism.

Readers will appreciate that once the I/O processing information 370 and any associated I/O payload 384 has been received (386) by the follower storage system 354B, each storage system 354A and 354B may begin processing the I/O operation 364 independently. As such, it is possible that the follower storage system 354B may even service (388) the I/O operation 364 and acknowledge (390) completion of the I/O operation 364 to the leader storage system 354A prior to the leader storage system 354A servicing (374) the I/O operation 364. As such, the only requirement that must be satisfied before the leader storage system 354A acknowledges (382) completion of the I/O operation 364 to the host 362 is that the leader storage system 354A has confirmed that both storage systems 354A and 354B have successfully completed servicing the I/O operation 364. There is no requirement that one storage system completes servicing the I/O operation before the other storage system completes servicing the I/O operation.

Readers will further appreciate that although the example depicted in FIG. 3E includes the leader storage system 354A generating (368) I/O processing information 370 and sending (372) the I/O processing information 370 along with any I/O payload 384 that is associated with the I/O operation 364 to the follower storage system 354B, in alternative embodiments the leader storage system 354A may simply forward

38

the I/O operation 364 to the follower storage system 354B. Such an embodiment may require each storage system 354A and 354B to reproduce work done by the other storage system, but such an embodiment may be desirable for various reasons.

For further explanation, FIG. 3F sets forth a diagram of storage system 350, where storage system 350 includes storage systems 354A-354N, and where storage systems 350 and 354A-354N implement storage efficiency driven migration in accordance with some embodiments of the present disclosure. Storage systems 350 and 354A-354N may each implement data storage features described above with reference to FIGS. 1A-1D, 2A-2G, and 3A-3E, as storage systems 320 and 330 may include some or all of the components described above—in addition to components, modules, or logic for implementing features of storage efficiency driven data migration. In this example, storage system 354A includes volumes 397A and 397M, storage system 354B includes volume 398A and 397M, and storage system 354N includes volumes 399A and 399P.

FIG. 3F depicts a dataset, dataset 395, where dataset 395 represents data that is synchronously replicated across multiple storage systems as described above with reference to FIG. 3E. In this example, dataset 395 represents synchronously replicated data across storage systems 354A and 354B, as depicted in FIG. 3E. However, in some examples, if data stored on storage system 354N is similar to the data within dataset 395 that is being synchronously replicated, then if dataset 395 is extended, or stretched, to include storage system 354N, then the synchronously replicated data within dataset 395 becomes available for application of one or more data reduction techniques as applied to data within dataset 395 on storage system 354N as well as data stored on storage system 354N that is not within dataset 395. In other words, by extending the scope of the synchronously replicated data within dataset 395, additional storage capacity reductions may be possible.

Further, a storage system may be identified as being a candidate for extending a dataset based on factors that include one or more of: servicing similar workload types, storing similar data, or a quantity of matching deduplication hashes. Further, extension of a dataset may be considered according to factors described below similar to determining whether data on different storage systems is similar enough to satisfy a threshold level of similarity, where if data on a potential storage system satisfies the threshold level of similarity to data within a dataset on other storage systems, then in response a dataset may be extended from the other storage systems to include the potential storage system.

For further explanation, FIG. 4 sets forth a flow chart illustrating an example method for storage efficiency driven migration according to some embodiments of the present disclosure. Although depicted in less detail, the storage system (402) depicted in FIG. 4 may be similar to the storage systems described above with reference to FIGS. 1A-1D, FIGS. 2A-2G, FIGS. 3A-3C, or any combination thereof. Further, the storage system depicted in FIG. 4 may include the same, fewer, or additional components as the storage systems described above.

The example method depicted in FIG. 4 includes determining (404) a level of similarity between first data 452 stored on a first storage system 402 and second data 456 stored on second storage system 454. Determining (404) a level of similarity between first data 452 and second data 456 may be implemented using different techniques. In some cases, a level of similarity may be represented as a percentage value indicating a percent similar. One technique

includes applying a deduplication algorithm and determining a level of similarity between data as described below with reference to FIG. 5, where first storage system 402 calculates a set of deduplication hashes for first data 452 and compares the first set of deduplication hashes with a second set of deduplication hashes based on second data 456 stored on storage system 454. In this first technique, different quantities of matching hashes from the different sets of deduplication hashes for the different data stored on the different storage systems may indicate different levels of similarity between the data, where a level of similarity is correlated to a quantity of matching hashes. Further, the level of similarity may serve as a basis for determining whether or not migrating data would result in more benefits than costs. In some examples, determining (404) a level of similarity may be performed by a secondary controller within a multiple controller storage array within first storage system 402, where the secondary controller is further described with reference to FIG. 1A above.

Another technique for determining (404) a level of similarity 458 between first data 452 and second data 456 may be implemented by first storage system 402 determining respective quantities of storage usage corresponding to respective types of data and comparing the respective quantities of storage usage and the corresponding respective types of data against respective quantities of storage usage corresponding to respective types of data stored on second storage system 454. In this example, each storage system may maintain a log that correlates quantities of stored data with a type for the stored data, and any particular storage system may request such log information from other storage systems. In this way, storage system 402, or some given storage system or some given cloud-based management system receiving the log information, may determine, based on a total of data storage usage of a data type that matches, or is similar to, data types on another storage system, that using a particular one or more algorithms or techniques for reducing storage space use may result in lower storage space use on a particular storage system. For example, if a data compression technique is particularly effective on image data, and if a large amount of stored data on the different systems is image data, then applying the data compression technique—after migrating the image data onto a single storage system—would reduce storage space usage as compared to not migrating the image data and applying the data compression technique independently on the respective image data on the separate storage systems. In other examples, instead of, or in addition to, data compression, other data reduction techniques may be applied, including data deduplication.

In another technique for determining (404) a level of similarity 458 between first data 452 and second data 456 may be implemented by first storage system 402 determining respective quantities of storage usage corresponding to respective workload types that generate the data and comparing the respective quantities of storage usage that correspond to the respective workload types on first storage system 402 against respective quantities of storage usage that correspond to the respective workload types on second storage system 454. For example, if a portion of the stored data on first storage system 402 was generated by a particular database service workload type, and if another portion of stored data on the second storage system 402 was also generated by a similar database service, then this similarity of workload types may serve as a basis for determining a level of similarity.

In another technique for determining (404) a level of similarity 458, an entity that is independent of any given storage system, such as a cloud-based data storage management service, may query or receive information that characterizes data stored on a given storage system, where the characterization data may serve as a basis for determining levels of similarity of data among different storage systems. For example, the characterization data may include one or more of: sets of deduplication hashes, types and quantity of data, or quantities of data corresponding to workload types.

The example method depicted in FIG. 4 also includes determining (406), in dependence upon the level of similarity 458, that an expected amount of storage space reduction from migrating similar data exceeds a threshold level 460 may be implemented by comparing a percentage value representing the level of similarity with a threshold level. Further, determining a threshold level may be implemented using different techniques. In one technique, where the level of similarity is based on a quantity of matching hash values between deduplication hash values of first data 452, a threshold level may be specified as a matching percentage of hash values, where a matching percentage may be a default value or a value specified by a user or administrator.

With regard to determining a threshold level, if the level of similarity is small, then the resources used in migrating the data may outweigh potential benefits of the reduction in storage space usage. As one example, if, based on a level of similarity, an expected reduction is data storage usage after migration is 3%, then the 3% expected reduction in storage space usage is compared to a threshold value determined to be a threshold above which benefits of a migration outweigh the resources used in migrating the data. In one example implementation, this threshold value may be 5%, however, the threshold value may be specified to be any value, and the threshold value may further be variable and dependent on current system workloads. For example, on an idle system, the threshold value may be correspondingly scaled lower, and on a busy system, the threshold value may be correspondingly scaled higher.

Generally, for each of these techniques for determining a threshold level, a matching percentage may be determined dynamically based on one or more metrics such as network latencies, bandwidth restrictions, time of day, current storage system workloads, or predicted storage system workload, where a matching percentage may be increased or decreased in dependence upon one or more of these or other metrics. For example, a matching percentage may initially be set to 10%, where if metrics indicate that storage system is lightly loaded, and is expected to remain lightly loaded for a window of time in which a migration may occur, then the matching percentage may be lowered to 5%. Generally, a matching percentage may be correlated to a resource impact of a migration, where the lower a resource impact of a migration is determined to be, then the lower the matching percentage is defined to be, where a resource impact of a migration may be defined according to one or more of the aforementioned metrics.

The example method depicted in FIG. 4 also includes, responsive to determining (406) that the expected amount of storage space reduction exceeds the threshold level, initiating (408) a migration of one or more portions of the first data 452 from the first storage system 402 to the second storage system 454. Initiating (408) a migration of one or more portions of the first data 452 from the first storage system 402 to the second storage system 454 may be implemented using different techniques. Further, the one or more portions of the first data may be all of the first data, a quantity

41

sufficient to meet threshold level **460**, or a quantity that corresponds to a unit of size resulting in a more efficient data migration as compared to other units of size. In one technique, initiating **(408)** the migration may include generating a notification indicating one or more aspects of a proposed data migration, and providing the notification to a user interface, such as a data management console, where a migration may be initiated in response to an authorization received from a user approving the data migration proposed via the management console. A notification may include one or more of: an amount of data to be migrated, an expected reduction in overall storage capacity, an expected amount of time or resources required for the data migration, levels of storage capacity subsequent to the proposed data migration, and the storage systems involved in the proposed data migration. In some cases, a user may authorize the entire proposed data migration, or the user may authorize a portion of the proposed data migration, where the quantity authorized is the quantity that is migrated.

In another technique, initiating **(408)** the migration of the one or more portions of the first data **452** from the first storage system **402** to the second storage system **454** may be implemented by automatically initiating a data migration in response to determining **(406)** that the expected amount of storage space reduction exceeds the threshold level **460**.

In this way, a specific advantage for a given set of independent storage systems, subsequent to a data migration of the one or more portions of first data **452** among the set of independent storage systems, is that the overall use of storage capacity among the set of independent storage systems is less than prior to migration of the one or more portions of first data **452**. In different cases, a storage system may be considered independent from another storage system based on being implemented on different storage area networks, being implemented at different geographic sites, or some other factor.

For further explanation, FIG. 5 sets forth a flow chart illustrating an example method for storage efficiency driven migration according to some embodiments of the present disclosure. The example method depicted in FIG. 5 is similar to the example method depicted in FIG. 4, as the example method depicted in FIG. 5 also includes: determining **(404)** a level of similarity between first data **452** stored on a first storage system **402** and second data **456** stored on a second storage system **454**; determining **(406)**, in dependence upon the level of similarity **458**, that an expected amount of storage space reduction from migrating similar data exceeds a threshold level **460**; and responsive to determining **(406)** that the expected amount of storage space reduction exceeds the threshold level, initiating **(408)** a migration of one or more portions of the first data **452** from the first storage system **402** to the second storage system **454**.

However, the example method depicted in FIG. 5 further specifies determining **(404)** the level of similarity between first data **452** stored on first storage system **402** and second data **456** stored on second storage system **454** to include: calculating **(502)** a first set of deduplication hashes **552** based on first data **452** stored on first storage system **402**; receiving **(504)** a second set of deduplication hashes **554** from second storage system **454**; and comparing **(506)** first set of deduplication hashes **552** to second set of deduplication hashes **554** to determine a quantity of matching hash values between hash values from first set of deduplication hashes **552** and hash values from second set of deduplication hashes **554**, where the level of similarity is dependent upon the quantity of matching hash values.

42

Calculating **(502)** first set of deduplication hashes **552** may be implemented by a management module implemented within storage system **402** selecting a particular deduplication algorithm from among multiple possible deduplication algorithms, and applying the deduplication algorithm some or all of the stored data within a storage system. In this example, the particular deduplication algorithm may be applied to first data **456** to calculate first set of deduplication hashes **554**. Similarly, any storage system participating within the data storage efficiency data migration may calculate **(502)** a corresponding set of deduplication hashes. Further, any given storage system from among the participating storage systems may serve as a leader, where a leader receives, or requests, information from other storage systems to determine a level of similarity, where in this example, the information includes sets of deduplication hashes. A leader may be designated as a leader for a period of time, through a voting procedure, or some other method, where loss of a leader may trigger a round of voting to determine a new leader storage system.

Receiving **(504)** second set of deduplication hashes from second storage system **454** may be implemented by storage system **402** receiving, at a communication port, a message from second storage system **454**, where the communication port may implement a bus protocol, a network protocol, or some other communication protocol. In another example, instead of receiving **(504)** second set of deduplication hashes, storage system **402** may request a set of deduplication hashes on a periodic or aperiodic basis, or in response to some triggering event.

Comparing **(506)** first set of deduplication hashes **552** to second set of deduplication hashes **554** to determine a quantity of matching hash values may be implemented by storage system **402** iterating over the hash values of first set of deduplication hashes **552**, and for each given hash value, searching for the given hash value within second set of deduplication hashes, where for each match, a counter is incremented. Further, a matching percentage may be calculated to represent the number of matches as compared to the total number of comparisons, where the matching percentage, as discussed above, may be used to indicate a level of similarity.

For further explanation, FIG. 6 sets forth a flow chart illustrating an example method for storage efficiency driven migration according to some embodiments of the present disclosure. The example method depicted in FIG. 6 is similar to the example method depicted in FIG. 4, as the example method depicted in FIG. 6 also includes: determining **(404)** a level of similarity between first data **452** stored on a first storage system **402** and second data **456** stored on a second storage system **454**; determining **(406)**, in dependence upon the level of similarity **458**, that an expected amount of storage space reduction from migrating similar data exceeds a threshold level **460**; and responsive to determining **(406)** that the expected amount of storage space reduction exceeds the threshold level, initiating **(408)** a migration of one or more portions of the first data **452** from the first storage system **402** to the second storage system **454**.

However, the example method depicted in FIG. 6 further specifies determining **(406)** that an expected amount of storage space reduction from migrating similar data exceeds a threshold level to include determining **(602)** one or more storage space reduction results from an application of one or more functions for storage space reduction, where the one or more functions includes one or more of: data deduplication or data compression.

Determining (602) one or more storage space reduction results from an application of one or more functions for storage space reduction may be implemented using different techniques. In one technique for determining (602) the one or more storage space reduction results, where the one or more functions include a data deduplication algorithm as depicted in FIG. 5, storage system 402 may use the matching percentage calculated from comparing (506) first set of deduplication hashes 552 to second set of deduplication hashes 554. In this example, the one or more storage space reduction results may be based on the matching percentage and the corresponding quantity of storage space represented by the matching hashes. Using another technique for determining (602) the one or more storage space reduction results, where the one or more functions include data compression, the one or more storage space reduction results may be based on expected data compression ratios for to a particular data compression algorithm and a data type. For example, a table may store an expected data compression ratio for a particular data compression algorithm applied to a particular data type, and based on a quantity of data of that particular data type, and the expected data compression ratio, an expected compressed size for the quantity of data may be determined. A similar calculation may be performed for each data quantity, and respective data type, where the data quantity, and respective data type, is specified to be a combined data quantity for that data type from among both first data 452 and second data 456.

For further explanation, FIG. 7 sets forth a flow chart illustrating an example method for storage efficiency driven migration according to some embodiments of the present disclosure. The example method depicted in FIG. 7 is similar to the example method depicted in FIG. 4, as the example method depicted in FIG. 7 also includes: determining (404) a level of similarity between first data 452 stored on a first storage system 402 and second data 456 stored on a second storage system 454; determining (406), in dependence upon the level of similarity 458, that an expected amount of storage space reduction from migrating similar data exceeds a threshold level 460; and responsive to determining (406) that the expected amount of storage space reduction exceeds the threshold level, initiating (408) a migration of one or more portions of the first data 452 from the first storage system 402 to the second storage system 454.

However, the example method depicted in FIG. 6 further specifies initiating (408) the migration of the one or more portions of first data 452 from first storage system 402 to second storage system 454 to include generating (702) a notification for authorization to migrate the one or more portions of first data 452 from first storage system 402 to second storage system 454, where initiating (408) the migration is in response to receiving an authorization from a user.

Generating (702) a notification for authorization to migrate the one or more portions of first data 452 may be implemented by generating an alert for a data management user interface, such as a management console, where the alert includes information describing one or more of: expected data storage usage reductions expected from migrating the one or more portions of first data 452 from first storage system 402 to second storage system 454, a size of the one or more portions of first data 452, one or more workloads associated with the one or more portions of first data 452, expected time needed to perform the migration, or expected computational resources to perform the migration. Given an alert, a user may indicate, for example by selecting a user interface element to approve the proposed migration described in an alert, that the migration is approved or

rejected. In response to the user selection, storage system 402 may cancel the migration, or initiate the (408) the proposed migration.

For further explanation, FIG. 8 sets forth a flow chart illustrating an example method for storage efficiency driven migration according to some embodiments of the present disclosure. The example method depicted in FIG. 8 is similar to the example method depicted in FIG. 4, as the example method depicted in FIG. 8 also includes: determining (404) a level of similarity between first data 452 stored on a first storage system 402 and second data 456 stored on a second storage system 454; determining (406), in dependence upon the level of similarity 458, that an expected amount of storage space reduction from migrating similar data exceeds a threshold level 460; and responsive to determining (406) that the expected amount of storage space reduction exceeds the threshold level, initiating (408) a migration of one or more portions of the first data 452 from the first storage system 402 to the second storage system 454.

However, the example method in FIG. 8 further includes determining (802) a migration direction to be from first storage system 402 to second storage system 454 based on one or more of: relative workload levels at the first storage system and the second storage system, relative storage capacity availability at the first storage system and the second storage system, a user indication, or relative storage capacity improvements dependent upon whether data is migrated from the first storage system to the second storage system or data is migrated from the second storage system to the first storage system. Determining (802) a migration direction may be implemented by, for each basis, determining a better direction and a corresponding weighted value. For example, if a relative workload is lighter on first storage system 402 than on storage system 454, then this may result in a weighted value in favor of moving in the direction of first storage system 402. Similarly, if a relative storage capacity is lower on storage system 402 than on storage system 454, then this may result in a weighted value in favor of moving in the direction of second storage system 454, where the weighted value may be further adjusted depending on the disparity in relative available storage capacities. Such a calculation may be applied for each basis, where a summation of each calculation may indicate a direction in which to migrate data. In some examples, a user may be provided with these calculations to generate a recommendation, and such a recommendation may be overridden by a user indication.

For further explanation, FIG. 9 sets forth a flow chart illustrating an example method for storage efficiency driven migration according to some embodiments of the present disclosure. The example method depicted in FIG. 9 is similar to the example method depicted in FIG. 4, as the example method depicted in FIG. 9 also includes: determining (404) a level of similarity between first data 452 stored on a first storage system 402 and second data 456 stored on a second storage system 454; determining (406), in dependence upon the level of similarity 458, that an expected amount of storage space reduction from migrating similar data exceeds a threshold level 460; and responsive to determining (406) that the expected amount of storage space reduction exceeds the threshold level, initiating (408) a migration of one or more portions of the first data 452 from the first storage system 402 to the second storage system 454.

However, the example method depicted in FIG. 9 further specifies initiating (408) the migration of the one or more portions of first data 452 from first storage system 402 to second storage system 454 to include determining (902) that

45

the one or more portions of first data **452** fit within second storage system **454**, where initiating (**408**) the migration of the one or more portions of first data **452** is dependent upon determining (**902**) that the one or more portions of first data **452** fit within second storage system **454**.

Determining (**902**) that the one or more portions of first data **452** fit within second storage system **454** may be implemented by identifying available storage resources, determining an amount of storage capacity on the storage resources, identifying one or more workload types executing on second storage system **454**, and determining an amount of system resources consumed in order to determine whether or not the one or more portions of first data **454** fit within second storage system **454**.

Further, in some examples, determining whether or not data to be migrated from a source storage system will fit within a target storage system may be based on the amount of storage occupied by the migrated data after one or more data reduction techniques have been applied. In this case, data migrated may be data that is deduplicated, or compressed, prior to being migrated, which has the added advantage of using fewer network resources than migrating the data as it is stored on the source storage system.

Identifying available storage resources may include identifying one or more types of system resources in a storage array. The one or more types of system resources can include, for example, computing resources such as CPUs, storage resources such as SSDs, memory resources such as RAM, write caching resources such as a write buffer device, and many others. Readers will appreciate that by identifying one or more types of system resources in a storage array, the storage array may be viewed and even graphically depicted as a multidimensional dimensional object, where each axis corresponds to a particular type of resource.

Determining an amount of storage capacity may include determining, for the one or more types of system resources on a storage array, an amount of availability of the resource type. Determining an amount of availability for the one or more types of system resources on the storage array may be carried out by determining the amount of storage capacity that may be provided by the storage resources such as SSDs in the storage array, by determining the number of IOPS that may be provided by the storage resources such as SSDs in the storage array, by determining an amount of data per unit of time that may be written to or read from by the storage resources such as SSDs in the storage array, and so on.

Identifying one or more workload types executing on second storage system **454** may include identifying one or more workload types executing on second storage system **454**. The one or more workload types may be specified in general terms such as, for example, a database application, a virtual desktop infrastructure, and many others. The workload types, to the extent that they are identifiable, may even be characterized at a finer level of granularity such as, for example, an Oracle™ database, an IBM™ DB2 database, and so on. Identifying one or more workload types executing on a storage system may be carried out, for example, by determining an I/O pattern for the storage array and comparing the I/O pattern to I/O patterns exhibited by known workload types, by examining data stored on the storage array to identify metadata that is indicative of a particular workload type, by identifying characteristics (e.g., block size) of data to identify data characteristics that are indicative of a particular workload type, and so on.

Determining an amount of system resources consumed may include determining, for each of the one or more workload supported by the storage system, an amount of

46

system resources consumed by each instance of the workload type. Determining an amount of system resources consumed by each instance of a particular workload type may be carried out, for example, by dividing the total amount of resources consumed by all instances of a particular workload type by the number of instances of the workload type. Consider an example where the workload type is a virtual machine. In such an example, determining an amount of system resources consumed by each instance of a virtual machine may be carried out, for example, by dividing the total amount of resources consumed by all virtual machines by the number of virtual machines supported by the storage array.

In this way, using this information, storage system **402** may determine whether or not the one or more portions of first data **454** fit within second storage system **454** according to available resources, workload types, and amounts of storage resources expected to be consumed.

For further explanation, FIG. **10** sets forth a flow chart illustrating an example method for storage efficiency driven migration according to some embodiments of the present disclosure. Although depicted in less detail, the data management service **1002** depicted in FIG. **10** may be similar to the storage systems described above with reference to FIGS. **1A-1D**, FIGS. **2A-2G**, FIGS. **3A-3C**, or any combination thereof. Further, the storage system depicted in FIG. **10** may include the same, fewer, or additional components as the storage systems described above.

In this example, data management service **1002** may be implemented within a cloud services provider, such as cloud services provider **302** depicted in FIG. **3A**, in communication with multiple different storage systems—similar to the communication network depicted between cloud service provider **302** and storage system **306** depicted in FIG. **3A**. In other examples, data management service **1002** may be implemented within one of the multiple storage systems implementing storage efficiency driven migration.

The example method depicted in FIG. **10** includes receiving (**1004**), from first storage system **1052**, first set of deduplication hashes **1056** based on data **1054** stored on second storage system **1052**. Receiving (**1004**) first set of deduplication hashes **1056** may be implemented by data management service **1002** receiving, at a communication port, a message from first storage system **1052**, where the communication port may implement a bus protocol, a network protocol, or some other communication protocol. In another example, instead of receiving (**1004**) first set of deduplication hashes, data management service **1002** may request a set of deduplication hashes on a periodic or aperiodic basis, or in response to some triggering event.

The example method depicted in FIG. **10** also includes receiving (**1006**), from second storage system **1062**, second set of deduplication hashes **1066** based on data **1064** stored on second storage system **1062**. Receiving (**1006**) second set of deduplication hashes **1066** may be implemented similar to receiving (**1004**) first set of deduplication hashes **1056**.

The example method depicted in FIG. **10** also includes initiating (**1008**), in dependence upon a comparison of one or more hash values from first set of deduplication hashes **1056** to one or more hash values from second set of deduplication hashes **1066**, a migration of data from first storage system **1052** to second storage system **1062**. Initiating (**1008**) the migration of data from first storage system **1052** to second storage system **1062** may be implemented similar to the techniques described above with reference to

FIG. 4 and initiating (408) a migration of one or more portions of first data 452 from first storage system 402 to second storage system 454.

Example embodiments are described largely in the context of a fully functional computer system implementing a storage system. Readers of skill in the art will recognize, however, that the present disclosure also may be embodied in a computer program product disposed upon computer readable storage media for use with any suitable data processing system. Such computer readable storage media may be any storage medium for machine-readable information, including magnetic media, optical media, or other suitable media. Examples of such media include magnetic disks in hard drives or diskettes, compact disks for optical drives, magnetic tape, and others as will occur to those of skill in the art. Persons skilled in the art will immediately recognize that any computer system having suitable programming means will be capable of executing the steps of the method as embodied in a computer program product. Persons skilled in the art will recognize also that, although some of the example embodiments described in this specification are oriented to software installed and executing on computer hardware, nevertheless, alternative embodiments implemented as firmware or as hardware are well within the scope of the present disclosure.

Embodiments can include a system, a method, and/or a computer program product. The computer program product may include a computer readable storage medium (or media) having computer readable program instructions stored thereon for causing a processor to carry out aspects of the present disclosure.

The computer readable storage medium can be a tangible device that can retain and store instructions for use by an instruction execution device. The computer readable storage medium may be, for example, but is not limited to, an electronic storage device, a magnetic storage device, an optical storage device, an electromagnetic storage device, a semiconductor storage device, or any suitable combination of the foregoing. A non-exhaustive list of more specific examples of the computer readable storage medium includes the following: a portable computer diskette, a hard disk, a random access memory (RAM), a read-only memory (ROM), an erasable programmable read-only memory (EPROM or Flash memory), a static random access memory (SRAM), a portable compact disc read-only memory (CD-ROM), a digital versatile disk (DVD), a memory stick, a floppy disk, a mechanically encoded device such as punch-cards or raised structures in a groove having instructions recorded thereon, and any suitable combination of the foregoing. A computer readable storage medium, as used herein, is not to be construed as being transitory signals per se, such as radio waves or other freely propagating electromagnetic waves, electromagnetic waves propagating through a waveguide or other transmission media (e.g., light pulses passing through a fiber-optic cable), or electrical signals transmitted through a wire.

Computer readable program instructions described herein can be downloaded to respective computing/processing devices from a computer readable storage medium or to an external computer or external storage device via a network, for example, the Internet, a local area network, a wide area network and/or a wireless network. The network may comprise copper transmission cables, optical transmission fibers, wireless transmission, routers, firewalls, switches, gateway computers and/or edge servers. A network adapter card or network interface in each computing/processing device receives computer readable program instructions from the

network and forwards the computer readable program instructions for storage in a computer readable storage medium within the respective computing/processing device.

Computer readable program instructions for carrying out operations of the present disclosure may be assembler instructions, instruction-set-architecture (ISA) instructions, machine instructions, machine dependent instructions, microcode, firmware instructions, state-setting data, or either source code or object code written in any combination of one or more programming languages, including an object oriented programming language such as Smalltalk, C++ or the like, and conventional procedural programming languages, such as the "C" programming language or similar programming languages. The computer readable program instructions may execute entirely on the user's computer, partly on the user's computer, as a stand-alone software package, partly on the user's computer and partly on a remote computer or entirely on the remote computer or server. In the latter scenario, the remote computer may be connected to the user's computer through any type of network, including a local area network (LAN) or a wide area network (WAN), or the connection may be made to an external computer (for example, through the Internet using an Internet Service Provider). In some embodiments, electronic circuitry including, for example, programmable logic circuitry, field-programmable gate arrays (FPGA), or programmable logic arrays (PLA) may execute the computer readable program instructions by utilizing state information of the computer readable program instructions to personalize the electronic circuitry, in order to perform aspects of the present disclosure.

Aspects of the present disclosure are described herein with reference to flowchart illustrations and/or block diagrams of methods, apparatus (systems), and computer program products according to some embodiments of the disclosure. It will be understood that each block of the flowchart illustrations and/or block diagrams, and combinations of blocks in the flowchart illustrations and/or block diagrams, can be implemented by computer readable program instructions.

These computer readable program instructions may be provided to a processor of a general purpose computer, special purpose computer, or other programmable data processing apparatus to produce a machine, such that the instructions, which execute via the processor of the computer or other programmable data processing apparatus, create means for implementing the functions/acts specified in the flowchart and/or block diagram block or blocks. These computer readable program instructions may also be stored in a computer readable storage medium that can direct a computer, a programmable data processing apparatus, and/or other devices to function in a particular manner, such that the computer readable storage medium having instructions stored therein comprises an article of manufacture including instructions which implement aspects of the function/act specified in the flowchart and/or block diagram block or blocks.

The computer readable program instructions may also be loaded onto a computer, other programmable data processing apparatus, or other device to cause a series of operational steps to be performed on the computer, other programmable apparatus or other device to produce a computer implemented process, such that the instructions which execute on the computer, other programmable apparatus, or other device implement the functions/acts specified in the flowchart and/or block diagram block or blocks.

The flowchart and block diagrams in the Figures illustrate the architecture, functionality, and operation of possible implementations of systems, methods, and computer program products according to various embodiments of the present disclosure. In this regard, each block in the flowchart or block diagrams may represent a module, segment, or portion of instructions, which comprises one or more executable instructions for implementing the specified logical function(s). In some alternative implementations, the functions noted in the block may occur out of the order noted in the figures. For example, two blocks shown in succession may, in fact, be executed substantially concurrently, or the blocks may sometimes be executed in the reverse order, depending upon the functionality involved. It will also be noted that each block of the block diagrams and/or flowchart illustration, and combinations of blocks in the block diagrams and/or flowchart illustration, can be implemented by special purpose hardware-based systems that perform the specified functions or acts or carry out combinations of special purpose hardware and computer instructions.

Readers will appreciate that the steps described herein may be carried out in a variety of ways and that no particular ordering is required. It will be further understood from the foregoing description that modifications and changes may be made in various embodiments of the present disclosure without departing from its true spirit. The descriptions in this specification are for purposes of illustration only and are not to be construed in a limiting sense. The scope of the present disclosure is limited only by the language of the following claims.

What is claimed is:

1. A method comprising:
 - a first workload, with a first characteristic, storing a first data in a first storage system;
 - a second workload, with a second characteristic, storing a second data in a second storage system;
 - determining, based on the first characteristic and the second characteristic, a level of similarity between the first data and the second data;
 - determining, based on the level of similarity, an expected amount of storage space reduction from migrating the first data and the second data together;
 - in response to determining that the expected amount of storage space reduction exceeds a threshold, migrating the second data from the second storage system to the first storage system; and
 - deduplicating the first data and the second data on the first storage system.
2. The method of claim 1, wherein determining the level of similarity between the first data stored on the first storage system and the second data stored on the second storage system comprises:
 - comparing a first set of deduplication hashes, based on the first data stored on the first storage system, to a second set of deduplication hashes, received from a second storage system, to determine a quantity of matching hash values between hash values from the first set of deduplication hashes and hash values from the second set of deduplication hashes, wherein the second set of deduplication hashes is based on the second data stored on the second storage system and the level of similarity is dependent upon the quantity of matching hash values.
3. The method of claim 1, wherein the level of similarity between the first data stored on the first storage system and

the second data stored on the second storage system is dependent upon a type of the first data and a type of the second data.

4. The method of claim 1, wherein determining, based on the level of similarity, the expected amount of storage space reduction from migrating the first data and the second data together comprises:

- determining one or more storage space reduction results from an application of one or more functions for storage space reduction, wherein the one or more functions include one or more of: data deduplication or data compression.

5. The method of claim 1, wherein the first storage system comprises a primary controller and a secondary controller, and wherein the migrating of the first data is performed by the secondary controller of the first storage system.

6. The method of claim 1:

- wherein migrating, by a processor, the second data from the second storage system to the first storage system further comprises:

- selecting a migration direction from one of either migrating the first data from the first storage system to the second storage system or migrating the second data from the second storage system to the first storage system.

7. The method of claim 6, wherein selecting the migration direction is based on at least one of: relative workload levels at the first storage system and the second storage system, relative storage capacity availability at the first storage system and the second storage system, or relative storage capacity improvements dependent upon whether data is migrated from the first storage system to the second storage system or data is migrated from the second storage system to the first storage system.

8. The method of claim 6, further comprising:

- determining that the second data fits within the first storage system; and
- in response to determining that the second data fits within the second storage system, migrating the second data to the first storage system.

9. The method of claim 1, wherein the level of similarity between the first data stored on the first storage system and the second data stored on the second storage system is dependent upon an identification of the workload that generated the first data and an identification of the workload that generated the second data.

10. An apparatus comprising:

- a memory; and
- a processor, operatively coupled to the memory, configured to:

- determine, based on a first characteristic of a first workload storing a first data in a first storage system and a second characteristic of a second workload storing a second data in a second storage system, a level of similarity between the first data and the second data;
- determine, based on the level of similarity, an expected amount of storage space reduction from migrating the first data and the second data together;

- in response to determining that the expected amount of storage space reduction exceeds a threshold, migrate the second data from the second storage system to the first storage system; and

- deduplicate the first data and the second data on the first storage system.

11. The apparatus of claim 10, wherein to determine the level of similarity between first data stored on the first

51

storage system and the second data stored on the second storage system, the processor is further configured to:

compare a first set of deduplication hashes, based on the first data stored on the first storage system, to a second set of deduplication hashes, received from a second storage system, to determine a quantity of matching hash values between hash values from the first set of deduplication hashes and hash values from the second set of deduplication hashes, wherein the second set of deduplication hashes is based on the second data stored on the second storage system and the level of similarity is dependent upon the quantity of matching hash values.

12. The apparatus of claim 10, wherein the level of similarity between the first data stored on the first storage system and the second data stored on the second storage system is dependent upon a type of the first data and a type of the second data or an identification of the workload that generated the first data and an identification of the workload that generated the second data.

13. The apparatus of claim 10, wherein to determine based on the level of similarity, the expected amount of storage space reduction from migrating the first data and the second data together, the processor is further configured to:

determine one or more storage space reduction results from an application of one or more functions for storage space reduction, wherein the one or more functions include one or more of: data deduplication or data compression.

14. The apparatus of claim 10, wherein the first storage system comprises a primary controller and a secondary controller, and wherein the migrating of the first data is performed by the secondary controller of the first storage system.

15. The apparatus of claim 10, wherein to migrate the second data from the second storage system to the first storage system, the processor is further configured to:

select a migration direction from one of either migrating the first data from the first storage system to the second storage system; and or migrating the second data from the second storage system to the first storage system.

16. The apparatus of claim 15, wherein selecting the migration direction is based on at least one of: relative workload levels at the first storage system and the second storage system, relative storage capacity availability at the first storage system and the second storage system, or relative storage capacity improvements dependent upon whether data is migrated from the first storage system to the second storage system or data is migrated from the second storage system to the first storage system.

52

17. The apparatus of claim 15, wherein the processor is further configured to:

determine that the second data fits within the first storage system; and

in response to determining that the second data fits within the first storage system, migrating the second data to the first storage system.

18. The apparatus of claim 10, wherein the level of similarity between the first data stored on the first storage system and the second data stored on the second storage system is dependent upon an identification of the workload that generated the first data and an identification of the workload that generated the second data.

19. A non-transitory computer readable storage medium storing instructions which, when executed, cause a processor to:

determine, by the processor, based on a first characteristic of a first workload storing a first data in a first storage system and a second characteristic of a second workload storing a second data in a second storage system, a level of similarity between the first data and the second data;

determine, based on the level of similarity, an expected amount of storage space reduction from migrating the first data and the second data together;

in response to determining that the expected amount of storage space reduction exceeds a threshold, migrate the second data from the second storage system to the first storage system; and

deduplicate the first data and the second data on the first storage system.

20. The non-transitory computer readable storage medium of claim 19, wherein the processor is further configured to:

determine the level of similarity between first data stored on the first storage system and the second data stored on the second storage system by:

comparing a first set of deduplication hashes, based on the first data stored on the first storage system, to a second set of deduplication hashes, received from a second storage system, to determine a quantity of matching hash values between hash values from the first set of deduplication hashes and hash values from the second set of deduplication hashes, wherein the second set of deduplication hashes is based on the second data stored on the second storage system and the level of similarity is dependent upon the quantity of matching hash values.

* * * * *